

Sistema de Manutenção Prescritiva para Máquinas Rotativas

Diagnóstico de defeitos por similaridade histórica com *LightGBM* e *KNN*, quantificação de ocorrências e geração de instruções de correção por *Retrieval-Augmented Generation* sobre a base documental da planta, com *guardrail* determinístico de cobertura.

DOCUMENTO	PMX-DOC-001 — Documentação Técnica Consolidada
VERSÃO	1.0 (revisão final)
DATA	23 de agosto de 2026
AUTOR	Lucas Tadeu Monteiro Guedes Fernandes Salomão
CONTEXTO	Processo seletivo SENAI/SC 02198-2026 — Desenvolvedor <i>Full Stack</i> I.A. e Python
CLASSIFICAÇÃO	Documento técnico de projeto — uso interno

Escopo do documento: requisitos, arquitetura nos quatro níveis do modelo C4, modelo de dados, *pipeline* de aprendizado de máquina, camada de recuperação e geração, integração industrial por MQTT, infraestrutura em nuvem e referência de código módulo a módulo. Diagramas vetoriais; fontes Mermaid equivalentes no Apêndice A.

Ficha técnica do documento

Identificação

Campo	Conteúdo
Título	Sistema de Manutenção Prescritiva para Máquinas Rotativas – Documentação Técnica Consolidada
Identificador	PMX-DOC-001
Versão	1.0
Data de emissão	23 de agosto de 2026
Autor	Lucas Tadeu Monteiro Guedes Fernandes Salomão
Contato	lucastadeusalomao@gmail.com
Repositório	Lucas-Salomao/FIESC-processo-seletivo-02198-2026
Versão do <i>software</i>	0.1.0 (pyproject.toml)
Linguagem principal	Python 3.12
Estado	Concluído e operacional (ambiente local e nuvem)

Escopo deste documento

O documento consolida, em fonte única, o que estava distribuído entre `README.md`, `ARCHITECTURE.md`, `ANALYTICS.md`, `DEPLOY.md` e as *docstrings* do código-fonte, acrescentando o material que não existia em forma escrita: especificação de requisitos, catálogo de casos de uso, modelo C4 completo nos quatro níveis, diagramas de atividade de todos os processos, dicionário de dados, especificação da API e referência de código módulo a módulo.

Histórico de revisões

Versão	Data	Autor	Alterações
--------	------	-------	------------

0.1	20/08/2026	L. Salomão	Estrutura inicial; arquitetura e ADRs em <code>ARCHITECTURE.md</code> .
0.2	21/08/2026	L. Salomão	Análise exploratória e fundamentação do <i>dashboard</i> (<code>ANALYTICS.md</code>).
0.3	22/08/2026	L. Salomão	Provisionamento em nuvem como código e guia de implantação (<code>DEPLOY.md</code>).
1.0	23/08/2026	L. Salomão	Documentação técnica consolidada: escopo, requisitos, casos de uso, modelo C4, diagramas de atividade, dicionário de dados, especificação de API, referência de código e bibliografia em ABNT.

Documentos de referência do projeto

Artefato	Conteúdo
<code>README.md</code>	Instruções de instalação, execução e demonstração.
<code>ARCHITECTURE.md</code>	Visão arquitetural resumida e registro de decisões.
<code>ANALYTICS.md</code>	Análise exploratória, consultas SQL e achados estatísticos.
<code>DEPLOY.md</code>	Provisionamento em nuvem, segredos e operação.
<code>.railway/railway.ts</code>	Infraestrutura como código (SDK Railway).
<code>docker-compose.yml</code>	Topologia do ambiente local.
<code>artifacts/metrics.json</code>	Métricas do modelo nos três protocolos de validação.

Sumário

Ficha técnica do documento	i
Lista de figuras	xiii
Lista de tabelas	xv
Lista de siglas e abreviaturas	xvii
1 Introdução	1
1.1 Contexto e motivação	1
1.1.1 O problema proposto	2
1.2 Objetivo do documento	2
1.3 Público-alvo e formas de leitura	3
1.4 Convenções adotadas	3
1.4.1 Tipografia e identificadores	3
1.4.2 Diagramas	4
1.4.3 Números e unidades	5
1.5 Estrutura do documento	5
2 Definições	7
2.1 Domínio: manutenção industrial	7
2.2 Domínio: análise de vibração	8
2.3 Domínio: dados deste projeto	9
2.4 Domínio: aprendizado de máquina	9
2.5 Domínio: recuperação e geração	10
2.6 Domínio: arquitetura e infraestrutura	11

3	Escopo, premissas e requisitos	13
3.1	Declaração do problema	13
3.2	Fronteiras do sistema	14
3.3	Premissas	14
3.4	Restrições	15
3.5	Requisitos funcionais	15
3.6	Requisitos não funcionais	17
3.7	Matriz de rastreabilidade	19
3.8	Critérios de aceite	20
4	Atores e casos de uso	22
4.1	Atores	22
4.2	Diagrama de casos de uso	23
4.3	Especificação dos casos de uso	25
4.3.1	UC-01 – Diagnosticar novo evento	25
4.3.2	UC-02 – Obter instruções de correção	25
4.3.3	UC-03 – Tratar falha não documentada	26
4.3.4	UC-04 – Registrar documento orientativo	27
4.3.5	UC-05 – Consultar cobertura documental	27
4.3.6	UC-06 – Conversar com assistente prescritivo	28
4.3.7	UC-07 – Carregar histórico de eventos	29
4.3.8	UC-08 – Treinar motor de diagnóstico	29
4.3.9	UC-09 – Ingerir telemetria industrial	29
4.3.10	UC-10 – Tratar telemetria inválida	30
4.3.11	UC-11 – Analisar histórico no painel	31
4.3.12	UC-12 – Verificar saúde do sistema	31
5	Arquitetura da solução	32
5.1	Drivers arquiteturais	32
5.2	Visões arquiteturais	33
5.3	C4 nível 1 – Contexto do sistema	33

5.4	C4 nível 2 – Contêineres	34
5.4.1	Plano de aplicação – síncrono, conduzido por pessoas	35
5.4.2	Plano de ingestão industrial – assíncrono, conduzido por máquinas	36
5.4.3	Plano de preparação – em lote, executado deliberadamente	36
5.4.4	Catálogo de contêineres	37
5.5	C4 nível 3 – Componentes da API	37
5.6	C4 nível 4 – Código do motor de diagnóstico	39
5.7	Fluxo do diagnóstico completo	40
5.8	Registro de decisões arquiteturais	42
5.9	Atributos de qualidade e táticas	44
6	Modelo de dados	46
6.1	Diagrama entidade-relacionamento	47
6.2	Dicionário de dados	48
6.2.1	Tabela <code>fault_families</code>	48
6.2.2	Tabela <code>label_map</code>	48
6.2.3	Tabela <code>events</code>	48
6.2.4	Tabelas <code>documents</code> e <code>doc_coverage</code>	50
6.2.5	Tabela <code>diagnoses</code>	50
6.3	Esquema <code>adk</code> – memória de conversa	51
6.4	Base vetorial – coleção <code>manuals</code>	52
6.5	Artefatos de aprendizado de máquina	52
7	Pipeline de dados e canonicalização	54
7.1	Caracterização da fonte de dados	54
7.2	Canonicalização de rótulos	55
7.2.1	O problema	55
7.2.2	A solução: regras ordenadas e auditáveis	55
7.2.3	Auditoria: falhar cedo em vez de classificar em silêncio	56
7.3	Fluxo do ETL	56
7.4	<i>Features</i> geradas na carga	57

7.5	Agregações analíticas	57
8	Motor de diagnóstico: engenharia de <i>features</i>, modelo e validação	58
8.1	Fundamentação física	58
8.2	<i>Features</i> do modelo	59
8.2.1	Grandezas brutas	60
8.2.2	<i>Features</i> derivadas	60
8.2.3	Padronização	60
8.3	Escolha do algoritmo	61
8.3.1	Por que dois modelos e não um	61
8.3.2	Por que <i>gradient boosting</i> sobre árvores	61
8.3.3	Hiperparâmetros e sua justificativa	62
8.3.4	Busca por vizinhos	62
8.4	Protocolo de validação	63
8.4.1	Por que a divisão aleatória seria inválida aqui	63
8.4.2	Por que a divisão temporal também não serve	63
8.4.3	Os três protocolos adotados	64
8.5	Resultados	64
8.5.1	Desempenho por classe	65
8.6	Diagnóstico do vazamento	65
8.6.1	Hipótese inicial	66
8.6.2	Teste: decomposição da variância	66
8.6.3	Ablação: onde a hipótese foi refutada	66
8.6.4	Por que remover não resolve: o vazamento é sistêmico	66
8.6.5	Mitigações no desenho atual	67
8.6.6	Encaminhamentos propostos	67
8.7	Gate de confiança	68
8.8	Análise exploratória que fundamentou o produto	69
8.8.1	A severidade é comparável entre famílias?	69
8.8.2	Os dados obedecem à física dos manuais?	70
8.8.3	O que realmente distingue cada falha?	70

8.9	Limitações do componente de aprendizado	72
9	Recuperação documental, geração e agente	73
9.1	Visão do pipeline	74
9.2	Ingestão documental	74
9.2.1	Extração e <i>chunking</i>	74
9.2.2	OCR: o caso do documento escaneado	75
9.3	<i>Embeddings</i> e base vetorial	75
9.4	O <i>guardrail</i> de cobertura	76
9.4.1	Definição operacional	76
9.4.2	Cobertura inicial e as lacunas deliberadas	77
9.5	Anti-alucinação: cinco barreiras	77
9.5.1	A instrução de sistema	78
9.5.2	Estrutura obrigatória da prescrição	78
9.6	Agente de conversa	79
9.6.1	Fronteira arquitetural	79
9.6.2	Ferramentas	79
9.6.3	Citações determinísticas no agente	80
9.6.4	Degradação graciosa	80
10	Integração industrial por MQTT	82
10.1	Escolha do protocolo	82
10.2	Topologia de tópicos	83
10.3	Fluxo de ingestão	84
10.4	Garantias de entrega e processamento	85
10.5	Segurança do <i>broker</i>	85
10.6	Demonstração e verificação	86
11	Interface do usuário	87
11.1	Arquitetura de informação	87
11.2	Seção de diagnóstico	88

11.3	Painéis analíticos	88
11.3.1	Priorização documental	88
11.3.2	Severidade e validação física	89
11.3.3	Assinaturas	89
11.3.4	Qualidade do modelo	89
11.4	Decisões de visualização	89
11.5	Estado do sistema	90
12	Especificação da API	91
12.1	Catálogo de <i>endpoints</i>	91
12.2	POST /api/v1/diagnose	92
12.2.1	Requisição	92
12.2.2	Resposta	92
12.2.3	Códigos de estado	94
12.3	POST /api/v1/chat	94
12.4	POST /api/v1/documents	95
12.5	GET /api/v1/health	95
12.6	<i>Endpoints</i> de consulta e analítica	96
13	Diagramas de atividade	98
13.1	AT-01 – Carga do histórico corporativo	99
13.2	AT-02 – Canonicalização de um rótulo bruto	100
13.3	AT-03 – Treino do motor de diagnóstico	101
13.4	AT-04 – Indexação de um documento orientativo	102
13.5	AT-05 – Diagnóstico de um novo evento	102
13.6	AT-06 – <i>Guardrail</i> e geração da prescrição	104
13.7	AT-07 – Conversa com o agente	105
13.8	AT-08 – Ingestão de telemetria industrial	105
13.9	AT-09 – Cadastro de documento pela interface	107
13.10	AT-10 – Indexação automática no primeiro <i>boot</i>	108

13.11	AT-11 – Consulta a um painel analítico	109
13.12	AT-12 – Integração contínua	110
13.13	AT-13 – Provisionamento e implantação em nuvem	111
13.14	AT-14 – Ciclo de tratamento de lacuna documental	112
14	Infraestrutura e implantação	114
14.1	Imagem de contêiner	114
14.2	Ambiente local	116
14.3	Ambiente em nuvem	116
14.3.1	Topologia declarada	118
14.3.2	Decisões de infraestrutura	119
14.3.3	Extrato da declaração	119
14.4	Gestão de segredos	120
14.5	Carga inicial em ambiente novo	120
14.6	Verificação pós-implantação	121
14.7	Operação	122
14.8	Custo e topologia reduzida	122
14.9	Adequação à restrição de hardware	122
15	Qualidade, testes e integração contínua	124
15.1	Estratégia de testes	124
15.1.1	Substituições que tornam a suíte offline	125
15.2	Suíte de testes	126
15.2.1	O teste mais importante da suíte	126
15.3	Integração contínua	127
15.4	Padrões de código	127
15.5	Lacunas conhecidas de teste	128
16	Documentação de código	129
16.1	Estrutura do repositório	129
16.2	Grafo de dependências	130

16.3	Pacote src/core – núcleo	132
16.3.1	config.py	132
16.3.2	schemas.py	132
16.3.3	db.py e severity.py	133
16.4	Pacote src/etl	133
16.5	Pacote src/ml	134
16.5.1	features.py	134
16.5.2	train.py	134
16.5.3	diagnose.py	135
16.6	Pacote src/rag	135
16.6.1	chunking.py	135
16.6.2	store.py – onde vive o guardrail	136
16.6.3	service.py	136
16.6.4	bootstrap.py	137
16.7	Pacote src/llm	137
16.7.1	client.py	137
16.7.2	prompts.py	138
16.7.3	agent.py	138
16.7.4	agent_tools.py	139
16.8	Pacote src/api	139
16.8.1	main.py	139
16.8.2	deps.py e analytics.py	140
16.9	Pacote src/ingestion	140
16.10	Pacote src/ui	141
16.11	Scripts de operação	142
16.12	Pontos de extensão	143
17	Riscos e plano de evolução	144
17.1	Matriz de riscos	144
17.2	Plano de evolução	146

17.2.1	Curto prazo — dados e modelo	146
17.2.2	Médio prazo — produto e integração	146
17.2.3	Longo prazo — plataforma	146
17.3	Dívida técnica reconhecida	147
18	Conclusão	148
18.1	Síntese da solução	148
18.2	Contribuições além do exigido	149
18.3	O que este documento afirma e o que não afirma	150
18.4	Consideração final	150
A	Fontes Mermaid dos diagramas	151
A.1	Visão geral do sistema	151
A.2	C4 nível 1 — Contexto (Figura 5.1)	152
A.3	C4 nível 2 — Contêineres (Figuras 5.2 a 5.4)	153
A.4	C4 nível 3 — Componentes da API (Figura 5.5)	153
A.5	C4 nível 4 — Classes (Figura 5.6)	154
A.6	Modelo entidade-relacionamento (Figura 6.1)	156
A.7	Sequência do diagnóstico (Figura 5.7)	156
A.8	Sequência da ingestão MQTT (Figura 10.1)	157
A.9	Sequência da conversa com agente (Figura 9.3)	158
A.10	Pipeline de dados e documentos (Figuras 7.1 e 9.1)	158
A.11	Casos de uso (Figura 4.1)	159
A.12	Diagramas de atividade (Capítulo 13)	160
A.13	Infraestrutura (Figuras 14.1 e 14.2)	164
A.14	Agente e ferramentas (Figura 9.2)	164
A.15	Integração contínua (Figura 13.12)	165
B	Dicionário completo de <i>features</i>	166
B.1	Grandezas brutas (18)	166

B.2	<i>Features</i> derivadas (5)	167
B.3	Colunas descartadas	168
B.4	Poder discriminativo observado	168
C	Taxonomia canônica de falhas	170
C.1	As 17 famílias	170
C.2	Regras de canonicalização	171
C.3	Distribuição de ocorrências e cobertura	172
D	<i>Payloads</i> e respostas de referência	173
D.1	CA-1 – Evento do enunciado (falha documentada)	173
D.2	CA-2 – Falha sem documento	174
D.3	CA-3 – Cobertura após cadastro	174
D.4	CA-5 – <i>Payload</i> inválido e fila de rejeição	174
D.5	Exemplos de perguntas para a conversa	175
E	Configuração e variáveis de ambiente	176
E.1	Variáveis de ambiente	176
E.2	Constantes de código	178
E.3	Arquivo de configuração local	178
F	Guia de operação	180
F.1	Instalação do zero	180
F.2	Operação diária	180
F.3	Diagnóstico de problemas	181
F.4	Compilação deste documento	182
	Referências	184

Lista de figuras

3.1	Fronteiras do escopo	14
4.1	Diagrama de casos de uso	24
5.1	C4 nível 1 – contexto	34
5.2	C4 nível 2 – plano de aplicação	35
5.3	C4 nível 2 – plano de ingestão industrial	36
5.4	C4 nível 2 – plano de preparação	36
5.5	C4 nível 3 – componentes da API	38
5.6	C4 nível 4 – classes do motor de diagnóstico	39
5.7	Sequência do diagnóstico	41
6.1	Modelo entidade-relacionamento	47
7.1	Pipeline do ETL	56
8.1	Resultados nos três protocolos	64
8.2	F1 por classe	65
8.3	Evidência de vazamento por <i>feature</i>	67
9.1	Pipeline de recuperação e geração	74
9.2	Agente de conversa e ferramentas	79
9.3	Sequência da conversa com agente	81
10.1	Sequência da ingestão MQTT	84
11.1	Arquitetura de informação da interface	87
13.1	AT-01 Carga do histórico	99

13.2 AT-02 Canonicalização	100
13.3 AT-03 Treino	101
13.4 AT-04 Indexação documental	102
13.5 AT-05 Diagnóstico de novo evento	103
13.6 AT-06 Guardrail e prescrição	104
13.7 AT-07 Conversa com agente	105
13.8 AT-08 Ingestão MQTT	106
13.9 AT-09 Cadastro de documento	107
13.10AT-10 Bootstrap documental	108
13.11AT-11 Consulta a painel	109
13.12AT-12 Integração contínua	110
13.13AT-13 Provisionamento em nuvem	111
13.14AT-14 Ciclo de lacuna documental	113
14.1 Implantação local	116
14.2 Implantação em nuvem	117
15.1 Distribuição dos testes	124
16.1 Grafo de dependências	131

Lista de tabelas

5.3	Catálogo de contêineres do sistema	37
5.5	Registro de decisões arquiteturais (ADR)	43
7.1	Caracterização do conjunto de dados <code>banner.csv</code>	54
8.1	Assinaturas de defeito segundo a literatura de análise de vibração	59
8.3	<i>Features</i> derivadas e sua motivação física	60
8.6	Hiperparâmetros do classificador	62
8.8	Protocolos de avaliação registrados a cada treino	64
8.10	Métricas detalhadas por protocolo	65
8.12	Decomposição da variância da temperatura (146 campanhas com ≥ 30 eventos)	66
8.18	Velocidade RMS mediana do desbalanceamento por regime de rotação	70
9.3	Cobertura documental inicial	77
9.5	Barreiras anti-alucinação	78
9.7	Ferramentas do agente	80
10.1	Tópicos MQTT	83
10.3	Garantias da ingestão industrial	85
10.5	Resultado esperado da verificação	86
11.2	Codificações visuais e sua justificativa	90
12.1	Catálogo de <i>endpoints</i> da API	91
13.1	Índice dos diagramas de atividade	98
14.1	Decisões de construção da imagem	115

14.3 Serviços em nuvem	118
14.5 Decisões de infraestrutura em nuvem	119
15.1 Substituições de dependência nos testes	125
15.3 Arquivos de teste e o que cada um garante	126
15.5 Etapas do fluxo de integração contínua	127
16.21 Como estender o sistema	143
17.1 Matriz de riscos	145
18.1 Aderência ao enunciado	149

Lista de siglas e abreviaturas

Sigla	Significado
ADK	<i>Agent Development Kit</i> – framework de agentes do Google
ADR	<i>Architecture Decision Record</i> – registro de decisão arquitetural
ANN	<i>Approximate Nearest Neighbors</i> – vizinhos mais próximos aproximados
ANOVA	<i>Analysis of Variance</i> – análise de variância
API	<i>Application Programming Interface</i>
BPMI	<i>Ball Pass Frequency Inner</i> – frequência de passagem na pista interna
BPMO	<i>Ball Pass Frequency Outer</i> – frequência de passagem na pista externa
BSF	<i>Ball Spin Frequency</i> – frequência de rotação do elemento rolante
C4	Modelo de visualização arquitetural em quatro níveis (Contexto, Contêiner, Componente, Código)
CBM	<i>Condition-Based Maintenance</i> – manutenção baseada em condição
CI/CD	<i>Continuous Integration / Continuous Delivery</i>
CLP	Controlador Lógico Programável
CMMS	<i>Computerized Maintenance Management System</i>
CSV	<i>Comma-Separated Values</i>
DLQ	<i>Dead-Letter Queue</i> – fila de mensagens não processáveis
DTO	<i>Data Transfer Object</i>
ER	Entidade-Relacionamento (modelo de dados)
ETL	<i>Extract, Transform, Load</i>
FFT	<i>Fast Fourier Transform</i> – transformada rápida de Fourier
GBDT	<i>Gradient Boosting Decision Trees</i>
GCP	<i>Google Cloud Platform</i>
HNSW	<i>Hierarchical Navigable Small World</i> – índice de busca vetorial
HTTP(S)	<i>HyperText Transfer Protocol (Secure)</i>

Sigla	Significado
IaC	<i>Infrastructure as Code</i> – infraestrutura como código
IIoT	<i>Industrial Internet of Things</i>
ISO	<i>International Organization for Standardization</i>
JSON	<i>JavaScript Object Notation</i>
KNN	<i>k-Nearest Neighbors</i> – <i>k</i> vizinhos mais próximos
LLM	<i>Large Language Model</i> – modelo de linguagem de grande escala
LGBM	<i>LightGBM</i> – implementação de <i>gradient boosting</i>
MQTT	<i>Message Queuing Telemetry Transport</i>
OCR	<i>Optical Character Recognition</i>
OPC UA	<i>Open Platform Communications Unified Architecture</i>
ORM	<i>Object-Relational Mapping</i>
PDF	<i>Portable Document Format</i>
QoS	<i>Quality of Service</i> – nível de garantia de entrega MQTT
RAG	<i>Retrieval-Augmented Generation</i> – geração aumentada por recuperação
REST	<i>Representational State Transfer</i>
RF	Requisito Funcional
RNF	Requisito Não Funcional
RMS	<i>Root Mean Square</i> – valor eficaz
RPM	Rotações Por Minuto
SCADA	<i>Supervisory Control and Data Acquisition</i>
SI	Sistema Internacional de Unidades
SQL	<i>Structured Query Language</i>
TLS	<i>Transport Layer Security</i>
UC	Caso de Uso (<i>Use Case</i>)
UI	<i>User Interface</i> – interface do usuário
UML	<i>Unified Modeling Language</i>
YAML	<i>YAML Ain't Markup Language</i>

Notação e símbolos

Símbolo	Significado
v_{RMS}	Velocidade de vibração eficaz, em mm/s
a_{pico}	Aceleração de pico, em g ($1\text{ }g \approx 9,81\text{ m/s}^2$)
ω	Velocidade angular do eixo, em rad/s
f_{rot}	Frequência de rotação, em Hz (RPM/60)
k	Número de vizinhos considerados pelo KNN ($k = 25$)
κ	Curtose do sinal (momento estatístico de 4. ^a ordem)
C_f	Fator de crista ($a_{\text{pico}}/a_{\text{RMS}}$)
F	Razão entre variância inter e intraclasse (estatística de discriminação)
z	Escore padronizado, $(x - \mu)/\sigma$
ΔE	Diferença de cor perceptual (CIEDE2000)
$p(c \mid x)$	Probabilidade da classe c dado o vetor de <i>features</i> x
$F1_{\text{macro}}$	Média não ponderada do F1 por classe

Introdução

1.1 Contexto e motivação

Máquinas rotativas — motores elétricos, bombas, ventiladores e redutores — concentram a maior parte dos ativos críticos de uma planta industrial. A falha desses equipamentos raramente é instantânea: ela se anuncia por semanas em sinais de vibração, temperatura e corrente, e a leitura correta desses sinais é o que separa uma parada programada de 2 h de uma parada não programada de 2 d.

A literatura de manutenção organiza essa capacidade em uma escala de maturidade crescente, e a distinção entre os degraus é operacional, não apenas terminológica ([ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 1994](#); [Jardine; Lin; Banjevic, 2006](#); [Zonta et al., 2020](#)):

Corretiva

atua depois da falha. Custo máximo, previsibilidade nula.

Preventiva

atua em intervalos fixos, independentemente da condição real. Troca peças boas e ainda assim é surpreendida por falhas precoces.

Preditiva

monitora a condição e estima *quando* a falha ocorrerá ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2018a, 2015](#)). Responde “este rolamento vai falhar”, mas deixa o “e agora?” para o julgamento do técnico.

Prescritiva

acrescenta a resposta que falta: *o que fazer, em que ordem e com base em qual procedimento documentado* ([Ansari; Glawar; Nemeth, 2019](#)).

O salto da manutenção preditiva para a prescritiva é o que este projeto implementa. Um diagnóstico que apenas classifica a falha transfere ao operador o trabalho de localizar, no acervo de manuais da empresa, qual procedimento se aplica — e é exatamente aí que o conhecimento organizacional se perde. A solução documentada aqui fecha esse ciclo: identifica o tipo de

defeito, quantifica quantas vezes ele já ocorreu no histórico, e devolve as instruções de correção extraídas dos documentos técnicos da própria planta, com citação de documento, seção e página.

1.1.1 O problema proposto

O enunciado do processo seletivo estabelece o seguinte problema: dado um novo evento de sensores de vibração em formato JSON, o sistema deve

1. identificar o **tipo de defeito** presente no evento;
2. quantificar as **ocorrências históricas semelhantes** — quantidade, distribuição temporal e frequência;
3. gerar as **instruções de correção** consultando a base documental da empresa;
4. restringir-se **apenas a falhas documentadas**: quando o defeito identificado não possui documento orientativo, informar explicitamente essa condição e sugerir o registro de um novo documento.

O quarto requisito é o mais exigente do ponto de vista de engenharia. Ele determina que o sistema saiba, com certeza verificável, o que ele *não* sabe. Modelos de linguagem são notoriamente ruins nisso: quando não encontram a informação, tendem a produzir uma resposta plausível em vez de admitir a lacuna ([Ji et al., 2023](#)). A resposta arquitetural adotada foi retirar essa decisão do modelo: a cobertura documental é verificada por um *guardrail* determinístico antes de qualquer chamada ao modelo de linguagem, conforme detalhado no Capítulo 9.

1.2 Objetivo do documento

Este documento é a especificação técnica completa da solução. Ele foi escrito para ser suficiente por si só: um engenheiro que nunca viu o repositório deve conseguir, apenas com este texto, compreender o problema, avaliar as decisões de projeto, reproduzir o ambiente, operar o sistema e modificar o código com segurança.

Especificamente, o documento cumpre os seguintes objetivos:

- **Delimitar o escopo** — o que a solução faz, o que deliberadamente não faz, sob quais premissas e restrições (Capítulo 3).
- **Descrever a arquitetura** segundo a norma ISO/IEC/IEEE 42010 ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2011b](#)), usando o modelo C4 ([Brown, 2026](#)) nos quatro níveis e registrando as decisões em formato ADR ([Nygard, 2011](#)) (Capítulo 5).
- **Especificar o comportamento** por meio de casos de uso UML detalhados (Capítulo 4) e de diagramas de atividade de todos os processos do sistema (Capítulo 13).

- **Fundamentar as escolhas de aprendizado de máquina** com referência à literatura, incluindo a justificativa do algoritmo, o protocolo de validação adotado e o diagnóstico honesto de suas limitações (Capítulo 8).
- **Documentar o código** módulo a módulo, com propósito, interface pública, invariantes e pontos de extensão (Capítulo 16).

NOTA

Este documento descreve o sistema *como ele foi construído*, não como se gostaria que ele fosse. Onde os resultados são desfavoráveis — notadamente na generalização do classificador, Seção 8.6 — eles são reportados com o número real, a causa diagnosticada e o encaminhamento proposto. Um documento técnico que esconde o resultado ruim perde a função de apoiar decisão.

1.3 Público-alvo e formas de leitura

Perfil	Interesse principal	Roteiro sugerido
Avaliador técnico	Aderência ao enunciado e qualidade de engenharia	Capítulos 3, 5, 8 e 9
Arquiteto de <i>software</i>	Estrutura, fronteiras e <i>trade-offs</i>	Capítulos 5, 6 e 14
Cientista de dados	Dados, <i>features</i> , modelo e validação	Capítulos 7, 8 e Apêndice B
Desenvolvedor	Como o código está organizado e como estendê-lo	Capítulos 12, 16 e Apêndice E
Engenheiro de manutenção	O que o sistema responde e com que confiança	Capítulos 4, 11 e Seção 8.7
Equipe de operação	Implantar, monitorar e diagnosticar falhas	Capítulos 14, 15 e Apêndice F

1.4 Convenções adotadas

1.4.1 Tipografia e identificadores

- `caminho/arquivo.py` indica arquivo ou diretório do repositório.

- **identificador** indica nome de função, classe, variável, tabela, coluna, tópico MQTT ou variável de ambiente.
- Termos técnicos sem tradução consolidada aparecem em *itálico* na primeira ocorrência.
- Identificadores rastreáveis seguem prefixos fixos: **RF-nn** (requisito funcional), **RNF-nn** (requisito não funcional), **UC-nn** (caso de uso), **ADR-nn** (decisão arquitetural), **AT-nn** (diagrama de atividade) e **RI-nn** (risco).

1.4.2 Diagramas

Todos os diagramas são vetoriais, desenhados nativamente em $\text{\TeX}$ com a biblioteca TikZ, o que garante tipografia uniforme e nitidez em qualquer nível de ampliação. O código-fonte **Mermaid** equivalente de cada diagrama — utilizável diretamente em `README.md`, GitHub, GitLab ou ferramentas de documentação — está reunido no Apêndice A, indexado pelo mesmo número de figura.

As notações empregadas são:

Notação	Onde é usada
Modelo C4 (Brown, 2026)	Contexto, contêineres, componentes e código (Capítulo 5)
UML — casos de uso (OBJECT MANAGEMENT GROUP, 2017)	Capítulo 4
UML — atividades (OBJECT MANAGEMENT GROUP, 2017)	Capítulo 13 e processos ao longo do texto
UML — sequência (OBJECT MANAGEMENT GROUP, 2017)	Fluxos de interação (Capítulos 5, 9 e 10)
UML — classes (OBJECT MANAGEMENT GROUP, 2017)	Nível 4 do C4 (Seção 5.6)
Entidade-Relacionamento	Modelo de dados (Capítulo 6)
Diagrama de implantação	Infraestrutura local e em nuvem (Capítulo 14)

1.4.3 Números e unidades

Todas as grandezas físicas são expressas em unidades do SI, com vírgula como separador decimal e ponto como separador de milhar, conforme a prática brasileira. As colunas redundantes do conjunto de dados original em in/s e °F foram descartadas do vetor de *features* por colinearidade exata com suas contrapartes métricas (Seção 8.2).

Métricas do modelo são reproduzidas exatamente como registradas em `artifacts/metrics.json`, gerado pela execução de treino de 22 de agosto de 2026 sobre 166.796 eventos.

1.5 Estrutura do documento

Cap.	Título	Conteúdo
1	Introdução	Contexto, objetivo, público e convenções.
2	Definições	Glossário de domínio e de tecnologia.
3	Escopo	Problema, fronteiras, premissas, requisitos e rastreabilidade.
4	Casos de uso	Atores, diagrama UML e especificação de doze casos de uso.
5	Arquitetura	Modelo C4 completo, ADRs e atributos de qualidade.
6	Modelo de dados	Diagrama ER, dicionário de dados e esquemas.
7	Pipeline de dados	ETL, canonicalização de rótulos e auditoria.
8	Aprendizado de máquina	<i>Features</i> , escolha do algoritmo, treino, validação e limitações.
9	RAG e linguagem	Ingestão documental, <i>guardrail</i> , <i>prompting</i> e agente.
10	Integração industrial	Tópicos MQTT, garantias de entrega, DLQ e alertas.
11	Interface	Navegação, painéis analíticos e decisões de visualização.
12	API	Especificação de todos os <i>endpoints</i> REST.
13	Atividades	Diagramas de atividade de todos os processos.
14	Infraestrutura	Ambiente local, nuvem, IaC, segredos e custos.
15	Qualidade	Estratégia de testes, suíte, CI/CD e padrões de código.
16	Documentação de código	Referência de todos os módulos.
17	Riscos e evolução	Matriz de riscos e <i>roadmap</i> técnico.

18

Conclusão

Síntese e aderência ao enunciado.

Definições

Este capítulo fixa o vocabulário do documento. A precisão aqui não é formalidade: vários termos — “família”, “sessão”, “documentado” — têm significado técnico específico neste sistema, e confundi-los altera a interpretação dos resultados apresentados nos Capítulos 8 e 9.

2.1 Domínio: manutenção industrial

Manutenção corretiva

Intervenção realizada após a ocorrência da falha, para restabelecer a função do item ([ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 1994](#)).

Manutenção preventiva

Intervenção realizada em intervalos predeterminados ou segundo critérios prescritos, com o objetivo de reduzir a probabilidade de falha, independentemente da condição observada ([ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 1994](#)).

Manutenção preditiva / baseada em condição (CBM)

Intervenção determinada pelo acompanhamento de parâmetros da condição real do equipamento — vibração, temperatura, análise de óleo — com estimativa do instante provável da falha ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2018a](#); [Jardine; Lin; Banjevic, 2006](#)).

Manutenção prescritiva

Extensão da preditiva que, além de detectar e prognosticar, recomenda a ação corretiva específica, ordenada e fundamentada, integrando conhecimento documental e histórico operacional ([Ansari; Glawar; Nemeth, 2019](#)). É o alvo deste projeto.

Ativo rotativo

Equipamento cujo princípio de funcionamento envolve rotação de eixo — motor elétrico, bomba, ventilador, redutor. Concentra as falhas tratadas neste sistema.

Bancada de ensaio

Conjunto experimental no qual falhas são **induzidas** de forma controlada para gerar dados

rotulados. O conjunto de dados deste projeto vem de uma bancada, o que tem consequências metodológicas discutidas na Seção 8.6.

2.2 Domínio: análise de vibração

Velocidade RMS (v_{RMS})

Valor eficaz da velocidade de vibração, em mm/s. É o indicador normativo de severidade global para máquinas rotativas na faixa de 10 Hz a 1.000 Hz ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1995, 2016](#)).

Aceleração de pico

Maior valor absoluto de aceleração na janela de medição, em g . Sensível a impactos, típicos de defeitos localizados em rolamentos.

Curtose (κ)

Momento estatístico de quarta ordem normalizado. Um sinal gaussiano tem $\kappa \approx 3$; valores superiores indicam presença de impulsos. Em teoria, é um dos melhores indicadores precoces de defeito de rolamento ([Randall; Antoni, 2011](#)); neste conjunto de dados, não discrimina (Seção 8.8.3).

Fator de crista (C_f)

Razão entre aceleração de pico e aceleração RMS. Cresce no início de um defeito localizado e volta a cair quando o defeito se generaliza.

Ordem de rotação

Frequência expressa como múltiplo da frequência de rotação do eixo: $\text{ordem} = f / f_{\text{rot}}$, com $f_{\text{rot}} = \text{RPM} / 60$. Os manuais de manutenção descrevem assinaturas em ordens ($1\times$, $2\times$), não em Hz absoluto — razão pela qual a conversão para ordem é uma das *features* derivadas do modelo (Seção 8.2).

BPFO, BPFI, BSF

Frequências características de defeito em rolamentos — pista externa, pista interna e elemento rolante, respectivamente. São múltiplos não inteiros da rotação, determinados pela geometria do rolamento ([Randall; Antoni, 2011](#)). Sua detecção exige o espectro completo, ausente deste conjunto de dados.

Zonas ISO A/B/C/D

Faixas de severidade da velocidade RMS definidas pela norma ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1995](#)): A (máquina recém-comissionada), B (operação contínua aceitável), C (insatisfatória — planejar intervenção) e D (inaceitável — risco de dano). Os limites dependem da classe da máquina (I a IV), que é função da potência e da rigidez da fundação.

Severidade relativa

Razão entre a velocidade RMS medida e a linha de base da própria máquina no mesmo

regime de rotação. É a leitura mais defensável em uma bancada com falhas induzidas, onde a zona absoluta perde significado (Seção 8.8.1).

2.3 Domínio: dados deste projeto

Evento

Registro único de telemetria: 18 grandezas de sensor, identificador, instante de criação e, opcionalmente, a anotação de falha do operador. É a unidade atômica de dado do sistema, definida uma única vez em `SensorEvent` (`src/core/schemas.py`).

Rótulo bruto (`raw_fault`)

Texto da coluna `fault` do conjunto de dados original, tal como digitado. Contém 151 valores distintos, incluindo erros de digitação reais (`ddesbalanceado`, `cockecocked_adxl_0`, `mortor_desligado`) e sufixos de variação experimental (`_2`, `_carga`, `_pos_2`, `new_`).

Família canônica

Classe de falha ou estado operacional obtida pela canonicalização auditável dos rótulos brutos. São **17 famílias**: 12 de falha e 5 de estado. É a única taxonomia usada pelo modelo, pela API e pela base documental (Apêndice C).

Sessão / campanha de coleta

Conjunto de leituras registradas sob a mesma configuração experimental. Neste conjunto de dados, o *rótulo bruto* identifica a sessão: `rolamento_inner`, `rolamento_inner_2` e `new_rolamento_inner_0` são a mesma família coletada em três campanhas distintas. Essa coincidência entre sessão e rótulo é a causa-raiz do vazamento analisado na Seção 8.6.

Estado operacional

Família cujo `is_fault` é falso — `normal`, `baseline`, `teste`, `acelerando`, `motor_desligado`. Um diagnóstico que aponta estado operacional não aciona prescrição alguma.

Linha de base

Distribuição da velocidade RMS no estado `normal` para um regime de rotação específico. Serve de referência para a severidade relativa.

2.4 Domínio: aprendizado de máquina

Feature

Variável de entrada do modelo. O sistema usa 23: 18 grandezas brutas de sensor e 5 derivadas fisicamente motivadas (Seção 8.2).

Padronização (`StandardScaler`)

Transformação $z = (x - \mu) / \sigma$ aplicada a cada *feature*. É obrigatória para o KNN, cuja distância euclidiana seria dominada pelas grandezas de maior magnitude — neste caso, a rotação, na ordem de 1.000.

KNN

Algoritmo que responde a uma consulta devolvendo os k registros mais próximos no espaço de *features* (Cover; Hart, 1967). Neste sistema, $k = 25$ e o índice cobre o histórico completo. É o componente que atende literalmente o requisito de “ocorrências históricas semelhantes”.

LightGBM

Implementação de *gradient boosting* sobre árvores de decisão, com amostragem por gradiente unilateral e agrupamento de *features* exclusivas (Ke et al., 2017). Fornece a classe prevista e sua probabilidade.

Concordância dos vizinhos

Fração dos k vizinhos recuperados que pertencem à família prevista pelo classificador. É uma segunda evidência, independente da probabilidade, sobre a confiabilidade do diagnóstico.

Holdout por sessão

Protocolo de validação em que sessões inteiras de coleta são separadas para teste, nunca leituras individuais. Mede a pergunta de produção — “o modelo reconhece a falha em uma campanha nova?” — e é a prática recomendada para dados com estrutura de agrupamento (Roberts et al., 2017).

Vazamento de dados (*data leakage*)

Situação em que informação indisponível no momento da predição real influencia o treino, inflando artificialmente a métrica (Kaufman et al., 2012; Kapoor; Narayanan, 2023). Neste projeto, o vazamento é de *grupo*: a identidade da campanha de coleta está impressa em quase todo o vetor de *features*.

F1 macro

Média não ponderada do F1 de cada classe. Trata todas as classes como igualmente importantes, o que é adequado em conjuntos desbalanceados (Sokolova; Lapalme, 2009).

2.5 Domínio: recuperação e geração

Embedding

Representação vetorial densa de um texto, na qual proximidade geométrica aproxima similaridade semântica. Este sistema usa *gemini-embedding-2* com dimensionalidade 768.

Chunk

Fragmento de documento indexado individualmente (aproximadamente 1.200 caracteres, com sobreposição de 150). Carrega os metadados de rastreo — documento, seção, página, família — que se tornam as citações exibidas ao usuário.

Base vetorial

Banco especializado em busca por similaridade. Aqui, ChromaDB em modo persistente, com índice HNSW (Malkov; Yashunin, 2020) e métrica de cosseno.

RAG (*Retrieval-Augmented Generation*)

Padrão em que o modelo de linguagem recebe, no *prompt*, trechos recuperados de uma base externa e responde restrito a eles (Lewis et al., 2020). Substitui conhecimento paramétrico por conhecimento verificável.

Contexto fechado

Regime de *prompting* no qual o modelo é instruído a responder **exclusivamente** com base nos trechos fornecidos e a declarar explicitamente quando eles não cobrem a pergunta.

Guardrail de cobertura

Verificação determinística, anterior a qualquer chamada ao modelo, de que a família de falha possui *chunks* indexados. Se não possui, o modelo não é invocado: o sistema devolve o aviso de “não documentado” e a sugestão de registro. É a implementação do requisito RF-04.

Citação determinística

Referência a documento, seção e página obtida dos *metadados* do trecho recuperado, e não do texto gerado pelo modelo. Torna a citação alucinada estruturalmente impossível.

Alucinação

Geração de conteúdo fluente e plausível, mas não sustentado pela fonte (Ji et al., 2023). O Capítulo 9 descreve as cinco barreiras adotadas contra ela.

Agente

Modelo de linguagem com autonomia para escolher e invocar ferramentas em vários passos até responder. Neste sistema, restringe-se ao *endpoint* de conversa; o caminho crítico de diagnóstico permanece determinístico (ADR-11).

Ferramenta (*tool*)

Função Python exposta ao agente, com assinatura e *docstring* que servem de contrato. São quatro: consulta ao histórico, busca documental, diagnóstico e cobertura documental.

2.6 Domínio: arquitetura e infraestrutura

Modelo C4

Notação de visualização arquitetural em quatro níveis de zoom — Contexto, Contêiner, Componente e Código — proposta por Simon Brown (Brown, 2026).

ADR (*Architecture Decision Record*)

Registro curto e imutável de uma decisão arquitetural, contendo contexto, decisão, consequências e alternativas rejeitadas (Nygard, 2011).

Contêiner (C4)

Unidade executável ou de armazenamento que hospeda código — neste sistema: API, interface, *worker*, *broker*, banco relacional e base vetorial. Não confundir com contêiner Docker, ainda que aqui coincidam.

Infraestrutura como código (IaC)

Declaração da topologia de nuvem em arquivo versionado, aplicada por ferramenta. Aqui, `.railway/railway.ts`, que é a fonte de verdade da infraestrutura.

DLQ (*Dead-Letter Queue*)

Destino das mensagens que não podem ser processadas, acompanhadas do motivo da rejeição. Garante que nenhum dado inválido seja descartado em silêncio.

QoS 1 (MQTT)

Nível de garantia *at-least-once*: a mensagem é entregue ao menos uma vez, podendo ser duplicada (OASIS, 2019). Exige idempotência no consumidor — implementada por `session.merge` sobre a chave primária do evento.

Volume persistente

Armazenamento cujo conteúdo sobrevive à substituição do contêiner. Necessário para o índice vetorial, que de outro modo seria perdido a cada implantação.

Escopo, premissas e requisitos

Este capítulo delimita o sistema. Ele segue a estrutura recomendada pela norma ISO/IEC/IEEE 29148 ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2018b](#)) para especificação de requisitos: declaração do problema, fronteiras, premissas, restrições, requisitos funcionais e não funcionais, rastreabilidade e critérios de aceite.

3.1 Declaração do problema

Dado um novo evento de sensores de vibração de uma máquina rotativa, em formato JSON, produzir: (i) o tipo de defeito presente; (ii) a quantificação das ocorrências históricas semelhantes — quantidade, distribuição temporal e frequência; e (iii) as instruções de correção fundamentadas na base documental da empresa, **restritas a falhas efetivamente documentadas**. Quando o defeito identificado não possui documento orientativo, o sistema deve informar essa condição e sugerir o registro de um novo documento técnico.

Três características desse enunciado moldaram toda a arquitetura:

1. **A resposta tem três partes de naturezas diferentes.** Classificação (aprendizado supervisionado), agregação histórica (consulta a banco de dados) e prescrição textual (recuperação e geração de linguagem) são três problemas distintos. Tratá-los com um único mecanismo — por exemplo, pedir tudo a um modelo de linguagem — degradaria todos os três.
2. **“Ocorrências semelhantes” é um requisito de similaridade, não de classificação.** A pergunta “quais registros históricos se comportam como este?” é respondida literalmente por uma busca de vizinhos mais próximos ([Cover; Hart, 1967](#)), não pela contagem de registros da classe predita. Daí a presença simultânea de KNN e classificador ([ADR-04](#)).
3. **A restrição a falhas documentadas é uma garantia, não uma instrução.** Se a verificação de cobertura fosse delegada ao modelo de linguagem, o sistema estaria sujeito a alucinação exatamente no ponto em que o enunciado exige certeza ([Ji et al., 2023](#)). A verificação é portanto determinística e anterior à chamada do modelo ([ADR-09](#)).

3.2 Fronteiras do sistema



Figura 3.1 – Fronteiras do sistema: capacidades entregues e exclusões deliberadas com sua justificativa. Cada exclusão decorre de uma limitação do dado disponível ou de uma decisão explícita de manter a decisão final sob controle humano.

3.3 Premissas

ID	Premissa
PR-01	O conjunto de dados <code>documentos/banner.csv</code> (166.796 registros, 30/04/2026 a 16/06/2026) representa o histórico corporativo de eventos de sensor.
PR-02	Os documentos <code>Doc1.pdf</code> a <code>Doc6.pdf</code> representam a base documental orientativa da empresa. Novos procedimentos entram pela interface ou pela API.
PR-03	As anotações de falha do conjunto de dados são confiáveis quanto ao tipo de defeito, ainda que ruidosas quanto à grafia.

PR-04	O projeto de nuvem GCP possui a API Vertex AI habilitada e credenciais válidas para os modelos <code>gemini-3.6-flash</code> e <code>gemini-embedding-2</code> .
PR-05	A rede da planta permite tráfego MQTT entre gateway de aquisição e <i>broker</i> .
PR-06	Existe um responsável de engenharia de manutenção apto a redigir e cadastrar novos procedimentos quando o sistema apontar lacuna documental.

3.4 Restrições

ID	Restrição	Consequência de projeto
RE-01	Estação de trabalho alvo: 32 GiB de RAM, GPU de 16 GiB	Nenhuma inferência local de modelo de linguagem; os modelos pesados são consumidos como serviço gerenciado. Artefatos de ML somam 74 MiB.
RE-02	Python como linguagem principal	Toda a solução — API, interface, <i>worker</i> e <i>scripts</i> — em Python 3.12.
RE-03	Ausência da forma de onda bruta	Assinaturas espectrais (BPFO/BPFI/BSF) são inalcançáveis; as <i>features</i> derivadas trabalham sobre as métricas pré-agregadas disponíveis.
RE-04	Janela temporal de 47 dias	Impossibilidade de observar tendência de degradação de longo prazo; distribuição temporal reflete cronograma de ensaios.
RE-05	Cada condição experimental coletada em uma única campanha	Confundimento entre sessão e rótulo; teto de generalização imposto pelo desenho experimental (Seção 8.6).
RE-06	Segredos não podem residir no repositório	Credenciais transportadas em variável de ambiente codificada em base64 e materializadas em arquivo temporário com permissão <code>0600</code> .
RE-07	Sistema de arquivos efêmero em nuvem	Índice vetorial exige volume persistente e rotina de <i>bootstrap</i> automático.

3.5 Requisitos funcionais

ID	Requisito	Critério de verificação
RF-01	Identificar o tipo de defeito de um evento JSON, com probabilidade associada	POST /diagnose retorna <code>predicted_fault</code> , <code>probability</code> e <code>is_fault</code> para o evento do enunciado.
RF-02	Quantificar ocorrências históricas semelhantes	Resposta contém <code>count</code> , <code>first_seen</code> , <code>last_seen</code> , <code>freq_per_day</code> , <code>timeline</code> e <code>neighbor_ids</code> .
RF-03	Gerar instruções de correção a partir da base documental	Resposta contém <code>prescription.instructions_md</code> com seções de segurança, diagnóstico, correção, validação e prevenção, e ao menos uma citação.
RF-04	Restringir a prescrição a falhas documentadas	Família sem <code>chunks</code> indexados produz <code>documented=false</code> , <code>prescription=null</code> e <code>suggestion</code> preenchido, sem chamada ao modelo de linguagem.
RF-05	Permitir o cadastro de novo documento orientativo	POST /documents com PDF, título e famílias indexa o conteúdo e a família passa a ser coberta na requisição seguinte.
RF-06	Expor a cobertura documental corrente	GET /documents e GET /health retornam <code>documented_families</code> .
RF-07	Oferecer conversa prescritiva com acesso a histórico e documentos	POST /chat responde usando as quatro ferramentas do agente, com memória por sessão e citações.
RF-08	Ingerir o histórico corporativo com canonicalização auditável	<code>scripts/ingest_data.py</code> carrega 166.796 eventos, mapeia 151 rótulos em 17 famílias e falha se algum rótulo não casar com regra.
RF-09	Treinar o motor de diagnóstico e registrar métricas	<code>python -m src.ml.train</code> gera quatro artefatos e <code>metrics.json</code> com os três protocolos de validação.
RF-10	Ingerir telemetria industrial por MQTT	Evento publicado em <code>sensors/+/telemetry</code> é validado, persistido, diagnosticado e — sendo falha — republicado em <code>sensors/{maquina}/alerts</code> .
RF-11	Não descartar mensagem inválida em silêncio	Payload inválido é publicado em <code>sensors/{maquina}/dlq</code> com o motivo da rejeição.

ID	Requisito	Critério de verificação
RF-12	Disponibilizar painel analítico do histórico	Interface apresenta quatro painéis: visão geral, severidade e física, assinaturas e qualidade do modelo.
RF-13	Consultar eventos recentes	<code>GET /events</code> aceita <code>limit</code> e <code>family</code> e devolve os eventos mais recentes.
RF-14	Fornecer estatísticas agregadas	<code>GET /stats</code> devolve contagem por família e série mensal de falhas.
RF-15	Reportar o estado de saúde do serviço	<code>GET /health</code> indica carga dos artefatos, famílias documentadas e modelos configurados.
RF-16	Simular sensor industrial para demonstração	<code>scripts/simulate_sensor.py</code> publica eventos reais do conjunto de dados, com filtro por família e modo de <code>payload</code> inválido.
RF-17	Auditar cada diagnóstico emitido	Tabela <code>diagnoses</code> registra evento, família prevista, probabilidade, vizinhos e resposta gerada.
RF-18	Indexar a base documental automaticamente em ambiente novo	Com <code>RAG_BOOTSTRAP=true</code> e índice vazio, a API indexa os PDFs em <code>thread</code> própria no <code>startup</code> .
RF-19	Sinalizar diagnóstico incerto	Resposta traz <code>confidence</code> em <code>{alta, media, baixa}</code> , calculado como probabilidade $\times$ concordância dos vizinhos.
RF-20	Processar documentos sem camada de texto	PDF escaneado é transcrito por OCR multimodal antes do <code>chunking</code> .

3.6 Requisitos não funcionais

Cada requisito está associado a uma característica de qualidade da norma ISO/IEC 25010 (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2011a).

ID	Característica	Requisito e como é atendido
RNF-01	Eficiência de recursos	Executar na estação alvo (32 GiB RAM). Artefatos de ML somam 74 MiB; <i>footprint</i> da API em torno de 1 GiB; nenhuma inferência local de LLM.
RNF-02	Desempenho temporal	Diagnóstico sem geração de texto em menos de 1 s; agregações analíticas em cerca de 0,2 s sobre 166.796 eventos, o que dispensa pré-computação.

ID	Característica	Requisito e como é atendido
RNF-03	Reprodutibilidade	Ambiente completo por <code>docker compose up</code> ; sementes fixas (<code>RANDOM_STATE=42</code>) no treino e no <i>holdout</i> por sessão.
RNF-04	Testabilidade	63 testes executáveis sem credenciais e sem rede: o cliente de linguagem é substituído por um <i>fake</i> determinístico e o banco por SQLite em memória.
RNF-05	Segurança — confidencialidade	Nenhum segredo no repositório ou na imagem. Credencial em base64 por variável de ambiente, materializada em arquivo temporário com permissão <code>0600</code> ; <i>broker</i> público com autenticação obrigatória.
RNF-06	Adequação funcional — correção	Toda afirmação factual da resposta é rastreável: citações vêm dos metadados do <i>retrieval</i> ; números vêm de SQL sobre o histórico.
RNF-07	Confiabilidade — tolerância a falhas	Ingestão com QoS 1 e persistência idempotente por chave primária; DLQ para <i>payloads</i> inválidos; persistência de auditoria em modo <i>best-effort</i> que nunca derruba a resposta.
RNF-08	Portabilidade	Imagem única para API, interface e <i>worker</i> , diferenciados apenas pelo comando de inicialização; toda a configuração por variável de ambiente (Wiggins, 2017).
RNF-09	Manutenibilidade — modularidade	Esquema de validação único compartilhado por API, MQTT e ETL; <i>lint</i> obrigatório (ruff , linha de 100 colunas) em porta de CI.
RNF-10	Usabilidade — acessibilidade	Paleta validada para daltonismo: separação $\Delta E \geq 6$ em deuteranopia (CIEDE2000 (Sharma; Wu; Dalal, 2005)); codificação redundante por ícone e rótulo.
RNF-11	Observabilidade	Registro estruturado em todos os serviços; <i>endpoint</i> de saúde com estado de artefatos e cobertura; histórico de implantações e <i>logs</i> acessíveis por CLI.
RNF-12	Disponibilidade	A conversa nunca fica indisponível: sem agente ou credenciais, o <i>endpoint</i> degrada automaticamente para RAG de um passo, com indicação visual do modo em uso.
RNF-13	Eficiência de custo	Topologia reduzível a três serviços (banco, API e interface) sem perda das capacidades exigidas pelo enunciado.
RNF-14	Integridade de dados	Rótulo bruto desconhecido interrompe o ETL com exceção explícita; nada é descartado ou classificado silenciosamente.

3.7 Matriz de rastreabilidade

A matriz liga cada requisito funcional ao componente que o implementa e ao teste que o verifica. É o instrumento que permite alterar o sistema sem perder garantias: ao modificar um componente, a coluna de testes indica o que precisa ser reexecutado.

RF	Implementação	Caso de uso	Teste
RF-01	src/ml/diagnose.py :: DiagnosisEngine.diagnose	UC-01	test_diagnose_fault_documented_flow
RF-02	src/ml/diagnose.py :: _similar_events	UC-01	test_diagnose_fault_documented_flow
RF-03	src/rag/service.py :: prescribe	UC-02	test_prescribe_documented
RF-04	src/rag/store.py :: is_documented	UC-03	test_undocumented_never_calls_llm
RF-05	src/api/main.py :: upload_document	UC-04	test_upload_document_invalid_family
RF-06	src/rag/store.py :: documented_families	UC-05	test_documented_families
RF-07	src/llm/agent.py :: MaintenanceAgent.ask	UC-06	test_chat_usa_agente_quando_disponivel
RF-08	scripts/ingest_data.py + src/etl/canonize.py	UC-07	test_canonize, test_unknown_label_raises
RF-09	src/ml/train.py :: train	UC-08	Fixture <code>env_dirs</code> (treino sintético de ponta a ponta)
RF-10	src/ingestion/mqtt_work	UC-09	Verificação manual documentada na Seção 10.6
RF-11	IngestionWorker._to_dlc	UC-10	scripts/simulate_sensor.py -invalid
RF-12	src/ui/dashboard.py + src/api/analytics.py	UC-11	test_analytics.py (5 testes)
RF-13	src/api/main.py :: events	UC-11	Cobertura por test_api.py
RF-14	src/api/main.py :: stats	UC-11	Cobertura por test_api.py
RF-15	src/api/main.py :: health	UC-12	test_health

RF	Implementação	Caso de uso	Teste
RF-16	scripts/simulate_sensor	UC-09	Verificação manual
RF-17	_persist_diagnosis	UC-01	Coberto indiretamente por <code>test_api.py</code>
RF-18	src/rag/bootstrap.py :: bootstrap_if_empty	UC-04	Verificação em implantação (Seção 14.6)
RF-19	DiagnosisEngine._confidence	UC-01	<code>test_diagnose_normal_state</code>
RF-20	src/rag/chunking.py :: _iter_blocks_ocr	UC-04	<code>test_ocr_fallback_pdf_sem_texto</code>

3.8 Critérios de aceite

O sistema é considerado aderente ao enunciado quando os seis cenários abaixo produzem o resultado esperado. Todos foram verificados; o Apêndice D traz os *payloads* completos.

#	Cenário	Resultado esperado e observado
CA-1	Evento do enunciado submetido a <code>POST /diagnose</code>	Família <code>cocked_rotor</code> com probabilidade de 92 %; 14.275 ocorrências similares com distribuição temporal e frequência diária; instruções de correção citando o <code>Doc6.pdf</code> .
CA-2	Evento de família sem documento (<code>ventoinha</code> , <code>eccentric_rotor</code> , <code>falta_fase</code>)	<code>documented=false</code> , prescrição nula, sugestão de registro preenchida e nenhuma chamada ao modelo de linguagem.
CA-3	Cadastro de PDF cobrindo família antes descoberta	A família passa a constar em <code>documented_families</code> e a prescrição é gerada na requisição seguinte.
CA-4	Evento de estado operacional (<code>normal</code>)	<code>is_fault=false</code> , sem prescrição e sem sugestão — não há ação a prescrever.
CA-5	<i>Payload</i> inválido publicado no tópico de telemetria	Mensagem redirecionada para a DLQ com o motivo; nenhum registro gravado no banco.
CA-6	Suíte de testes e <i>lint</i> em ambiente limpo	<code>ruff check</code> sem apontamentos; 63 testes aprovados sem credenciais de nuvem.

Os critérios de aceite foram escritos a partir do enunciado, não a partir do código já escrito. O cenário CA-2 em particular existe porque o enunciado exige o comportamento de “problema não documentado”: três famílias presentes no histórico (`eccentric_rotor`, `ventoinha`, `falta_fase`) foram deliberadamente mantidas *sem* documento, para que esse caminho seja demonstrável com dados reais em vez de simulado.

Atores e casos de uso

Este capítulo especifica o comportamento observável do sistema em notação UML ([OBJECT MANAGEMENT GROUP, 2017](#)). Ele responde a uma pergunta diferente da do Capítulo 5: não *como* o sistema é construído, mas *o que* ele faz, para quem, sob quais condições e com qual resultado.

4.1 Atores

O sistema tem seis atores: três humanos e três de sistema. A distinção importa porque os atores de sistema operam sem supervisão, o que impõe requisitos de robustez — validação, idempotência e fila de rejeição — que os atores humanos não impõem.

Ator	Tipo	Responsabilidade e interesse
Técnico de manutenção	Humano, primário	Recebe o evento suspeito, submete o diagnóstico e executa o procedimento prescrito. Interessa-lhe a resposta acionável: qual o defeito, com que confiança, quais passos seguir e qual documento consultar.
Engenheiro de manutenção	Humano, primário	Analisa o histórico, prioriza lacunas documentais e cadastra novos procedimentos. Interessa-lhe a visão agregada: quais falhas são frequentes, quais são severas e quais ainda não têm procedimento escrito.
Engenheiro de dados / MLOps	Humano, primário	Executa a carga do histórico, treina o motor de diagnóstico e acompanha a saúde do serviço. Interessa-lhe reprodutibilidade e a honestidade das métricas.
Gateway de aquisição IIoT	Sistema, primário	Publica telemetria no <i>broker</i> MQTT. Pode publicar <i>payload</i> malformatado — o sistema precisa tratar isso sem perder o dado.

SCADA / CMMS	Sistema, secundário	Consome os alertas de falha e as mensagens da fila de rejeição publicados pelo sistema, integrando o diagnóstico ao fluxo de ordens de serviço da planta.
Vertex AI (Gemini)	Sistema, secundário	Fornecer <i>embeddings</i> , geração de texto e transcrição de páginas escaneadas. É um serviço externo: sua indisponibilidade precisa degradar o sistema, não derrubá-lo.

4.2 Diagrama de casos de uso

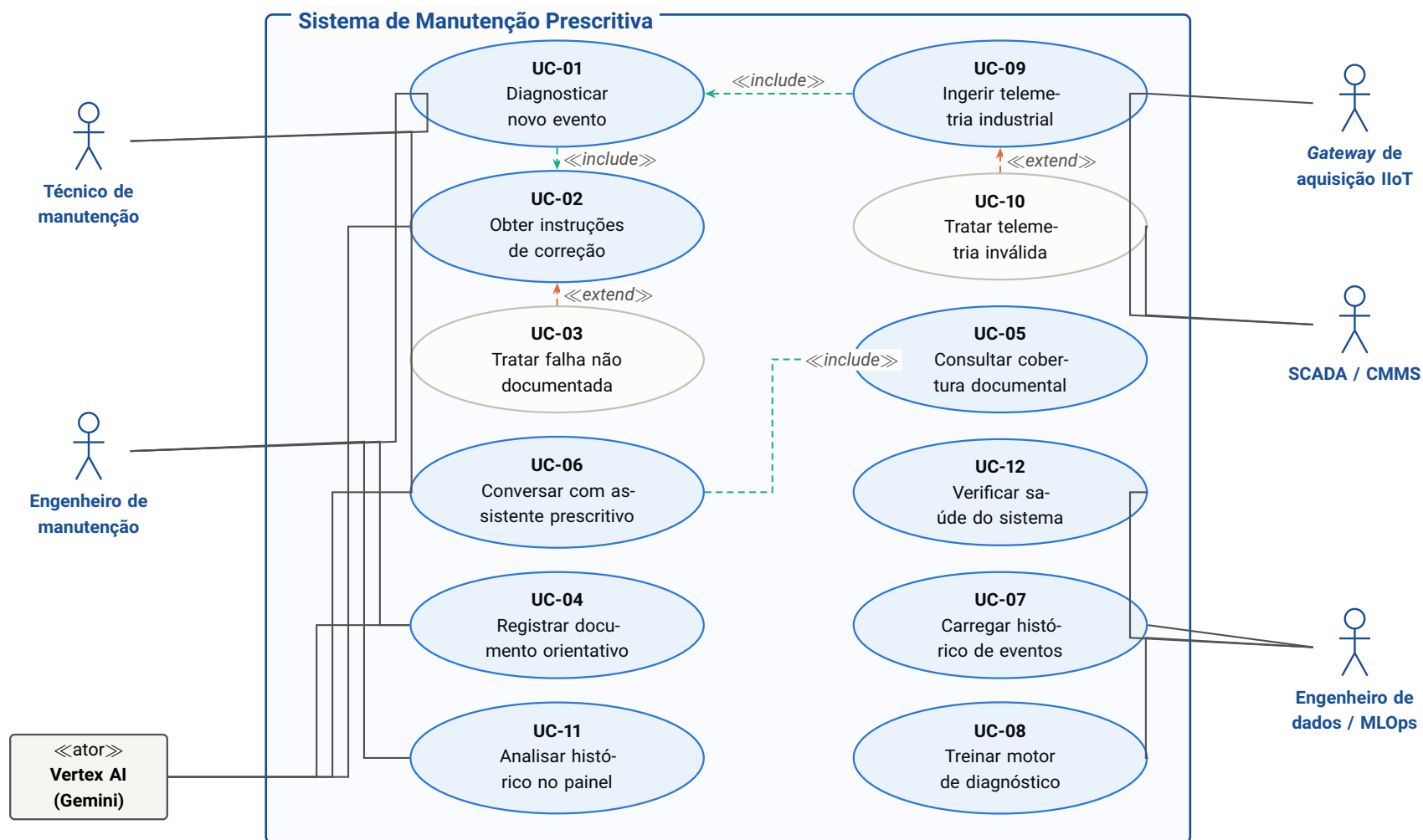


Figura 4.1 – Diagrama de casos de uso do sistema. As elipses em tom claro (**UC-03** e **UC-10**) são extensões condicionais: ocorrem apenas quando a condição de exceção se verifica — falha sem documento e *payload* inválido, respectivamente. Ambas são exigências explícitas do enunciado, não tratamentos de erro incidentais.

Legenda: — associação ator–caso de uso --→ relação «include» (sempre executada) --→ relação «extend» (executada sob condição).

4.3 Especificação dos casos de uso

Cada caso de uso é especificado com ator primário, pré-condições, pós-condições, fluxo principal, fluxos alternativos e rastreabilidade aos requisitos da Seção 3.5.

4.3.1 UC-01 — Diagnosticar novo evento

Campo	Conteúdo
Ator primário	Técnico de manutenção; Engenheiro de manutenção; UC-09 (quando automático)
Requisitos	RF-01, RF-02, RF-17, RF-19
Pré-condições	Artefatos de ML presentes em <code>artifacts/</code> ; histórico carregado no banco.
Pós-condições	Diagnóstico retornado ao solicitante e registrado na tabela <code>diagnoses</code> para auditoria.
Fluxo principal	1. O ator submete o JSON do evento a <code>POST /api/v1/diagnose</code>. 2. O sistema valida o <code>payload</code> contra o esquema <code>SensorEvent</code>. 3. Calcula as 5 <code>features</code> derivadas e padroniza o vetor de 23 dimensões. 4. O classificador devolve a família mais provável e sua probabilidade. 5. O KNN recupera os 25 vizinhos mais próximos e calcula a concordância com a família prevista. 6. O sistema agrega as estatísticas históricas da família — contagem, primeira e última ocorrência, frequência diária e série mensal. 7. Classifica a confiança em <code>alta</code>, <code>media</code> ou <code>baixa</code>. 8. Se a família for de falha, invoca UC-02. 9. Persiste a auditoria e devolve a resposta.
Fluxo alt. A	2a. <code>Payload</code> inválido: retorna HTTP 422 com a lista de campos inconsistentes; nada é persistido.
Fluxo alt. B	8a. Família de estado operacional (<code>is_fault=false</code>): não há prescrição nem sugestão — não existe ação a prescrever.
Fluxo alt. C	9a. Banco indisponível: a auditoria falha em silêncio (<code>rollback</code>) e a resposta é devolvida normalmente. O diagnóstico não depende da gravação.
Exceção	Artefatos ausentes: <code>GET /health</code> reporta <code>artifacts_loaded=false</code> e a interface orienta a executar o treino.

4.3.2 UC-02 — Obter instruções de correção

Campo	Conteúdo
Ator primário	UC-01 (<i><<include>></i>); ator secundário: Vertex AI
Requisitos	RF-03 , RNF-06
Pré-condições	Família de falha identificada; base documental indexada.
Pós-condições	Instruções em Markdown com seções de segurança, diagnóstico, correção, validação e prevenção, acompanhadas das citações de documento, seção e página.
Fluxo principal	1. O sistema consulta o <i>guardrail</i> de cobertura para a família. 2. Sendo documentada, monta a consulta com o nome de exibição da família e os sintomas do evento. 3. Gera o <i>embedding</i> da consulta. 4. Recupera os <i>top-k</i> trechos filtrados pela família. 5. Monta o <i>prompt</i> de contexto fechado com os trechos numerados. 6. Solicita a geração ao modelo com <i>temperature=0.2</i> . 7. Extrai as citações dos <i>metadados</i> dos trechos recuperados — nunca do texto gerado. 8. Devolve prescrição e citações.
Fluxo alt. A	2a. Família não documentada: aciona UC-03 ; o modelo de linguagem não é chamado.
Exceção	Vertex AI indisponível: HTTP 503 com orientação de configuração de credenciais; o diagnóstico das etapas anteriores permanece válido.

4.3.3 UC-03 — Tratar falha não documentada

Campo	Conteúdo
Ator primário	UC-02 (<i><<extend>></i>)
Requisitos	RF-04
Condição de extensão	A família de falha prevista não possui nenhum <i>chunk</i> indexado na base vetorial.
Pós-condições	Resposta com <i>documented=false</i> , <i>prescription=null</i> e <i>suggestion</i> contendo o aviso e a orientação de registro. Nenhuma chamada ao modelo de linguagem é feita.
Fluxo principal	1. O <i>guardrail</i> determina ausência de cobertura. 2. O sistema recupera o nome de exibição da família. 3. Formata a mensagem padrão: informa que não existe documento orientativo cadastrado para aquela falha e sugere registrar o procedimento na aba <i>Documentos</i> . 4. Devolve a resposta e encerra.

Observação	Este é o caminho exigido pelo enunciado. Três famílias do histórico — <code>eccentric_rotor</code> (16.497 ocorrências), <code>ventoinha</code> (12.299) e <code>falta_fase</code> (800) — foram mantidas sem documento justamente para que o comportamento seja demonstrável com dado real.
------------	--

4.3.4 UC-04 – Registrar documento orientativo

Campo	Conteúdo
Ator primário	Engenheiro de manutenção; ator secundário: Vertex AI
Requisitos	RF-05, RF-18, RF-20
Pré-condições	PDF do procedimento disponível; famílias cobertas conhecidas.
Pós-condições	<i>Chunks</i> indexados com metadados de rastreo; registro em <code>documents</code> e <code>doc_coverage</code> ; famílias passam a ser cobertas imediatamente.
Fluxo principal	1. O ator envia PDF, título e lista de famílias. 2. O sistema valida cada família contra a taxonomia canônica. 3. Remove <i>chunks</i> anteriores do mesmo arquivo (idempotência). 4. Extrai o texto página a página, rastreando o título de seção corrente. 5. Acumula <i>chunks</i> de cerca de 1.200 caracteres com sobreposição de 150. 6. Gera os <i>embeddings</i> . 7. Indexa um conjunto de <i>chunks</i> por família coberta. 8. Registra o documento e a cobertura no banco relacional. 9. Devolve a contagem de <i>chunks</i> e as famílias atendidas.
Fluxo alt. A	2a. Família inexistente: HTTP 422 com a lista de famílias válidas.
Fluxo alt. B	4a. PDF sem camada de texto (escaneado): cada página é renderizada em imagem e transcrita por OCR multimodal antes do <i>chunking</i> .
Fluxo alt. C	5a. Nenhum <i>chunk</i> útil extraído: HTTP 422 informando que não houve conteúdo aproveitável.
Fluxo alt. D	8a. Banco indisponível: a indexação vetorial — que é o que determina a cobertura — permanece válida; o registro relacional é revertido.
Variante automática	Em ambiente novo com índice vazio e <code>RAG_BOOTSTRAP=true</code> , a própria API executa este caso de uso em <i>thread</i> separada durante o <i>startup</i> , sem intervenção humana.

4.3.5 UC-05 – Consultar cobertura documental

Campo	Conteúdo
Ator primário	Engenheiro de manutenção; UC-04 e UC-06 (<code><<include>></code>)

Requisitos	RF-06
Pré-condições	Base vetorial acessível.
Pós-condições	Lista de famílias documentadas e sem documento, com os documentos cadastrados e suas datas de ingestão.
Fluxo principal	1. O ator solicita a cobertura. 2. O sistema consulta os metadados de todos os <i>chunks</i> indexados e extrai o conjunto de famílias presentes. 3. Cruza com a taxonomia canônica para obter as famílias de falha sem cobertura. 4. Lista os documentos ativos do banco relacional. 5. Devolve o resultado consolidado.
Observação	A cobertura é sempre derivada do índice, nunca de uma configuração estática. Um documento cadastrado em tempo de execução altera a resposta na consulta seguinte, sem reinício de serviço.

4.3.6 UC-06 – Conversar com assistente prescritivo

Campo	Conteúdo
Ator primário	Técnico de manutenção; ator secundário: Vertex AI
Requisitos	RF-07, RNF-12
Pré-condições	Nenhuma obrigatória — o caso de uso degrada em vez de falhar.
Pós-condições	Resposta em Markdown, com citações quando houver fundamentação documental, e memória de conversa persistida por sessão.
Fluxo principal	1. O ator envia a pergunta com o identificador da sessão. 2. O agente decide qual ferramenta usar. 3. Para perguntas quantitativas, invoca a consulta ao histórico (SQL). 4. Para procedimentos, invoca a busca documental — que aplica o <i>guardrail</i> internamente. 5. Para um JSON colado, invoca o diagnóstico. 6. Para perguntas de cobertura, invoca UC-05 . 7. Compõe a resposta citando os trechos por marcador. 8. O coletor de citações devolve as fontes efetivamente recuperadas.
Fluxo alt. A	4a. Família sem documento: a ferramenta retorna <code>documentado=false</code> com o aviso pronto; o modelo transmite o aviso e sugere o registro, sem descrever procedimento próprio.
Fluxo alt. B	2a. Agente indisponível (sem biblioteca ou sem credenciais): o <i>endpoint</i> degrada automaticamente para RAG de um passo, e a interface exibe o modo em uso.
Exceção	Falha do serviço de linguagem: HTTP 503 com orientação acionável de configuração.

4.3.7 UC-07 – Carregar histórico de eventos

Campo	Conteúdo
Ator primário	Engenheiro de dados / MLOps
Requisitos	RF-08, RNF-14
Pré-condições	Banco acessível; arquivo CSV disponível.
Pós-condições	Tabelas <code>fault_families</code> , <code>label_map</code> e <code>events</code> populadas e consistentes.
Fluxo principal	1. O ator executa <code>scripts/ingest_data.py</code>. 2. O sistema lê o CSV e normaliza os rótulos brutos. 3. Canonicaliza todos os rótulos distintos <i>antes</i> de gravar qualquer linha. 4. Cria as tabelas se necessário. 5. Popula as famílias e o mapa de rótulos. 6. Carrega os eventos de forma idempotente. 7. Imprime o resumo da carga.
Fluxo alt. A	3a. Rótulo sem regra de canonicalização: o processo é interrompido com exceção nomeando o rótulo. Nada é gravado – é preferível falhar cedo a produzir uma base silenciosamente incorreta.

4.3.8 UC-08 – Treinar motor de diagnóstico

Campo	Conteúdo
Ator primário	Engenheiro de dados / MLOps
Requisitos	RF-09, RNF-03
Pré-condições	CSV disponível; taxonomia de rótulos válida.
Pós-condições	Quatro artefatos gravados (<code>scaler</code> , <code>lgbm</code> , <code>knn</code> , <code>index_meta</code>) e <code>metrics.json</code> com os três protocolos de avaliação.
Fluxo principal	1. O ator executa <code>python -m src.ml.train</code>. 2. O sistema carrega e deduplica o conjunto de dados. 3. Constrói a matriz de <i>features</i> (18 brutas + 5 derivadas). 4. Avalia com <i>holdout</i> por sessão de coleta – métrica principal. 5. Avalia com divisão aleatória estratificada – referência otimista. 6. Avalia com <i>holdout</i> por sessão sem a temperatura – teste de ablação. 7. Retreina o modelo final com 100 % dos dados. 8. Indexa o histórico completo no KNN. 9. Grava artefatos e métricas.
Observação	As três avaliações são registradas juntas por decisão metodológica: reportar apenas a divisão aleatória produziria uma promessa de 89 % de acurácia que o sistema não cumpre em produção (Seção 8.6).

4.3.9 UC-09 – Ingerir telemetria industrial

Campo	Conteúdo
Ator primário	Gateway de aquisição IIoT; ator secundário: SCADA / CMMS
Requisitos	RF-10, RF-16, RNF-07
Pré-condições	Broker MQTT acessível; <i>worker</i> conectado e inscrito em <code>sensors/+/telemetry</code> .
Pós-condições	Evento persistido de forma idempotente; diagnóstico executado; alerta publicado quando houver falha.
Fluxo principal	1. O <i>gateway</i> publica o evento com QoS 1. 2. O <i>worker</i> recebe a mensagem e extrai o identificador da máquina do tópico. 3. Valida o <i>payload</i> com o mesmo esquema da API. 4. Canonicaliza a anotação de falha, se presente. 5. Persiste com <code>merge</code> pela chave primária — entrega duplicada não gera linha duplicada. 6. Invoca UC-01 pela API. 7. Sendo falha, publica o alerta em <code>sensors/{maquina}/alerts</code> .
Fluxo alt. A	3a. <i>Payload</i> inválido: aciona UC-10 .
Fluxo alt. B	4a. Rótulo desconhecido: registra advertência em <i>log</i> e persiste o evento sem família — diferente do ETL em lote, aqui perder o dado seria pior que perder o rótulo.
Fluxo alt. C	6a. API indisponível: registra erro e segue consumindo; o evento já está persistido e pode ser rediagnosticado depois.

4.3.10 UC-10 – Tratar telemetria inválida

Campo	Conteúdo
Ator primário	UC-09 (<code><<extend>></code>); ator secundário: SCADA / CMMS
Requisitos	RF-11, RNF-07
Condição de extensão	JSON malformatado ou <i>payload</i> que não satisfaz o esquema de evento.
Pós-condições	Mensagem publicada em <code>sensors/{maquina}/dlq</code> com motivo, <i>payload</i> original truncado e <i>timestamp</i> ; nenhuma linha gravada no banco.
Fluxo principal	1. A validação falha. 2. O sistema compõe o registro de rejeição com o motivo (limitado a 2.000 caracteres) e o <i>payload</i> original (limitado a 5.000). 3. Publica na fila de rejeição com QoS 1. 4. Registra advertência em <i>log</i> .
Justificativa	Descartar em silêncio é a falha de operação mais comum em ingestão industrial: o dado desaparece e ninguém percebe. Com a fila de rejeição, o problema fica visível e o <i>payload</i> recuperável.

4.3.11 UC-11 – Analisar histórico no painel

Campo	Conteúdo
Ator primário	Engenheiro de manutenção
Requisitos	RF-12 , RF-13 , RF-14 , RNF-02 , RNF-10
Pré-condições	Histórico carregado; API disponível.
Pós-condições	Painel renderizado com os quatro conjuntos de análises e a fila de prioridade documental.
Fluxo principal	1. O ator escolhe o painel na navegação lateral. 2. A interface requisita apenas os dados daquele painel. 3. Visão geral: distribuição por família, série temporal e matriz de priorização (frequência × severidade × cobertura). 4. Severidade e física: velocidade RMS estratificada por regime de rotação, com zonas normativas selecionáveis e validação da lei $F = m r \omega^2$. 5. Assinaturas: mapa de escores padronizados por família, ordenado por poder discriminativo. 6. Qualidade do modelo: os três protocolos lado a lado, importância das <i>features</i> e evidência de vazamento por <i>feature</i>.
Fluxo alt. A	2a. Agregação indisponível (banco sem os recursos estatísticos necessários): o painel exibe aviso e mantém os demais gráficos funcionais.

4.3.12 UC-12 – Verificar saúde do sistema

Campo	Conteúdo
Ator primário	Engenheiro de dados / MLOps; orquestrador de contêineres
Requisitos	RF-15 , RNF-11
Pré-condições	Serviço em execução.
Pós-condições	Estado reportado sem efeito colateral.
Fluxo principal	1. O ator consulta <code>GET /api/v1/health</code>. 2. O sistema verifica a presença do artefato do classificador. 3. Lista as famílias documentadas a partir do índice vetorial. 4. Informa os modelos de linguagem e de <i>embedding</i> configurados. 5. Devolve o estado consolidado.
Fluxo alt. A	3a. Índice inacessível ou ainda em <i>bootstrap</i> : devolve lista vazia em vez de erro — o serviço está parcialmente pronto, e o orquestrador não deve reprovar a instância por isso.

Arquitetura da solução

A descrição arquitetural segue a norma ISO/IEC/IEEE 42010 ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2011b](#)): identifica os interessados, os *drivers* que restringem o desenho, apresenta visões complementares do sistema e registra as decisões com suas consequências. A notação de visualização é o modelo C4 ([Brown, 2026](#)), que organiza a arquitetura em quatro níveis de aproximação sucessiva.

5.1 Drivers arquiteturais

Um *driver* arquitetural é uma exigência que, se ignorada, invalida a solução. Foram cinco.

ID	Driver	Consequência arquitetural
DR-01	Garantia verificável de cobertura documental	A verificação é um componente determinístico, posicionado <i>antes</i> da camada de linguagem no fluxo de controle. O modelo generativo nunca decide se existe documento (ADR-09).
DR-02	Resposta composta de três naturezas distintas	Três mecanismos especializados — classificador, busca por similaridade e recuperação com geração — em vez de um mecanismo único (ADR-04).
DR-03	Integração com o chão de fábrica	Ingestão assíncrona por MQTT, com garantias de entrega, idempotência e fila de rejeição; alertas devolvidos à planta pelo mesmo canal (ADR-10).
DR-04	Reprodutibilidade e testabilidade sem nuvem	Cliente de linguagem definido como interface fina, substituível por implementação <i>fake</i> ; toda configuração externalizada em variáveis de ambiente.

DR-05	Restrição de hardware da estação alvo	Nenhuma inferência local de modelo de linguagem; artefatos de ML mantidos compactos; base vetorial embarcada em vez de servidor dedicado (ADR-01 , ADR-03).
--------------	---------------------------------------	---

5.2 Visões arquiteturais

O sistema é descrito por cinco visões complementares, no espírito do modelo 4+1 ([Kruchten, 1995](#)). Cada visão responde a perguntas de um grupo distinto de interessados; nenhuma delas é suficiente isoladamente.

Visão	Notação	Onde está neste documento
Lógica	C4 níveis 1–3, diagrama de classes	Seções 5.3 a 5.6
Processos	Diagramas de sequência e de atividade	Seção 5.7 , Capítulo 13
Desenvolvimento	Estrutura de módulos e dependências	Capítulo 16
Física / implantação	Diagramas de implantação	Capítulo 14
Cenários	Casos de uso	Capítulo 4

5.3 C4 nível 1 – Contexto do sistema

O nível de contexto responde à pergunta mais elementar: quem usa o sistema, com que outros sistemas ele conversa e onde está sua fronteira.

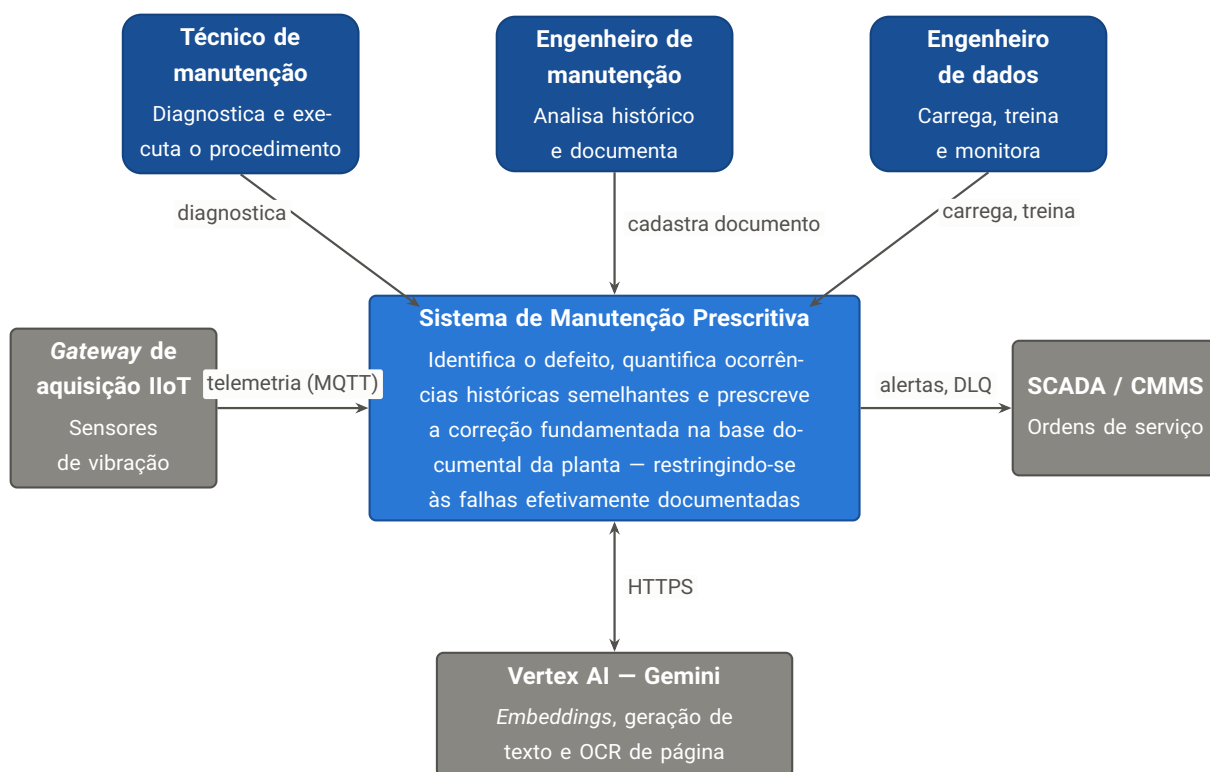


Figura 5.1 – C4 nível 1: contexto do sistema. Três atores humanos com objetivos distintos, um sistema de origem de dados (*gateway IIoT*), um sistema de destino de alertas (*SCADA/CMMS*) e um único serviço externo na dependência crítica de linguagem.

NOTA

A única dependência externa *não substituível em tempo de execução* é o Vertex AI, e ainda assim sua indisponibilidade não derruba o sistema: o diagnóstico e a quantificação histórica continuam funcionando, e a conversa degrada para RAG de um passo. Essa contenção de acoplamento é deliberada (**RNF-12**).

5.4 C4 nível 2 – Contêineres

O nível de contêineres é onde a arquitetura fica interessante — e onde um diagrama único ficaria ilegível. Os contêineres do sistema participam de **três planos de operação** com características muito diferentes: síncrono e conduzido por pessoas; assíncrono e conduzido por máquinas; e em lote, executado deliberadamente. Cada plano recebe uma vista própria, e a Tabela 5.3 consolida o catálogo.

5.4.1 Plano de aplicação – síncrono, conduzido por pessoas

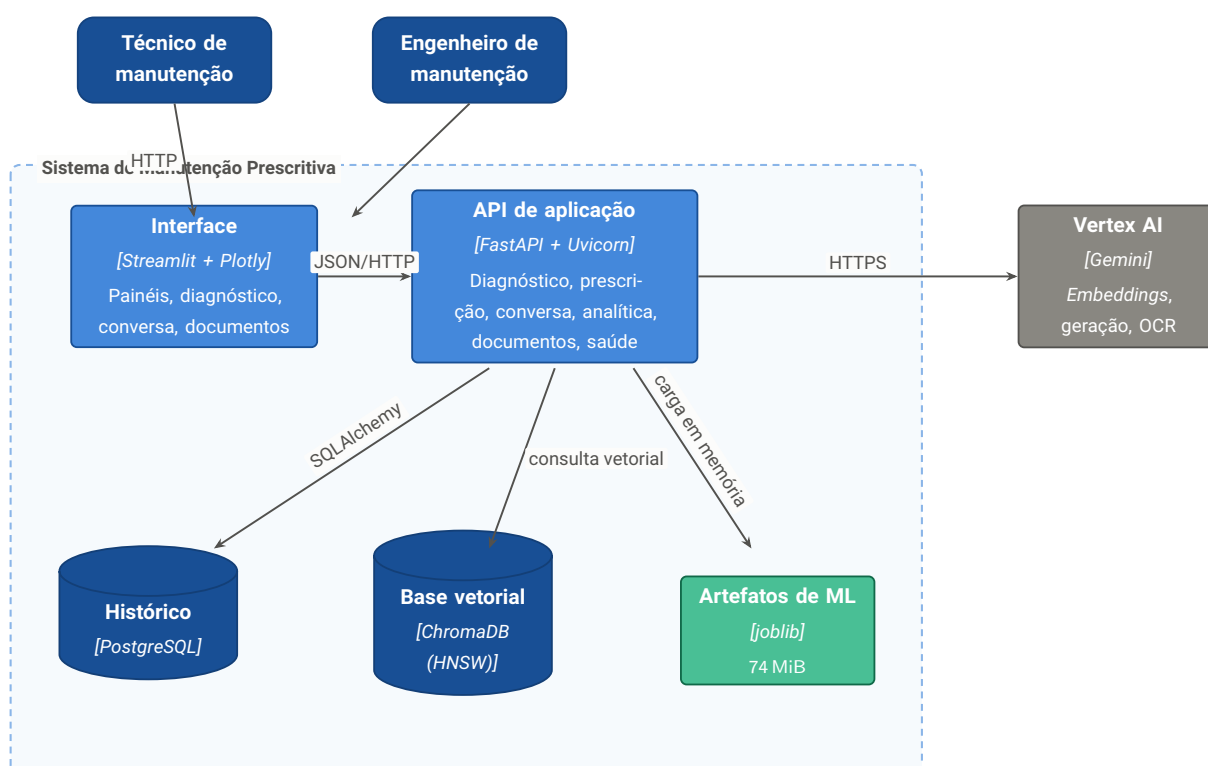


Figura 5.2 – C4 nível 2, plano de aplicação. A interface **não** acessa banco algum: todo dado passa pela API, o que mantém a camada de serviço reutilizável por integradores externos (**ADR-06**).

5.4.2 Plano de ingestão industrial – assíncrono, conduzido por máquinas

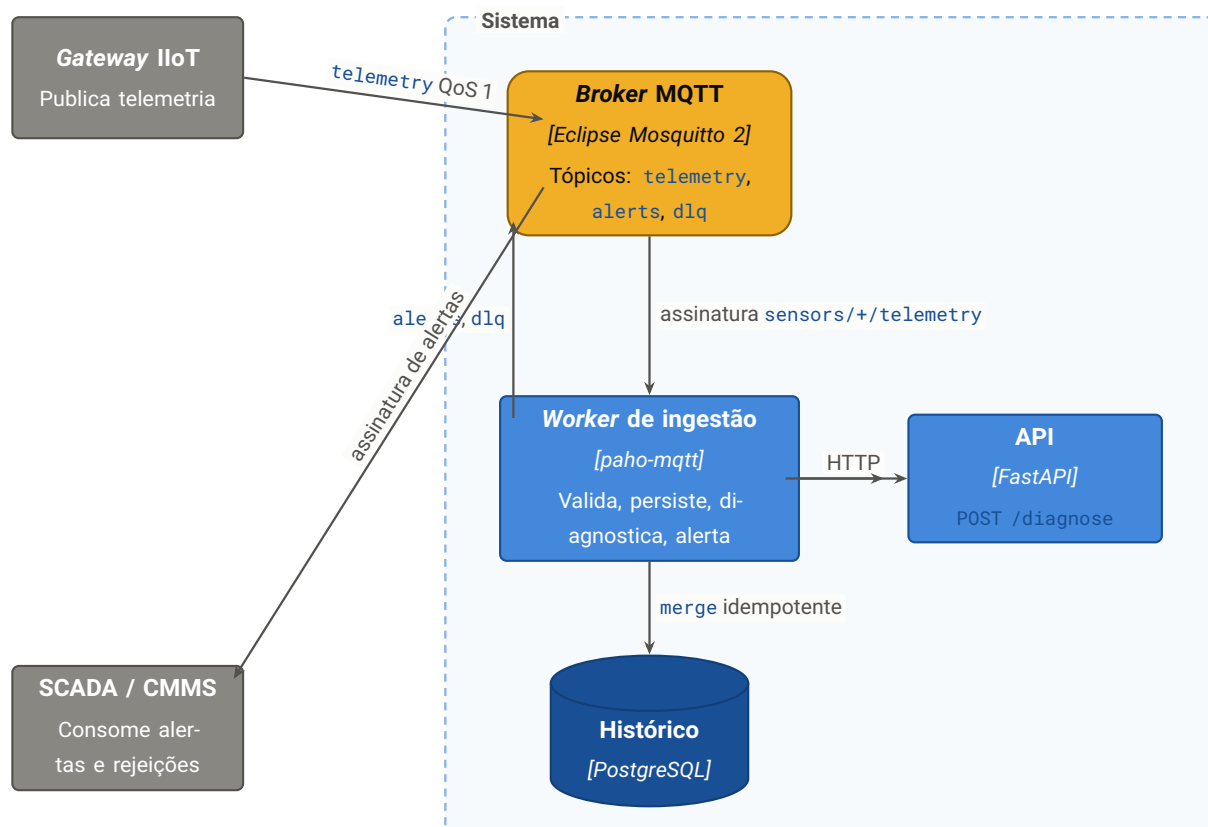


Figura 5.3 – C4 nível 2, plano de ingestão industrial. O ciclo se fecha: a telemetria entra pelo *broker* e o diagnóstico volta à planta pelo mesmo *broker*, em tópico distinto. É assim que um SCADA ou um CMMS consumiria a solução sem qualquer acoplamento ao código do sistema ([ADR-10](#)).

5.4.3 Plano de preparação – em lote, executado deliberadamente

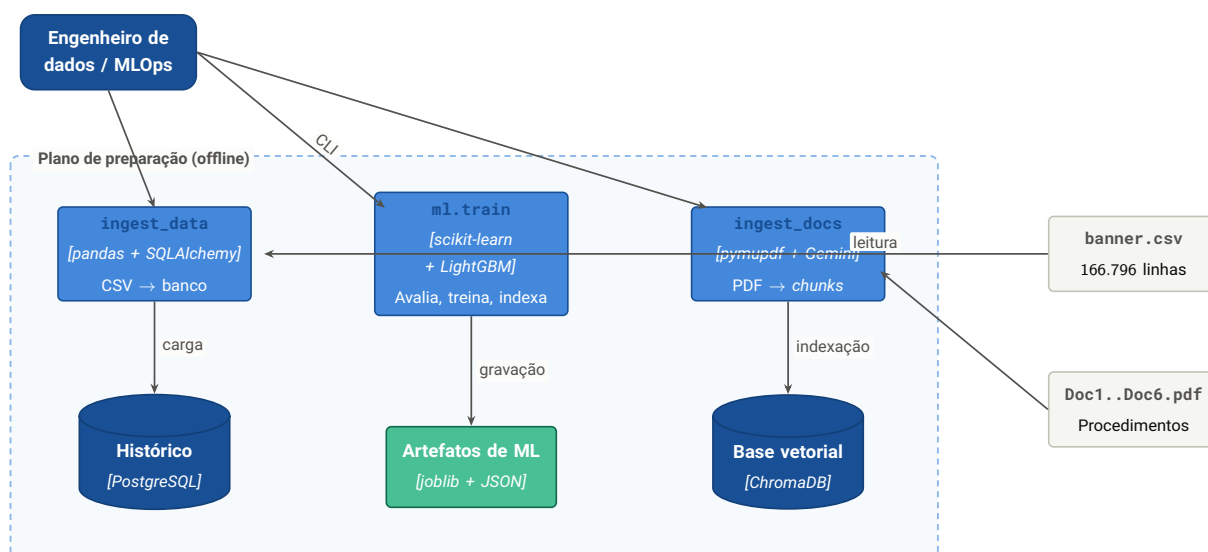


Figura 5.4 – C4 nível 2, plano de preparação. É deliberadamente offline: treinar ou reindexar é operação auditada, não contínua. O `banner.csv` nunca entra na imagem de contêiner — a carga é feita da máquina do operador apontando para o banco de destino.

5.4.4 Catálogo de contêineres

Tabela 5.3 – Catálogo de contêineres do sistema

Contêiner	Tecnologia	Responsabilidade
Interface	Streamlit 1.38+, Plotly	Renderizar painéis, formulários e conversa. Não possui lógica de domínio nem acesso a dados.
API de aplicação	FastAPI, Uvicorn	Única porta de entrada de domínio: diagnóstico, prescrição, conversa, analítica, documentos e saúde.
Worker de ingestão	paho-mqtt 2.1+	Consumir telemetria, validar, persistir de forma idempotente, acionar diagnóstico e publicar alertas.
Broker MQTT	Eclipse Mosquitto 2	Transportar telemetria, alertas e rejeições entre planta e sistema.
Histórico	PostgreSQL 16	Armazenar eventos, taxonomia, documentos, cobertura e auditoria de diagnósticos; executar as agregações analíticas.
Base vetorial	ChromaDB (HNSW, cosseno)	Indexar <i>chunks</i> com metadados; ser a fonte de verdade da cobertura documental.
Artefatos de ML	joblib	Persistir padronizador, classificador, índice de vizinhos e metadados do índice.
Scripts de preparação	Python 3.12 (CLI)	Executar ETL, treino, ingestão documental e simulação de sensor.

5.5 C4 nível 3 – Componentes da API

O contêiner de API concentra a lógica de domínio. Seus componentes estão organizados em três camadas mais uma faixa transversal; as dependências apontam sempre para baixo, o que mantém o domínio independente da infraestrutura.

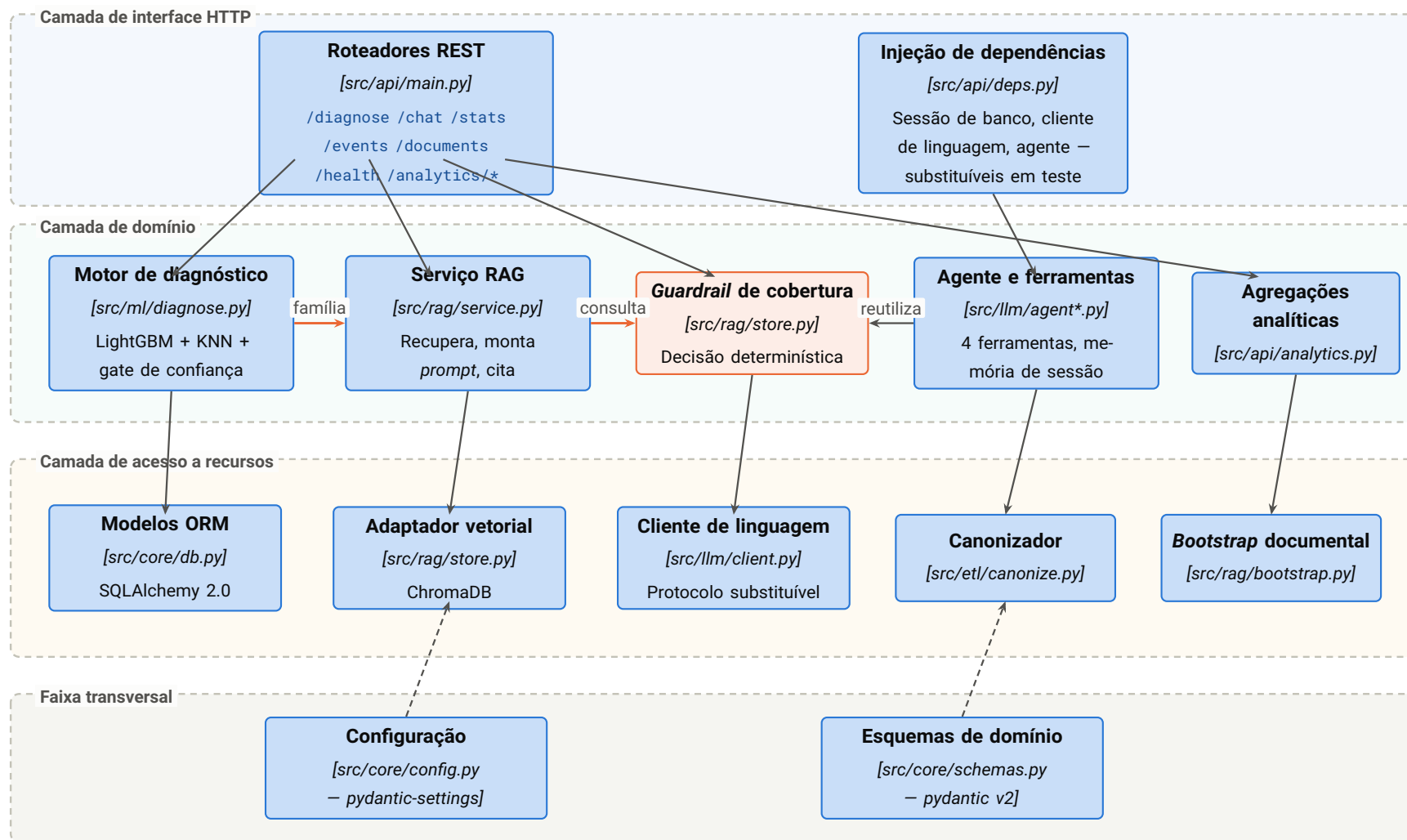


Figura 5.5 – C4 nível 3: componentes do contêiner de API. Em destaque laranja, o *guardrail* de cobertura: ele fica *entre* o motor de diagnóstico e a geração de texto, e é o único componente com poder de impedir a chamada ao modelo de linguagem. As setas em laranja marcam o caminho crítico da garantia exigida pelo enunciado.

5.6 C4 nível 4 – Código do motor de diagnóstico

O nível 4 é aplicado de forma seletiva, como recomenda o próprio modelo (Brown, 2026): apenas onde a estrutura de classes carrega uma decisão de projeto. Aqui, esse ponto é o motor de diagnóstico e o esquema de evento — o esquema é **único** para API, MQTT e ETL, o que torna impossível divergência de validação entre os três caminhos de entrada.

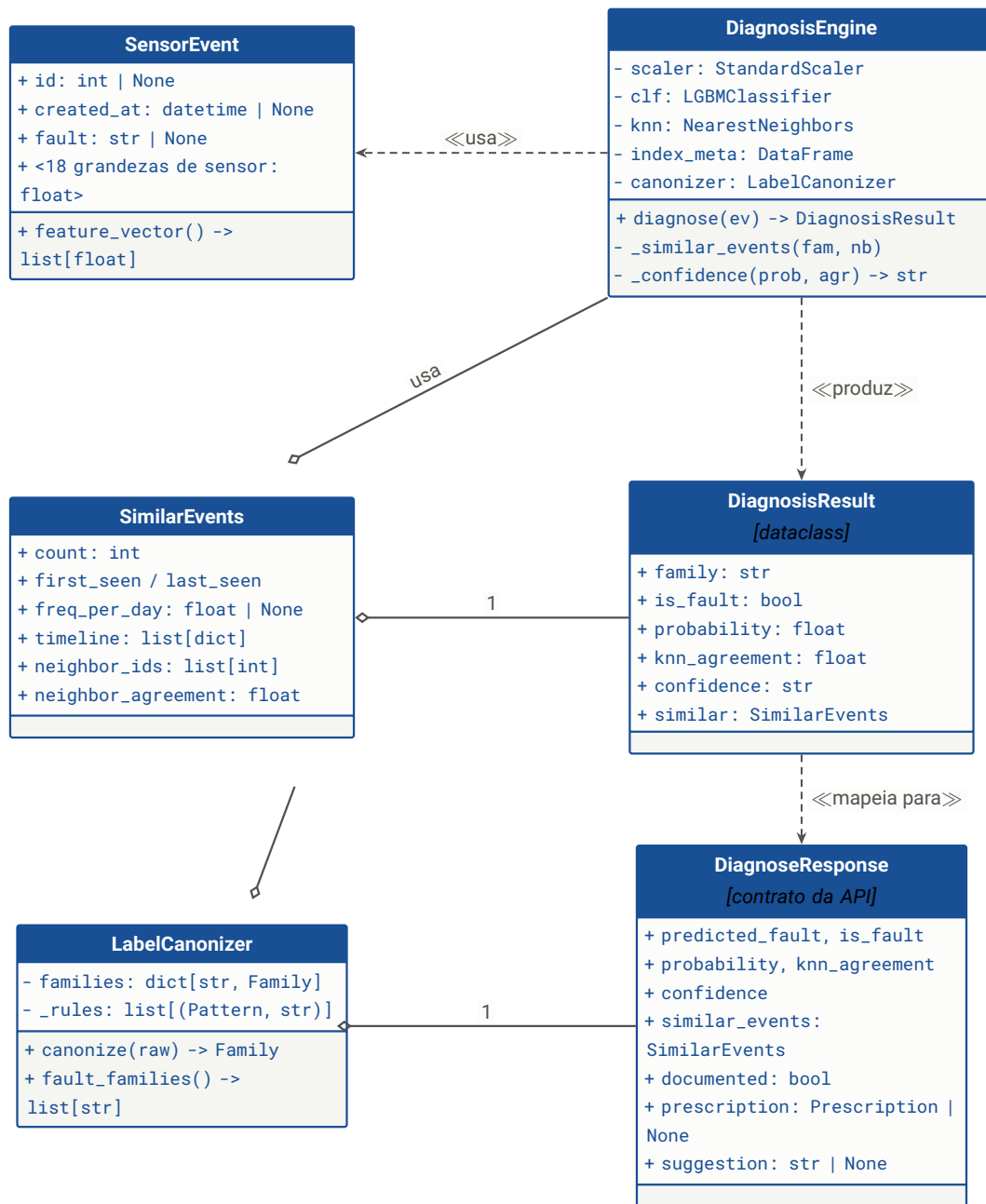


Figura 5.6 – C4 nível 4: classes do motor de diagnóstico. **SensorEvent** é o esquema único de entrada, compartilhado por API, worker MQTT e ETL. **DiagnosisResult** é interno ao domínio; **DiagnoseResponse** é o contrato público da API — a separação permite evoluir um sem quebrar o outro.

5.7 Fluxo do diagnóstico completo

O diagrama de sequência abaixo mostra o caminho crítico — o que acontece entre a submissão do evento e a resposta. Observe a posição do *guardrail*: ele decide *antes* de qualquer interação com o modelo de linguagem, e o ramo alternativo simplesmente não chega ao Vertex AI.

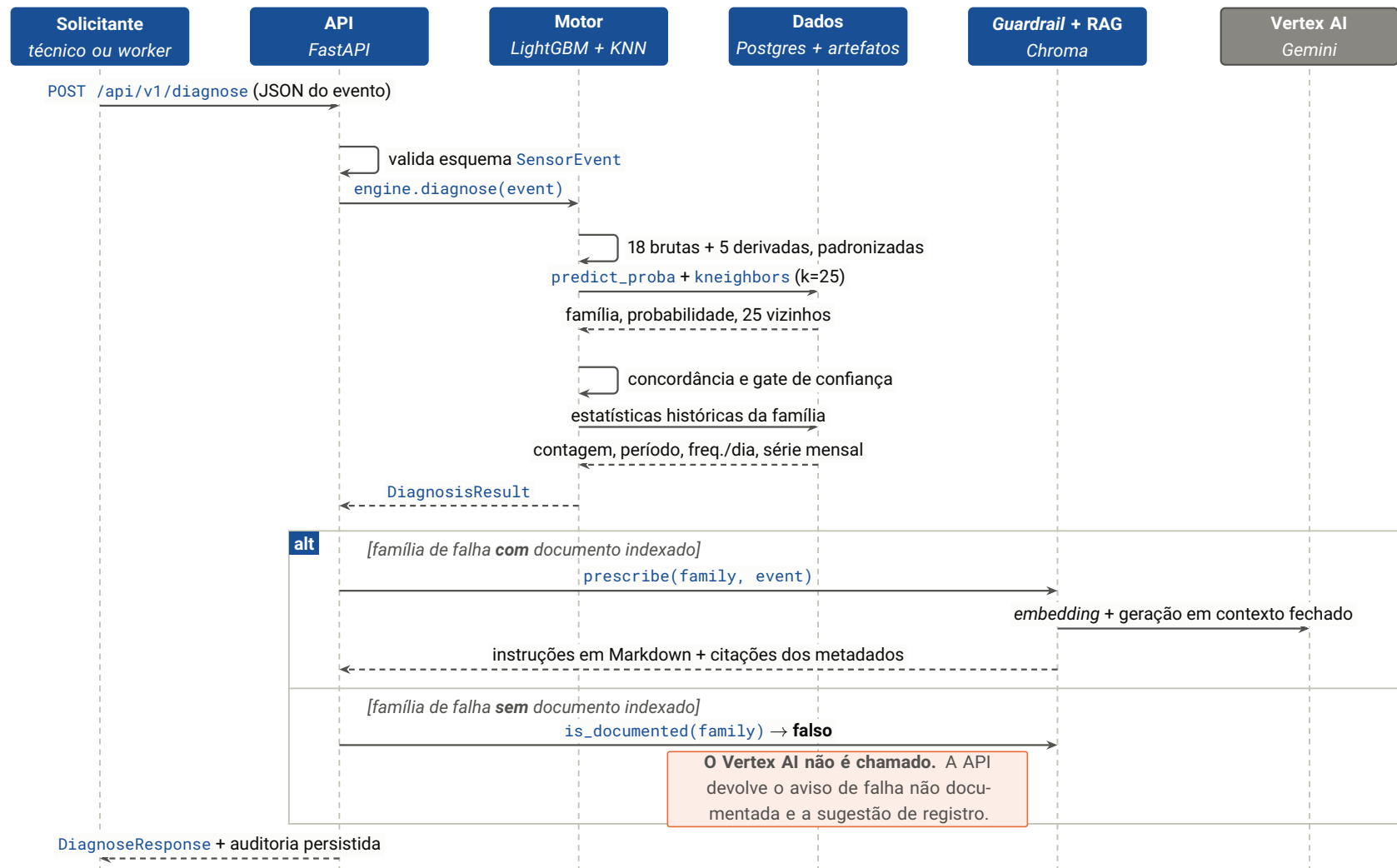


Figura 5.7 – Diagrama de sequência do diagnóstico completo. O bloco **alt** explicita a garantia do enunciado: no ramo inferior, a raia do Vertex AI permanece intocada — não há chamada ao modelo de linguagem quando a falha não está documentada.

5.8 Registro de decisões arquiteturais

Cada decisão está registrada no formato ADR ([Nygard, 2011](#)): contexto, decisão, justificativa e alternativa rejeitada. O valor do registro está na última coluna — saber o que foi *descartado* e por quê é o que evita refazer a discussão a cada revisão.

Tabela 5.5 – Registro de decisões arquiteturais (ADR)

ID	Decisão	Justificativa	Alternativa rejeitada
ADR-01	ChromaDB persistente e embarcado como base vetorial	Roda na própria estação, aceita filtro por metadados (essencial para restringir a busca a uma família) e usa índice HNSW (Malkov; Yashunin, 2020). Zero infraestrutura adicional.	FAISS (sem metadados nativos); <i>pgvector</i> (acoplaria a recuperação ao banco relacional).
ADR-02	<i>gemini-embedding-2</i> com dimensionalidade 768	Multilíngue, contexto de 8.192 <i>tokens</i> e dimensões aninhadas (Kusupati et al., 2022), o que permite truncar de 3.072 para 768 sem retreino e com perda marginal de qualidade.	<i>gemini-embedding-001</i> ; modelo local de sentenças (Reimers; Gurevych, 2019) (qualidade inferior em português técnico).
ADR-03	<i>gemini-3.6-flash</i> para geração, <i>temperature=0.2</i>	Baixa latência e custo, contexto amplo, e o identificador do modelo é parametrizado por variável de ambiente — trocar de modelo não exige alterar código.	Modelo local no hardware alvo (qualidade insuficiente em português técnico e competiria por memória com o índice KNN).
ADR-04	Diagnóstico híbrido: LightGBM e KNN	São respostas a perguntas diferentes. O KNN responde literalmente “quais registros históricos se parecem com este”; o classificador dá o tipo com probabilidade. A concordância entre os dois vira medida de confiança sem custo adicional.	Só KNN (não fornece probabilidade calibrada); rede neural profunda (desnecessária em dado tabular deste porte (Grinsztajn; Oyallon; Varoquaux, 2022)).
ADR-05	Canonicalização de rótulos por regras <i>regex</i> ordenadas em YAML	Transforma 151 rótulos ruidosos em 17 famílias de forma auditável — cada regra é legível e revisável. Rótulo sem regra gera erro explícito, nunca classificação silenciosa.	Agrupamento automático (não auditável, e erro de grafia não é problema de similaridade semântica).
ADR-06	FastAPI como camada de serviço; a interface só consome a API	Torna a solução integrável por SCADA, CMMS ou <i>historian</i> sem reimplementar domínio. A interface	Monólito em 43 / 187 Streamlit (rápido de escrever, impossível

5.9 Atributos de qualidade e táticas

Cada atributo de qualidade priorizado foi endereçado por uma tática concreta e verificável (Bass; Clements; Kazman, 2012). A coluna de verificação é o que distingue uma tática de uma intenção.

Atributo	Tática aplicada	Como se verifica
Correção factual	Guardrail determinístico + citação derivada de metadados + contexto fechado	Teste que assegura que o cliente de linguagem não é chamado para família sem documento.
Confiabilidade da ingestão	QoS 1 + merge idempotente por chave primária + fila de rejeição	Publicar o mesmo evento duas vezes produz uma linha; <i>payload</i> inválido aparece na DLQ.
Disponibilidade	Degradação graciosa do agente para RAG de um passo; <i>bootstrap</i> documental em <i>thread</i> separada	Remover credenciais mantém a conversa funcional; a instância passa o <i>healthcheck</i> durante a indexação inicial.
Testabilidade	Cliente de linguagem como protocolo; injeção de dependências com sobrescrita em teste	63 testes executam sem rede e sem credenciais.
Desempenho	Agregações em SQL sob demanda; artefatos carregados uma única vez em processo (<i>singleton</i> memorizado)	Agregação em torno de 0,2 s sobre 166.796 eventos.
Manutenibilidade	Esquema de domínio único para os três caminhos de entrada; dependências apontando sempre para baixo	Alterar uma grandeza de sensor exige edição em um único lugar.
Segurança	Segredos fora do repositório; credencial materializada com permissão 0600 ; <i>broker</i> com autenticação obrigatória quando exposto	Inspeção do repositório e da imagem; <i>log</i> do <i>broker</i> confirmando autenticação habilitada.
Auditabilidade	Tabela de diagnósticos; métricas versionadas em arquivo; ETL que falha diante de rótulo desconhecido	Cada resposta emitida é recuperável no banco.

A fronteira mais importante da arquitetura não é entre camadas de *software*: é entre o que é **determinístico** e o que é **generativo**. Tudo que constitui garantia — cobertura documental, citações, contagens históricas, classificação — é determinístico e testável. O modelo generativo atua apenas na redação do texto final, sobre um contexto que ele não escolheu e não pode ampliar. Essa separação é o que permite afirmar, com verificação e não com esperança, que o sistema respeita a restrição de responder somente sobre falhas documentadas.

Modelo de dados

O sistema mantém dados em três repositórios de naturezas distintas: o banco relacional (histórico, taxonomia e auditoria), a base vetorial (*chunks* documentais) e o sistema de arquivos (artefatos de aprendizado de máquina). Este capítulo especifica os três, com ênfase no relacional, que é onde vive o modelo de domínio.

6.1 Diagrama entidade-relacionamento

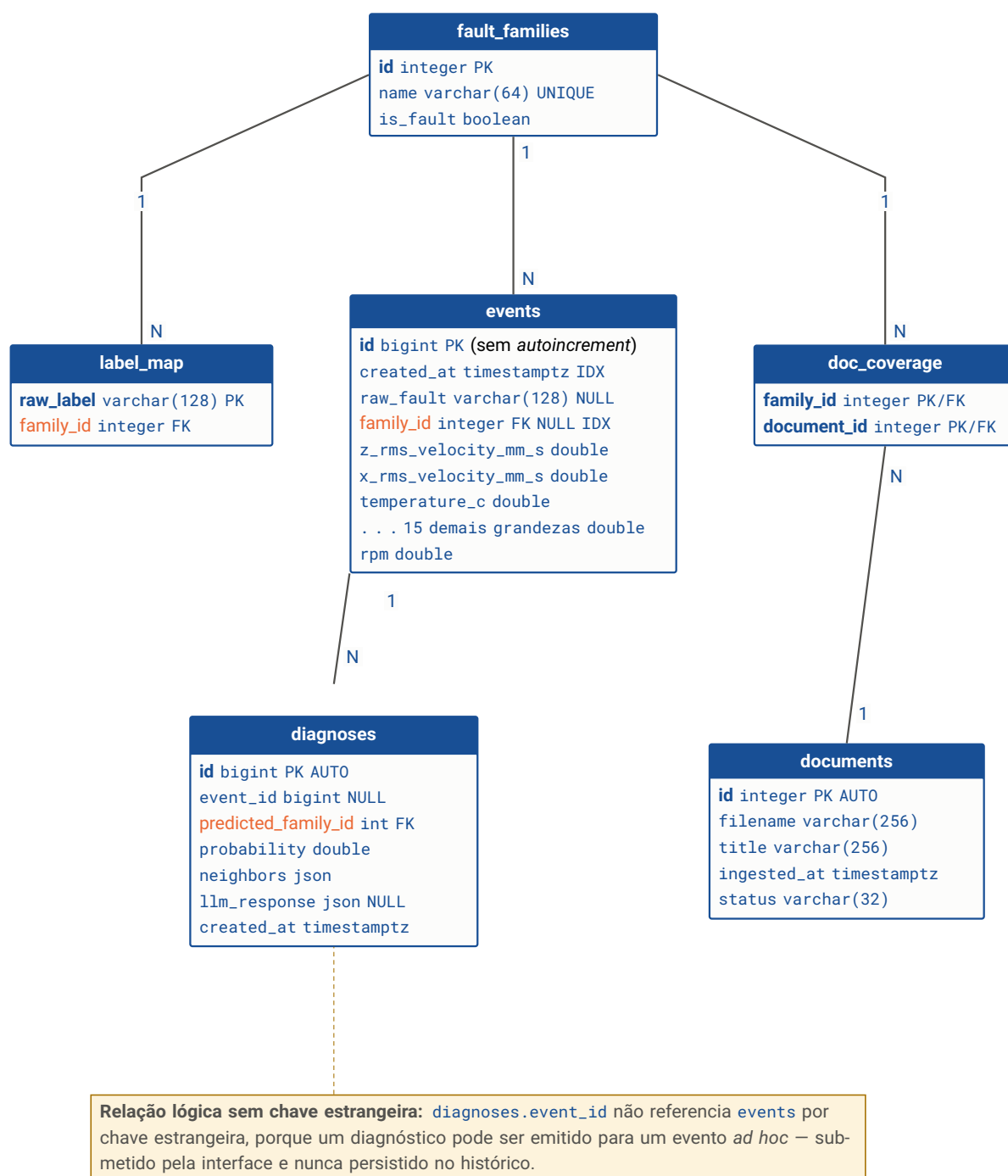


Figura 6.1 – Modelo entidade-relacionamento do banco corporativo. `fault_families` é a entidade central: ela ancora a taxonomia, os eventos, a cobertura documental e a auditoria de diagnósticos. A ausência deliberada de uma chave estrangeira entre `diagnoses` e `events` está justificada na nota.

A chave primária de `events` é o identificador **de origem** do evento, sem geração automática. Essa escolha é o que torna a ingestão MQTT idempotente: com entrega *at-least-once*, a mesma mensagem pode chegar duas vezes, e um `merge` pela chave primária resolve o problema na origem, sem tabela de deduplicação nem janela de tempo (**RNF-07**).

6.2 Dicionário de dados

6.2.1 Tabela `fault_families`

Taxonomia canônica de falhas e estados operacionais. É a fonte de verdade da classificação em todo o sistema — modelo, API, base documental e interface usam exatamente estes nomes.

Coluna	Tipo	Restrição	Descrição
<code>id</code>	<code>integer</code>	PK	Identificador da família.
<code>name</code>	<code>varchar(64)</code>	UNIQUE	Nome canônico (<i>snake_case</i>), ex. <code>rolamento_inner</code> . Usado como chave em todas as camadas.
<code>is_fault</code>	<code>boolean</code>	NOT NULL	Distingue falha (12 registros) de estado operacional (5 registros). Determina se há prescrição a emitir.

6.2.2 Tabela `label_map`

Registro auditável da canonicalização: preserva a correspondência entre cada rótulo bruto do conjunto de dados original e a família resultante.

Coluna	Tipo	Restrição	Descrição
<code>raw_label</code>	<code>varchar(128)</code>	PK	Rótulo bruto normalizado (minúsculas, sem espaços nas bordas). São 151 valores distintos.
<code>family_id</code>	<code>integer</code>	FK	Família canônica atribuída.

NOTA

Esta tabela existe para *auditoria*, não para consulta em tempo de execução: a canonicalização em *runtime* usa as regras de `label_map.yaml`. Manter o resultado materializado permite responder, meses depois, “por que este registro foi classificado assim?” sem reexecutar o ETL.

6.2.3 Tabela `events`

Histórico de telemetria. É a maior tabela do sistema (166.796 registros na carga inicial) e a base de todas as agregações analíticas.

Coluna	Tipo	Índice	Descrição
id	bigint	PK	Identificador de origem do evento. Não é gerado pelo banco — ver decisão acima.
created_at	timestampz	sim	Instante da leitura. Base das séries temporais e da frequência diária.
raw_fault	varchar(128)	—	Anotação original do operador. Identifica também a <i>campanha de coleta</i> , fato central na Seção 8.6.
family_id	integer	sim	Família canônica. Nulo quando o evento chega sem anotação (caso normal em produção).
z_rms_velocity_mm_s	double	—	Velocidade eficaz no eixo Z (mm/s).
x_rms_velocity_mm_s	double	—	Velocidade eficaz no eixo X (mm/s). Principal indicador de severidade.
temperature_c	double	—	Temperatura (°C). <i>Feature</i> mais influente do classificador — e principal suspeita de vazamento por sessão.
z_peak_acceleration_g	double	—	Aceleração de pico em Z (<i>g</i>).
x_peak_acceleration_g	double	—	Aceleração de pico em X (<i>g</i>).
z_peak_vel_comp_freq_hz	double	—	Frequência da componente de pico de velocidade em Z (Hz).
x_peak_vel_comp_freq_hz	double	—	Idem, eixo X. Base da <i>feature</i> derivada de ordem de rotação.
z_rms_acceleration_g	double	—	Aceleração eficaz em Z (<i>g</i>).
x_rms_acceleration_g	double	—	Aceleração eficaz em X (<i>g</i>).
z_kurtosis	double	—	Curtose em Z.
x_kurtosis	double	—	Curtose em X.
z_crest_factor	double	—	Fator de crista em Z.
x_crest_factor	double	—	Fator de crista em X.
z_peak_velocity_mm_s	double	—	Velocidade de pico em Z (mm/s).
x_peak_velocity_mm_s	double	—	Velocidade de pico em X (mm/s).
z_high_freq_rms_acce	double	—	Aceleração eficaz de alta frequência em Z (<i>g</i>).

Coluna	Tipo	Índice	Descrição
x_high_freq_rms_accel_double	double	—	Idem, eixo X. Base da razão de alta/baixa frequência.
rpm	double	—	Rotação do eixo. Variável de condição operacional — estratificar por ela é obrigatório em qualquer leitura de severidade.

6.2.4 Tabelas **documents** e **doc_coverage**

Coluna	Tipo	Restrição	Descrição
documents			
id	integer	PK, AUTO	Identificador do documento.
filename	varchar(256)	NOT NULL	Nome do arquivo, usado como chave de deduplicação na base vetorial.
title	varchar(256)	NOT NULL	Título técnico exibido nas citações.
ingested_at	timestampz	NOT NULL	Momento da indexação.
status	varchar(32)	default active	Permite retirar um procedimento de circulação sem apagar o histórico.
doc_coverage			
family_id	integer	PK, FK	Família coberta.
document_id	integer	PK, FK	Documento que a cobre.

doc_coverage é um **espelho relacional** da cobertura, mantido para consulta e relatório. A cobertura efetiva — aquela que o *guardrail* consulta — é sempre derivada da presença de *chunks* na base vetorial. Manter uma única fonte de verdade evita a falha clássica em que a tabela diz “coberto” e o índice está vazio.

6.2.5 Tabela **diagnoses**

Trilha de auditoria: registra cada diagnóstico emitido, permitindo reconstruir a posteriori o que o sistema respondeu e com que base.

Coluna	Tipo	Restrição	Descrição
<code>id</code>	<code>bigint</code>	PK, AUTO	Identificador do registro de auditoria.
<code>event_id</code>	<code>bigint</code>	NULL	Evento diagnosticado, se houver. Sem FK – ver nota da Figura 6.1.
<code>predicted_family_id</code>	<code>integer</code>	FK, NULL	Família prevista.
<code>probability</code>	<code>double</code>	NOT NULL	Probabilidade atribuída pelo classificador.
<code>neighbors</code>	<code>json</code>	–	Identificadores dos 25 vizinhos que sustentaram a resposta.
<code>llm_response</code>	<code>json</code>	NULL	Prescrição gerada e citações. Nulo quando a família não é documentada.
<code>created_at</code>	<code>timestampz</code>	NOT NULL	Momento da emissão.

6.3 Esquema `adk` – memória de conversa

O *framework* de agentes cria tabelas próprias para persistir sessões de conversa, e uma delas chama-se `events` – exatamente o nome da tabela central do domínio. A colisão é resolvida por isolamento em esquema dedicado.

Aspecto	Solução adotada
Isolamento	Esquema <code>adk</code> , criado na inicialização do agente (<code>CREATE SCHEMA IF NOT EXISTS adk</code>).
Roteamento	<code>search_path</code> fixado em <code>adk</code> nos parâmetros de conexão da engine assíncrona, de modo que o <i>framework</i> nunca “veja” as tabelas de domínio.
Driver	As sessões exigem driver assíncrono; a URL do banco é convertida de <code>psycopg2</code> para <code>asyncpg</code> em tempo de inicialização.
Ciclo de vida	Tabelas criadas pelo próprio <i>framework</i> . O domínio não depende delas: se o esquema não existir, o agente falha na inicialização e o sistema degrada para RAG de um passo.

6.4 Base vetorial – coleção manual¹s

Uma única coleção, com espaço métrico de cosseno e índice HNSW ([Malkov; Yashunin, 2020](#)). Cada registro é um *chunk* de documento.

Campo	Tipo	Descrição
<code>id</code>	texto	Chave composta <code>{arquivo}::{família}::{índice}</code> . Torna a reindexação idempotente e permite remover um documento por filtro.
<code>embedding</code>	vetor(768)	Representação densa gerada por <code>gemini-embedding-2</code> com dimensionalidade truncada.
<code>document</code>	texto	Conteúdo do <i>chunk</i> (aproximadamente 1.200 caracteres).
<code>metadata.doc</code>	texto	Nome do arquivo de origem — vira a citação exibida.
<code>metadata.title</code>	texto	Título técnico do procedimento.
<code>metadata.section</code>	texto	Seção corrente no momento da extração (ex. 4.2 Alinhamento a laser).
<code>metadata.page</code>	inteiro	Página de origem.
<code>metadata.family</code>	texto	Família canônica coberta. É o campo que sustenta o guardrail: a existência de ao menos um registro com determinada família é a definição de “documentado”.

NOTA

Um documento que cobre F famílias é indexado F vezes, uma por família. Isso multiplica o armazenamento, mas permite que o filtro por família seja executado pelo próprio índice, e não por pós-filtragem — o que preservaria menos resultados relevantes para um mesmo *top-k*. Com 268 *chunks* na base inicial, o custo é irrelevante; em escala de milhares de documentos, a alternativa seria um campo multivalorado com filtro de inclusão.

6.5 Artefatos de aprendizado de máquina

Arquivo	Tamanho	Conteúdo
<code>scaler.joblib</code>	1,1 KiB	Médias e desvios das 23 <i>features</i> , ajustados sobre 100 % dos dados.
<code>lgbm.joblib</code>	38 MiB	Classificador multiclasse de 400 árvores para 17 classes.
<code>knn.joblib</code>	29 MiB	Índice de vizinhos sobre o histórico completo padronizado.

<code>index_meta.joblib</code>	5,8 MiB	Tabela com <code>id</code> , <code>created_at</code> e <code>family</code> de cada linha indexada — é o que traduz posição no índice em evento real.
<code>metrics.json</code>	3,9 KiB	Métricas dos três protocolos de avaliação, classes, lista de <i>features</i> e <i>timestamp</i> do treino.

Total de 74 MiB, carregados uma única vez por processo em um *singleton* memorizado. O índice de vizinhos é o componente dominante de memória em execução e a razão pela qual o serviço de API é o mais pesado da topologia (**RNF-01**).

Pipeline de dados e canonicalização

7.1 Caracterização da fonte de dados

O conjunto de dados original é um arquivo de 31 MB com telemetria de uma bancada instrumentada de máquina rotativa.

Tabela 7.1 – Caracterização do conjunto de dados `banner.csv`

Propriedade	Valor
Registros	166.796
Período	30/04/2026 a 16/06/2026 (aproximadamente 47 dias)
Colunas de sensor utilizadas	18 (em unidades SI)
Colunas descartadas	Duplicatas em in/s e °F — colineares por construção com as contrapartes métricas
Rótulos brutos distintos	151
Famílias canônicas	17 (12 falhas + 5 estados operacionais)
Campanhas com ≥ 30 eventos	146
Regimes de rotação	500 rpm (55.857) · 2.000 rpm (55.160) · 1.000 rpm (53.414) · 3.000 rpm (1.707) · 0 rpm (658)
Equipamentos	1 (bancada única)

Os regimes de rotação são fortemente desbalanceados: três regimes concentram 98,6 % dos registros, e o regime de 3.000 rpm — justamente onde o desbalanceamento se mani-

feira de forma inequívoca (Seção 8.8.2) — tem apenas 1.707 amostras. Toda conclusão sobre alta rotação repousa sobre 1 % dos dados.

7.2 Canonicalização de rótulos

7.2.1 O problema

A coluna de anotação do conjunto de dados contém 151 valores distintos para 17 condições reais. A dispersão tem três causas, todas presentes em dados industriais reais:

- Variação de campanha:** sufixos e prefixos que identificam a sessão de coleta, não a falha — `_2, _3, _pos_2, _carga, _adx1_0, new_`.
- Erros de digitação:** `ddesbalanceado, cockecocked_adxl_0, mortor_desligado, normla_carga_3_3, desbanlanceado, desabalanceado, desabanceado`.
- Mistura de falhas com estados:** `baseline, teste, acelerando` e `motor_desligado` não são defeitos, e tratá-los como tal produziria prescrições para condições normais.

7.2.2 A solução: regras ordenadas e auditáveis

A canonicalização é feita por uma lista *ordenada* de expressões regulares declaradas em YAML. A primeira regra que casa vence. Regras de falha precedem regras de estado, para que `rolamento_outer_novo_teste` caia em `rolamento_outer` e não em `teste`.

Listing 7.1 – Extrato de `src/etl/label_map.yaml` — declaração de famílias e regras ordenadas

```

1 families:
2   rolamento_inner: { is_fault: true, display: "Defeito na pista interna do rolamento (BPFI)" }
3   desbalanceamento: { is_fault: true, display: "Desbalanceamento do rotor" }
4   cocked_rotor: { is_fault: true, display: "Rotor inclinado (cocked rotor)" }
5   normal: { is_fault: false, display: "Operacao normal" }
6   motor_desligado: { is_fault: false, display: "Motor desligado" }
7
8   # A ordem importa: a primeira regra que casar vence.
9   rules:
10    - { pattern: "inner", family: rolamento_inner }
11    - { pattern: "outer", family: rolamento_outer }
12    - { pattern: "ball", family: rolamento_ball }
13    - { pattern: "comb", family: rolamento_combination }
14    # cobre desb-, e os erros reais ddesb-, desbanlanceado, desabalanceado
15    - { pattern: "desb|abal|abanc|balanc", family: desbalanceamento }
16    - { pattern: "desalinh", family: desalinhamento }
17    - { pattern: "cock", family: cocked_rotor } # cobre "cockecocked"
18    # --- estados operacionais, avaliados por ultimo ---
19    - { pattern: "desligado", family: motor_desligado } # cobre "mortor_"
20    - { pattern: "norm", family: normal } # cobre "normla"
21    - { pattern: "^(new_)?tes", family: teste }
```

A alternativa óbvia — agrupamento automático por similaridade de texto — foi rejeitada por duas razões. Primeiro, não é auditável: ninguém consegue justificar, meses depois, por que dois rótulos caíram no mesmo grupo. Segundo, erro de grafia não é problema de similaridade semântica: `cockecocked` e `cocked` são vizinhos por acidente de digitação, e `rolamento_inner` e `rolamento_outer` são textualmente próximos mas descrevem defeitos distintos, com procedimentos distintos. Regras explícitas erram menos e explicam-se sozinhas (**ADR-05**).

7.2.3 Auditoria: falhar cedo em vez de classificar em silêncio

O canonizador levanta exceção diante de rótulo sem regra. O ETL, por sua vez, canonicaliza **todos** os rótulos distintos *antes* de gravar qualquer linha:

Listing 7.2 – Auditoria integral antes da gravação — `scripts/ingest_data.py`

```
1 # Canonicalizacao: audita 100% dos rotulos antes de gravar qualquer coisa
2 raw_labels = df["fault"].astype(str).str.strip().str.lower()
3 unique_labels = sorted(raw_labels.unique())
4 mapping = {lbl: canonizer.canonize(lbl) for lbl in unique_labels} # levanta se desconhecido
5 print(f"{len(unique_labels)} rotulos brutos mapeados para "
6       f"{len({f.name for f in mapping.values()})} familias canonicas.")
```

A consequência prática: um rótulo novo em uma carga futura interrompe o processo com o nome exato do rótulo, exigindo curadoria humana. É o oposto do comportamento usual — descartar a linha ou atribuí-la a uma categoria “outros” — que produz uma base silenciosamente incorreta (**RNF-14**).

7.3 Fluxo do ETL

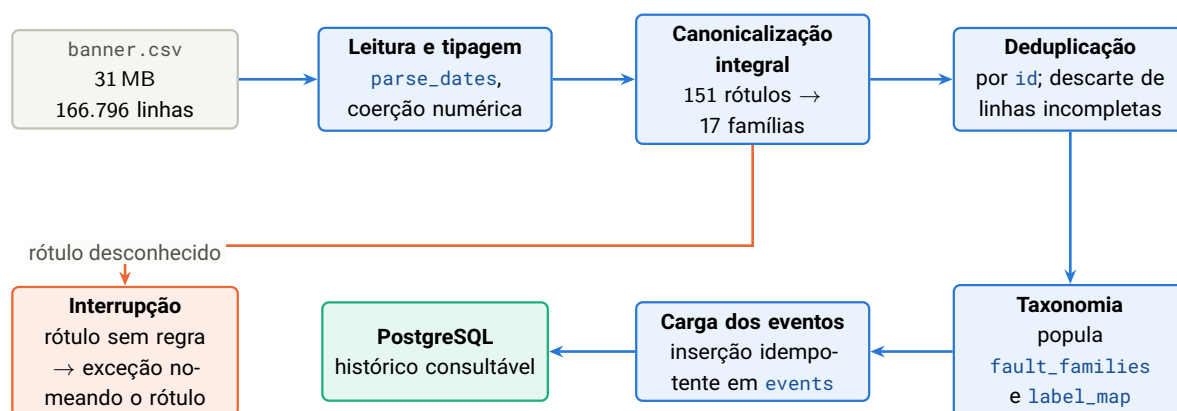


Figura 7.1 – Pipeline do ETL do histórico. A canonicalização integral acontece *antes* de qualquer escrita: se algum rótulo não casar com regra, o processo termina sem tocar o banco. O caminho em laranja é a porta de qualidade de dados.

7.4 Features geradas na carga

O ETL grava as 18 grandezas brutas; as 5 *features* derivadas são calculadas em tempo de treino e de inferência pelo *mesmo* módulo (`src/ml/features.py`), nunca materializadas no banco.

Não materializar as derivadas evita a classe de defeito mais insidiosa em sistemas de aprendizado de máquina: a divergência entre a transformação aplicada no treino e a aplicada na inferência — o *training-serving skew* descrito por Sculley et al. (Sculley et al., 2015). Com uma única função compartilhada, a divergência é impossível por construção. O custo é recalculando cinco divisões por evento, o que é irrelevante.

7.5 Agregações analíticas

As análises do painel são consultas SQL executadas sob demanda, não artefatos pré-computados. Três agregações não triviais merecem registro:

Agregação	O que calcula e por quê
<code>severity_by_rpm</code>	Percentis 50, 95 e 99 da velocidade eficaz por família e por regime de rotação, mais a linha de base do estado normal no mesmo regime. Usa percentis em vez de média porque as distribuições têm cauda longa e a média seria puxada por transientes de partida.
<code>fault_signatures</code>	Escore padronizado de cada família em cada <i>feature</i> e a razão entre variância inter e intrafamília — no espírito da estatística F da análise de variância. Mostra objetivamente quais indicadores separam as falhas.
<code>leakage_evidence</code>	Decomposição da dispersão de cada <i>feature</i> entre campanhas de coleta e dentro de cada campanha. É a evidência quantitativa do confundimento analisado na Seção 8.6.

NOTA

As três agregações rodam em torno de 0,2s sobre 166.796 eventos. Por isso não há pré-cálculo: o painel reflete inclusive os eventos que chegam por MQTT durante uma demonstração ao vivo. Interpolação de nomes de coluna nas consultas é protegida por lista branca derivada da constante de *features* do próprio código — coluna fora da lista levanta exceção.

Motor de diagnóstico: engenharia de *features*, modelo e validação

Este capítulo documenta o componente que produz o *tipo de defeito* e as *ocorrências semelhantes*. Ele é organizado na ordem em que as decisões foram tomadas: primeiro o que os manuais dizem sobre a física do problema, depois as *features* que traduzem essa física, depois a escolha do algoritmo, o protocolo de validação, os resultados — e, por fim, o diagnóstico honesto de por que os resultados são o que são.

8.1 Fundamentação física

A análise de vibração de máquinas rotativas não é um problema genérico de classificação tabular: existe teoria estabelecida que relaciona cada defeito a uma assinatura espectral específica ([Randall, 2011](#); [Randall; Antoni, 2011](#)). Essa teoria é o que justifica as *features* derivadas.

Tabela 8.1 – Assinaturas de defeito segundo a literatura de análise de vibração

Defeito	Assinatura esperada	Implicação para as <i>features</i>
Desbalanceamento	Pico dominante em $1\times$ a rotação; amplitude proporcional a $m r \omega^2$	Exige <i>feature</i> de <i>ordem</i> (frequência normalizada pela rotação), não de frequência absoluta. A dependência quadrática com ω é verificável – e foi verificada (Seção 8.8.2).
Desalinhamento e rotor inclinado	Componentes em $1\times$ e $2\times$; presença de vibração <i>axial</i> apreciável	Justifica a razão entre eixos radial e axial como <i>feature</i> .
Defeitos de rolamento (BPFO, BPFI, BSF)	Múltiplos não inteiros da rotação, determinados pela geometria; impactos elevam curtose e fator de crista	Justifica a razão entre energia de alta frequência e energia global. A detecção das frequências características exigiria o espectro completo, indisponível.
Falhas de correia e polia	Componentes subsíncronas ($< 1\times$) e harmônicas da frequência da correia	Recuperáveis apenas parcialmente a partir de um único valor de pico.

O conjunto de dados fornece **métricas pré-agregadas** pelo sensor, não a forma de onda. Isso significa que as frequências características de rolamento – o instrumento mais poderoso do diagnóstico de vibração – são estruturalmente inalcançáveis: elas exigem análise de envelope sobre o sinal bruto (Randall; Antoni, 2011). As *features* derivadas extraem o máximo do que existe, mas não substituem o espectro.

8.2 Features do modelo

O vetor de entrada tem 23 dimensões: 18 grandezas brutas e 5 derivadas.

8.2.1 Grandezas brutas

As 18 grandezas brutas são as colunas do conjunto de dados em unidades SI. As duplicatas em in/s e °F foram excluídas: são transformações lineares exatas das contrapartes métricas e sua inclusão introduziria colinearidade perfeita sem acrescentar informação. A lista completa está no Apêndice B.

8.2.2 Features derivadas

Tabela 8.3 – Features derivadas e sua motivação física

Feature	Definição	Motivação
x_peak_order z_peak_order	$\frac{f_{\text{pico}}}{\max(\text{RPM}/60, \varepsilon)}$, limitada a 50	Converte frequência absoluta em <i>ordem</i> de rotação. Os manuais descrevem assinaturas em ordens; sem essa normalização, o mesmo defeito em regimes de rotação distintos apareceria como <i>features</i> distintas.
radial_axial_rat:	$\frac{v_{\text{RMS},X}}{v_{\text{RMS},Z} + \varepsilon}$	Desbalanceamento é predominantemente radial; desalinhamento e rotor inclinado produzem componente axial apreciável. A razão captura essa diferença de forma invariante à amplitude absoluta.
hf_lf_ratio_x hf_lf_ratio_z	$\frac{a_{\text{HF,RMS}}}{a_{\text{RMS}} + \varepsilon}$	Defeitos localizados em rolamentos excitam preferencialmente alta frequência. A razão aproxima, com o dado disponível, o que a análise de envelope faria com o sinal bruto.

O cálculo usa proteção numérica explícita ($\varepsilon = 10^{-6}$ e limite superior de ordem igual a 50) para tratar os 658 eventos com rotação nula — motor desligado — sem produzir divisão por zero nem valores extremos que distorceriam a padronização.

8.2.3 Padronização

Todas as 23 *features* são padronizadas ($z = (x - \mu)/\sigma$) com parâmetros ajustados no conjunto de treino. A padronização é **obrigatória**, não opcional: a busca por vizinhos usa distância euclidiana, e sem normalização a rotação — na ordem de 1.000 — dominaria completamente a métrica, tornando a curtose (ordem de 2,7) irrelevante.

8.3 Escolha do algoritmo

8.3.1 Por que dois modelos e não um

O enunciado pede duas coisas diferentes: o *tipo de defeito* e as *ocorrências históricas semelhantes*. São perguntas distintas, e a resposta honesta é usar o mecanismo adequado a cada uma.

Pergunta do enunciado	Mecanismo	Por quê
“Qual o tipo de defeito?”	Classificador LightGBM	Produz a classe e sua probabilidade, permitindo quantificar incerteza (Ke et al., 2017).
“Quais registros históricos se comportam de forma semelhante?”	Busca de k vizinhos mais próximos	Responde <i>literalmente</i> à pergunta, devolvendo os registros reais e não uma abstração de classe (Cover; Hart, 1967).

Um efeito colateral valioso: como os dois mecanismos são independentes, a *concordância* entre eles é uma medida de confiança que não custa nada calcular (Seção 8.7).

8.3.2 Por que *gradient boosting* sobre árvores

A escolha de família de modelo para dado tabular foi feita com base em evidência empírica publicada, não por preferência.

- Dado tabular heterogêneo favorece modelos baseados em árvores.** A comparação sistemática de Grinsztajn, Oyallon e Varoquaux ([Grinsztajn; Oyallon; Varoquaux, 2022](#)) sobre 45 conjuntos de dados mostra que modelos de árvore permanecem no estado da arte em dado tabular de porte médio, mesmo desconsiderando sua vantagem de velocidade. As causas identificadas — robustez a *features* não informativas, invariância a transformações monotônicas e viés indutivo adequado a fronteiras de decisão não suaves — descrevem exatamente este problema. Shwartz-Ziv e Armon ([Shwartz-Ziv; Armon, 2022](#)) chegam à mesma conclusão em estudo independente.
- Boosting sobre bagging.** O *gradient boosting* ([Friedman, 2001](#)) ajusta árvores sequencialmente ao resíduo, o que costuma superar florestas aleatórias ([Breiman, 2001](#)) em problemas com estrutura de interação entre *features* — caso aqui, em que a relação entre amplitude e rotação é multiplicativa, não aditiva.
- LightGBM entre as implementações de boosting.** Frente ao XGBoost ([Chen; Guestrin, 2016](#)), o LightGBM traz duas otimizações relevantes nesta escala: amostragem por gradiente unilateral, que concentra o esforço nas amostras de maior erro, e agrupamento de *features*

mutuamente exclusivas (Ke et al., 2017). Em 166.796 amostras por 23 *features*, o treino completo dos três protocolos de avaliação mais o modelo final leva poucos minutos na estação alvo.

4. **Rede neural profunda foi descartada.** Além da evidência dos itens anteriores, o volume de dados por classe (800 eventos na classe minoritária) e a restrição de hardware (RE-01) não sustentariam o custo de ajuste de hiperparâmetros de uma arquitetura profunda para um ganho que a literatura não prevê.

8.3.3 Hiperparâmetros e sua justificativa

Tabela 8.6 – Hiperparâmetros do classificador

Parâmetro	Valor	Justificativa
<code>n_estimators</code>	400	Suficiente para convergir com a taxa de aprendizado escolhida; além disso o ganho é marginal e o artefato cresce.
<code>learning_rate</code>	0,08	Valor conservador: taxas maiores aceleram o treino mas aumentam a variância entre execuções, o que prejudicaria a comparação entre protocolos de avaliação.
<code>num_leaves</code>	63	Controla a capacidade. Com 23 <i>features</i> e 17 classes, valores muito maiores memorizam a campanha de coleta — efeito agravado pelo confundimento descrito na Seção 8.6.
<code>subsample</code>	0,9	Amostragem estocástica de linhas por árvore, para reduzir sobreajuste.
<code>colsample_bytree</code>	0,9	Amostragem de colunas por árvore, com o mesmo objetivo.
<code>class_weight</code>	<code>balanced</code>	Essencial: a classe majoritária tem 16.497 eventos e a minoritária 800. Sem reponderação, o modelo maximizaria acurácia ignorando as classes raras — e o F1 macro, que é a métrica reportada, penaliza exatamente isso (Sokolova; Lapalme, 2009).
<code>random_state</code>	42	Reprodutibilidade (RNF-03).

8.3.4 Busca por vizinhos

- $k = 25$ vizinhos, distância euclidiana no espaço padronizado.

- O índice cobre o **histórico completo** — 166.796 eventos — e não uma amostra. Isso responde ao requisito de forma literal: os vizinhos devolvidos são eventos reais, identificáveis por *id* no banco.
- A escolha de $k = 25$ equilibra dois objetivos: número suficiente para que a fração de concordância seja informativa (com $k = 5$, a concordância assume apenas seis valores discretos) e pequeno o bastante para permanecer local no espaço de *features*.

8.4 Protocolo de validação

8.4.1 Por que a divisão aleatória seria inválida aqui

Este é o ponto metodológico central do projeto. O conjunto de dados é composto de **campanhas de coleta**: cada rótulo bruto identifica uma sessão em que a bancada foi configurada em determinada condição e leituras foram registradas em sequência. Leituras da mesma campanha são quase idênticas.

Uma divisão aleatória colocaria leituras quase duplicadas da mesma campanha em treino e em teste, o que constitui vazamento de dados no sentido preciso de Kaufman *et al.* (Kaufman *et al.*, 2012): informação indisponível no momento da predição real influenciando a estimativa de desempenho. Kapoor e Narayanan (Kapoor; Narayanan, 2023) documentam esse padrão em pelo menos 294 artigos de 17 disciplinas, e o classificam como a principal causa de resultados irreprodutíveis em ciência baseada em aprendizado de máquina.

A literatura de validação para dados com estrutura de agrupamento é explícita sobre a correção: quando as observações são agrupadas — temporal, espacial ou hierarquicamente — a divisão deve ocorrer *no nível do grupo*, não da observação (Roberts *et al.*, 2017). Ignorar isso subestima sistematicamente o erro de predição.

8.4.2 Por que a divisão temporal também não serve

A alternativa natural — separar o período final para teste — é impossível aqui: quatro famílias existem *apenas* no período final do histórico. Um corte temporal produziria classes presentes no teste e ausentes do treino, o que mede outra coisa (adaptação a classes novas), não generalização.

8.4.3 Os três protocolos adotados

Tabela 8.8 – Protocolos de avaliação registrados a cada treino

Protocolo	Papel	O que mede
Holdout por sessão de coleta	Métrica principal	Para cada família, cerca de 20 % das campanhas inteiras vão para o teste. Responde à pergunta de produção: “o modelo reconhece uma falha conhecida em uma <i>nova</i> campanha de coleta?”
Divisão aleatória estratificada	Referência otimista	Separabilidade dos padrões <i>dentro</i> das campanhas. Reportada apenas para quantificar o tamanho do vazamento.
Holdout por sessão sem a temperatura	Teste de ablação	Isola o efeito da <i>feature</i> mais suspeita de funcionar como carimbo da campanha.

O modelo final servido — os artefatos carregados pela API — é retreinado com 100 % dos dados após as avaliações. Essa separação entre “modelo avaliado” e “modelo servido” evita o viés otimista de selecionar configuração pela métrica de teste e depois reportar essa mesma métrica (Varma; Simon, 2006).

8.5 Resultados

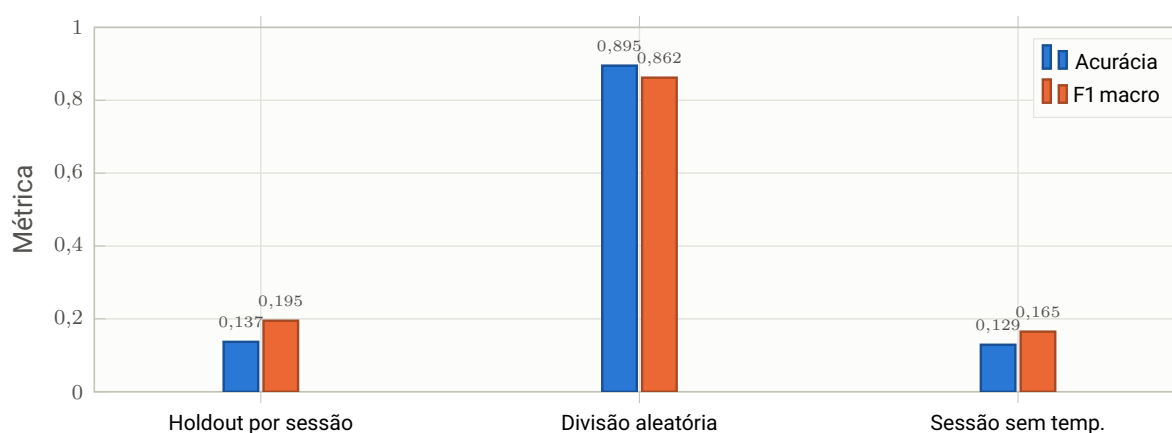


Figura 8.1 – Resultados nos três protocolos de avaliação, conforme registrado em `artifacts/metrics.json` (treino de 22/08/2026, 166.796 eventos). A distância entre a primeira e a segunda barra — 0,137 contra 0,895 de acurácia — é a medida do vazamento por campanha de coleta, não ruído de amostragem.

Tabela 8.10 – Métricas detalhadas por protocolo

Protocolo	Acurácia	F1 macro	F1 ponderado	<i>n</i> teste
<i>Holdout</i> por sessão (principal)	0,1373	0,1950	0,1862	38.233
Divisão aleatória estratificada	0,8950	0,8625	0,8949	33.360
<i>Holdout</i> por sessão sem temperatura	0,1289	0,1653	0,1697	38.233

8.5.1 Desempenho por classe

O F1 macro agregado esconde uma distribuição muito desigual. O detalhamento mostra o que efetivamente generaliza:

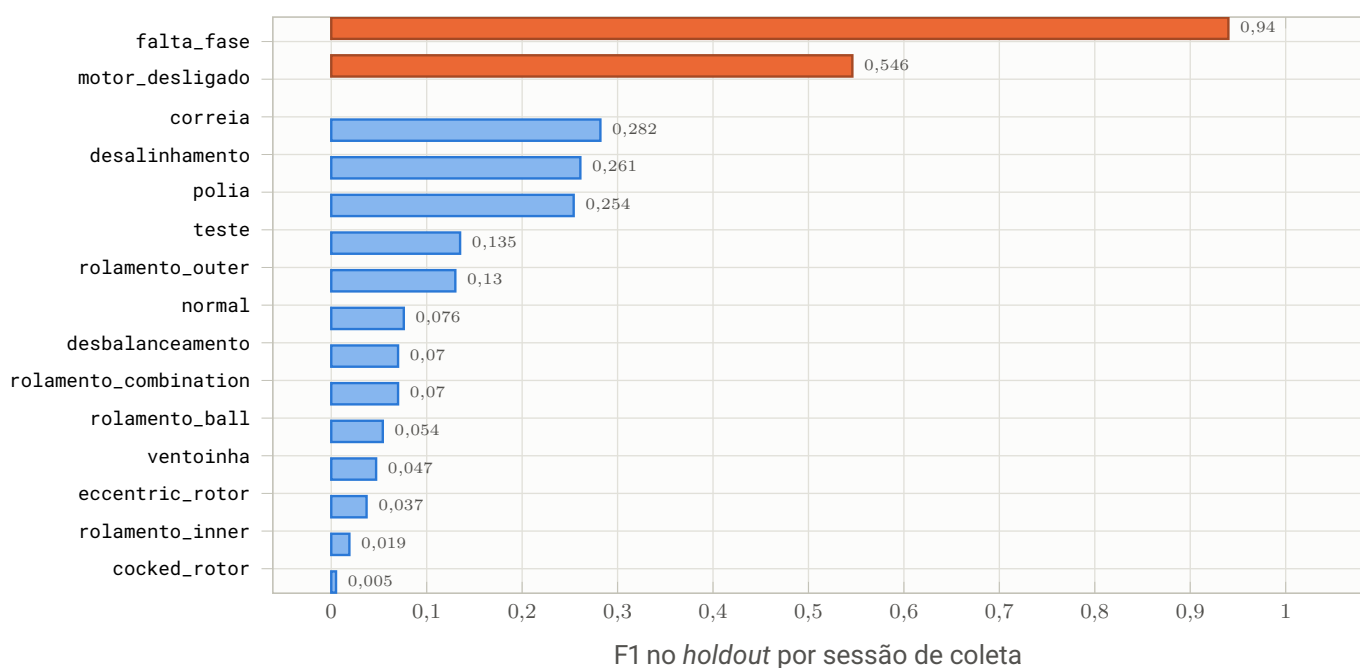


Figura 8.2 – F1 por classe no *holdout* por sessão. Em laranja, as duas classes que efetivamente generalizam. *falta_fase* tem assinatura elétrica inequívoca; *motor_desligado* é um estado, não uma falha — ausência de vibração é trivial de reconhecer. As famílias de rolamento, que são o alvo diagnóstico mais valioso, praticamente não generalizam.

8.6 Diagnóstico do vazamento

Uma queda de 0,895 para 0,137 de acurácia entre protocolos não é um resultado que se reporta sem explicação. Esta seção documenta a investigação que identificou o mecanismo — incluindo o ponto em que a hipótese inicial foi *refutada*.

8.6.1 Hipótese inicial

A *feature* mais influente do classificador é a temperatura (11,7 % da importância por ganho). Temperatura ambiente não é sintoma mecânico de rotor inclinado. A hipótese formulada foi: a temperatura funciona como **relógio da campanha de coleta**. Cada campanha foi gravada em um momento distinto, com temperatura própria; como cada campanha carrega um único rótulo, “temperatura $\approx 25,3^\circ\text{C}$ ” prediz o rótulo sem qualquer conteúdo físico.

8.6.2 Teste: decomposição da variância

A hipótese é verificável. Se a temperatura identifica a campanha, sua dispersão *entre* as médias das campanhas deve ser muito maior que a dispersão *dentro* de cada campanha.

Tabela 8.12 – Decomposição da variância da temperatura (146 campanhas com ≥ 30 eventos)

Componente	Desvio-padrão
Entre as médias das campanhas	2,70 °C
Dentro de cada campanha (mediana)	0,31 °C
Razão	8,8×

A confirmação direta vem de comparar a *mesma* família em campanhas diferentes: `rolamento_inner` aparece a 19,5 °C, 18,0 °C e 25,3 °C em três campanhas; `cocked_rotor` a 21,9 °C, 22,6 °C e 26,9 °C. A variação entre campanhas da mesma condição é maior que a variação entre condições.

8.6.3 Ablação: onde a hipótese foi refutada

Se a temperatura *causasse* o *gap*, removê-la deveria recuperar generalização. O treino passou a registrar o protocolo de ablação. O resultado contraria a previsão:

Protocolo	Acurácia	F1 macro
Holdout por sessão (completo)	0,1373	0,1950
Holdout por sessão sem temperatura	0,1289	0,1653

Remover a *feature* **piorou** o resultado. A *medição* do carimbo de campanha permanece válida; a *inferência causal* de que a temperatura causa o *gap* estava errada.

8.6.4 Por que remover não resolve: o vazamento é sistêmico

Aplicando a mesma decomposição de variância às 18 grandezas brutas, o quadro fica claro:

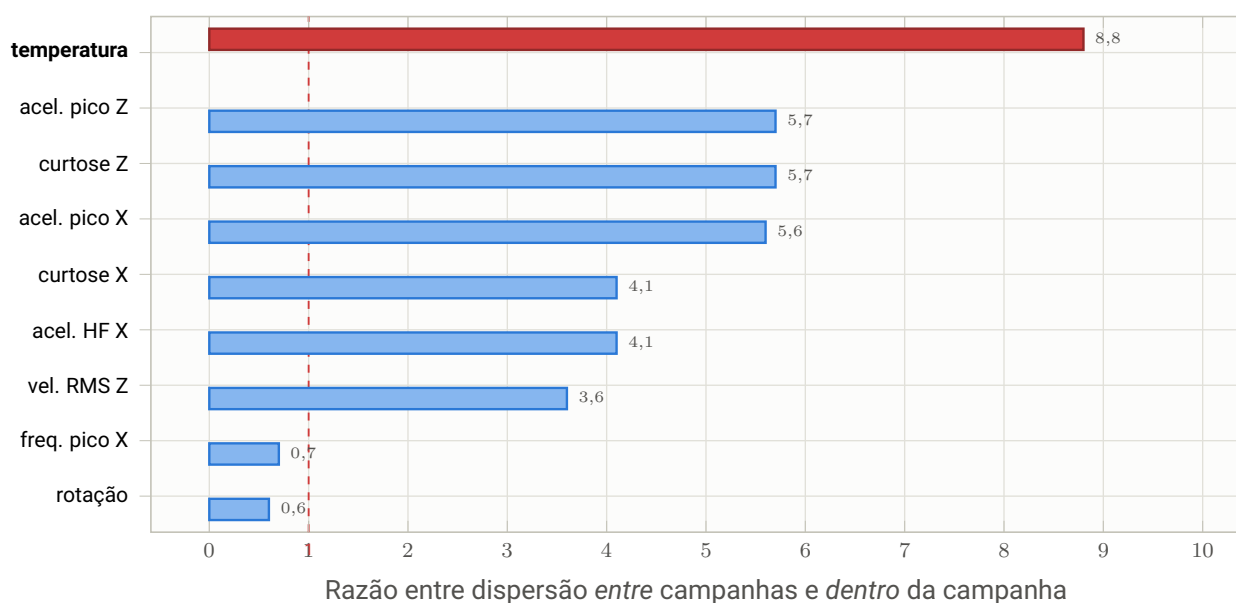


Figura 8.3 – Razão entre a dispersão *entre* campanhas de coleta e *dentro* de cada campanha, por *feature*. A linha tracejada em 1,0 é o limiar: acima dela, a *feature* carrega mais informação sobre *qual campanha* do que sobre *qual condição*. **Dezesseis das dezoito** grandezas estão acima. A temperatura (em vermelho) é a mais extrema, mas não é a causa — é o sintoma mais visível.

Conclusão revista. O gap de 0,89 para 0,14 não decorre de uma *feature* específica: decorre de cada condição experimental ter sido coletada em uma **única campanha**, o que confunde *sessão* com *rótulo* em praticamente todo o espaço de *features*. Retirar a *feature* mais extrema apenas faz o modelo migrar para a próxima variável *proxy*, e ainda perde o sinal físico legítimo que a temperatura carrega — aquecimento de mancal é sintoma real, segundo os manuais. Nenhuma seleção de *features* corrige isso: é limitação do desenho experimental, e a correção é de *coleta*, não de modelagem.

8.6.5 Mitigações no desenho atual

Diante desse teto, três decisões de projeto tornam o sistema defensável:

- 1. A busca por similaridade é a resposta operacional principal.** Em vez de *afirmar* uma classe, o sistema mostra ocorrências próximas do histórico completo e deixa a decisão com o técnico. É a saída mais defensável sob esse teto — e é literalmente o que o enunciado pede.
- 2. O gate de confiança sinaliza incerteza** em lugar de afirmar com falsa segurança (Seção 8.7).
- 3. O painel expõe os três protocolos** e a evidência do vazamento. Um painel que anunciasse apenas os 89 % da divisão aleatória seria uma promessa que o sistema não cumpre em produção.

8.6.6 Encaminhamentos propostos

#	Encaminhamento
1	Coletar múltiplas campanhas por condição — única correção que ataca a causa. Sem isso, qualquer métrica de generalização fica limitada por construção.
2	Normalizar contra a linha de base da própria máquina em vez de remover <i>features</i> : substituir valores absolutos pelo desvio contra a referência recente do equipamento. Essa referência existe em produção, ao contrário da identidade da campanha — o que remove o deslocamento de sessão preservando o sinal.
3	Adotar <i>features</i> invariantes à condição operacional : razões entre eixos e entre bandas, e a derivada da amplitude em relação a ω^2 (Seção 8.8.2).
4	Com acesso à forma de onda, extrair envelope e espectro — sem isso as frequências características de rolamento são inalcançáveis (Randall; Antoni, 2011).
5	Instrumentar explicabilidade por atribuição de contribuição (por exemplo, valores de Shapley (Lundberg; Lee, 2017)) para expor ao técnico <i>quais</i> grandezas sustentaram cada diagnóstico — hoje a interface mostra a importância global, não a local.

8.7 Gate de confiança

A confiança combina duas evidências independentes:

$$\text{escore} = p(\hat{c} \mid x) \times \frac{1}{k} \sum_{i=1}^k \mathbb{I}[\text{família}(v_i) = \hat{c}] \quad (8.1)$$

em que $p(\hat{c} \mid x)$ é a probabilidade atribuída pelo classificador à classe predita e o segundo fator é a fração dos $k = 25$ vizinhos que pertencem a essa mesma classe. A classificação final segue os limiares:

Escore	Rótulo	Leitura operacional
$\geq 0,70$	alta	Classificador e histórico concordam. O diagnóstico pode orientar intervenção.
$[0,40; 0,70)$	media	Concordância parcial. Vale inspecionar os vizinhos retornados antes de agir.
$< 0,40$	baixa	Evidências divergem. A interface sinaliza diagnóstico incerto; a prescrição, quando emitida, deve ser lida como hipótese.

O produto — e não a média — é deliberado: ele exige que *ambas* as evidências sejam favoráveis. Probabilidade alta com vizinhos discordantes indica que o classificador está extrapolando para uma região do espaço pouco povoada, e é exatamente o caso em que a resposta merece ressalva.

8.8 Análise exploratória que fundamentou o produto

As três análises seguintes não são ilustrativas: cada uma alterou uma decisão de produto. Todas são reproduzíveis pelos *endpoints* de analítica.

8.8.1 A severidade é comparável entre famílias?

Pergunta. O time de manutenção pergunta “esta vibração é grave?”. A resposta natural seria ordenar as famílias pela velocidade eficaz. Isso é válido?

Resultado sem condicionar pelo regime de rotação:

Família	RMS mediano X (mm/s)	p95
polia	2,77	3,61
desbalanceamento	2,56	3,98
ventoinha	2,53	4,35
rolamento_inner	2,42	3,09
desalinhamento	2,32	3,01
cocked_rotor	2,30	3,00
normal	2,29	3,13
correia	2,22	2,74

Achado. A faixa inteira cabe entre 2,2 e 2,8 mm/s, e o estado **normal** fica *no meio* do ranqueamento. Um gráfico de “severidade por família” com esses números afirmaria que operar normalmente é mais grave que uma correia defeituosa. É artefato de agregação: as famílias não estão igualmente distribuídas entre os regimes de rotação, e a rotação domina a amplitude.

Nenhum painel de severidade agrega regimes de rotação. Toda leitura é estratificada por rotação, e o gráfico oferece a alternativa **relativa à linha de base** — o estado **normal** do mesmo regime — que é a única comparação com significado físico. As zonas normati-

vas (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1995) são sobrepostas porque são a linguagem do mantenedor, mas com duas ressalvas explícitas: a classe da máquina é selecionável, não fixada; e trata-se de bancada com falhas induzidas, operando deliberadamente fora da faixa saudável.

8.8.2 Os dados obedecem à física dos manuais?

Pergunta. O procedimento de desbalanceamento afirma $F = m r \omega^2$: a força cresce com o *quadrado* da rotação. Se os dados forem fisicamente coerentes, o desbalanceamento deve escalar com a rotação muito mais rápido que as demais falhas. Isso é verificável – e é um teste de sanidade do conjunto de dados inteiro.

Tabela 8.18 – Velocidade RMS mediana do desbalanceamento por regime de rotação

Rotação	Desbalanceament	Linha de base ()	Severidade relativa
500 rpm	2,52	2,35	1,08×
1.000 rpm	2,11	2,05	1,03×
2.000 rpm	3,45	2,66	1,30×
3.000 rpm	7,50	2,83	2,65×

As demais falhas a 3.000 rpm ficam entre 2,5 e 3,4 mm/s.

Achado. A previsão do manual se confirma: o desbalanceamento é indistinguível do normal em baixa rotação e dispara em alta. Nenhuma outra família apresenta esse comportamento.

Virou painel próprio (“Validação física”), com as demais famílias em cinza como referência. Cumpre dois papéis: valida a qualidade do conjunto de dados e demonstra entendimento do domínio, não apenas correlação. A implicação diagnóstica, registrada como trabalho futuro: a assinatura útil do desbalanceamento não é a amplitude, é a *derivada* da amplitude em relação a ω^2 – uma *feature* do tipo $d(\text{RMS})/d(\omega^2)$ por máquina provavelmente separaria essa falha muito melhor que qualquer indicador instantâneo.

8.8.3 O que realmente distingue cada falha?

Pergunta. Os manuais associam falhas a assinaturas específicas – curtose e fator de crista altos para impactos de rolamento, $1 \times$ para desbalanceamento, $1 \times +2 \times$ para desalinhamento. Essas assinaturas aparecem nestes dados?

Método. Duas medidas complementares. A *assinatura* é o escore padronizado da média de cada família contra a média global, o que põe grandezas de escalas heterogêneas ($^{\circ}\text{C}$, g , mm/s , Hz) na mesma régua. O *poder discriminativo* é a razão entre a variância *entre* famílias e a variância *dentro* das famílias, no espírito da estatística F :

$$F = \frac{\sum_i n_i (\bar{x}_i - \bar{x})^2 / (k - 1)}{\sum_i (n_i - 1) s_i^2 / (N - k)} \quad (8.2)$$

Feature	Razão F
velocidade RMS X	1.193,6
velocidade de pico X	1.193,6
frequência de pico X	1.106,8
temperatura	885,3
...	...
rotação	208,3
curtose Z	189,2
aceleração de pico Z	188,2

Três achados.

- 1. Curtose e fator de crista não discriminam.** A curtose fica entre 2,65 e 2,75 e o fator de crista entre 3,73 e 3,85 em *todas* as famílias — inclusive as de rolamento, justamente onde os manuais previam impactos. Causa provável: as métricas vêm pré-agregadas pelo sensor em janelas longas, que suavizam os impulsos que a curtose deveria capturar.
- 2. Velocidade RMS e velocidade de pico em X têm F idêntico (1.193,6):** são colineares, porque o pico é aproximadamente o produto do RMS pelo fator de crista, e o fator de crista é quase constante. Uma das duas é redundante.
- 3. A frequência de pico é quase degenerada:** 61 Hz responde por 48,7 % dos registros (81.194 de 166.796). A *feature* derivada de ordem fica presa em 3,66 para quase todas as famílias a 1.000 rpm — ou seja, as assinaturas de ordem ($1\times$, $2\times$) dos manuais **não são recuperáveis** a partir de um único valor de pico. Precisariam do espectro completo.

O painel de assinaturas mostra o mapa de calor **ordenado pelo poder discriminativo** e marca em cinza os indicadores fracos, com legenda explicando o descompasso frente aos manuais. Esconder isso seria mais bonito e menos verdadeiro.

8.9 Limitações do componente de aprendizado

- **Uma única máquina.** Todos os dados vêm de uma bancada; nada foi validado entre equipamentos distintos.
- **Métricas pré-agregadas.** Sem forma de onda, análise de envelope e frequências características de rolamento são impossíveis.
- **Rótulos por campanha.** Não há repetição independente da mesma condição em campanhas diferentes para a maioria das famílias — o teto de generalização é imposto pelo desenho experimental.
- **Janela curta (47 dias).** Não há tendência de degradação de longo prazo observável; a distribuição temporal reflete o cronograma de ensaios, não a vida do equipamento.
- **Sem custo por falha.** A priorização documental usa frequência $\times$ severidade vibracional como *proxy*; com custo de parada e criticidade do ativo, ficaria melhor.

Recuperação documental, geração e agente

Este capítulo documenta a camada que transforma um diagnóstico em *instrução de correção*. É onde o requisito mais exigente do enunciado — responder apenas sobre falhas documentadas — é tecnicamente garantido.

9.1 Visão do pipeline

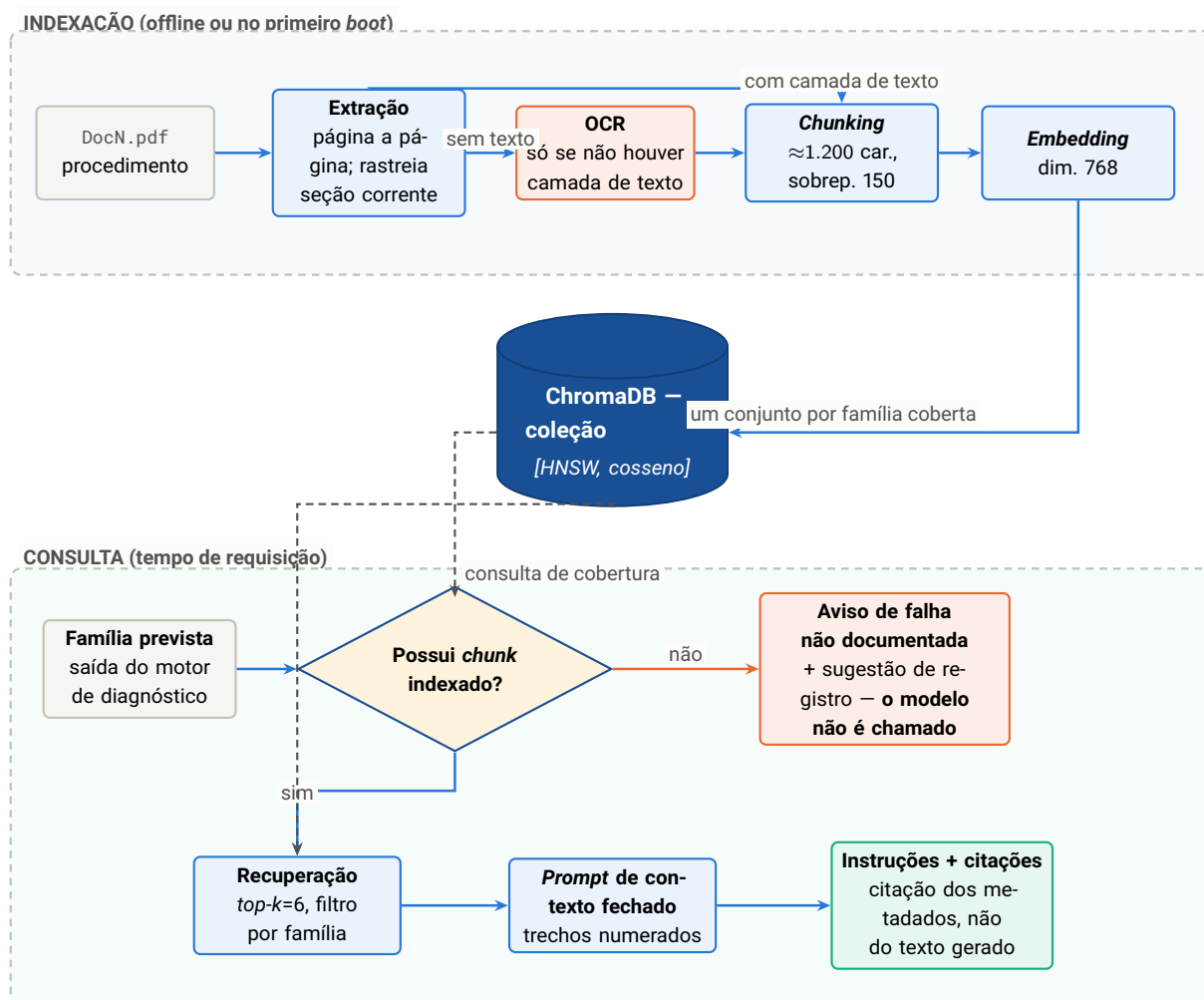


Figura 9.1 – Pipeline completo de recuperação e geração. As duas trilhas compartilham a mesma base vetorial, que cumpre dois papéis: repositório dos trechos e fonte de verdade da cobertura documental. O ramo laranja é o caminho exigido pelo enunciado – e nele o modelo de linguagem não é invocado.

9.2 Ingestão documental

9.2.1 Extração e chunking

A extração percorre o PDF página a página, mantendo o rastreo da seção corrente por reconhecimento de títulos numerados no padrão **N. Título** ou **N.N Título**. Os parágrafos são acumulados em *chunks* de aproximadamente 1.200 caracteres com sobreposição de 150.

Parâmetro	Valor	Justificativa
Tamanho do <i>chunk</i>	≈1.200 caracteres	Grande o bastante para conter um passo de procedimento completo com seu contexto; pequeno o bastante para que o <i>top-k</i> de 6 trechos caiba confortavelmente no <i>prompt</i> .

Sobreposição	150 caracteres	Evita que um passo de procedimento seja cortado exatamente na fronteira entre dois <i>chunks</i> , o que o tornaria irrecuperável.
Descarte mínimo	80 caracteres	Fragmentos menores são cabeçalhos, números de página e rodapés — ruído que degradaria a busca.
Metadados	doc, título, seção, página, família	São a citação. O usuário vê estes campos, não uma referência produzida pelo modelo.

9.2.2 OCR: o caso do documento escaneado

Um dos procedimentos da base inicial é um PDF escaneado, sem camada de texto — situação comum em acervos industriais, onde procedimentos antigos existem apenas em papel digitalizado. A extração convencional devolve zero *chunks*.

O tratamento é um *fallback* explícito: se a extração de texto não produzir *chunk* algum e houver um transcritor disponível, cada página é renderizada em imagem a 150 dpi e transcrita por modelo multimodal, com instrução para preservar títulos numerados em linhas próprias — o que mantém o rastreo de seção funcionando sobre o texto transcrito.

O OCR é *condicional*, não incondicional. Aplicá-lo sempre custaria uma chamada de modelo por página em todo documento — caro e mais sujeito a erro que a extração direta, que é exata quando a camada de texto existe. A degradação só ocorre quando é necessária.

9.3 *Embeddings* e base vetorial

Aspecto	Decisão e justificativa
Modelo	gemini-embedding-2 , multilíngue, contexto de 8.192 <i>tokens</i> .
Dimensionalidade	768, obtida por truncamento de representação aninhada (Kusupati et al., 2022). A propriedade dessas representações é que os primeiros componentes concentram a informação mais relevante, de modo que truncar não exige retreino e degrada pouco a qualidade — em troca de um quarto do armazenamento e da latência de busca.

Tipo de tarefa	<code>RETRIEVAL_DOCUMENT</code> na indexação e <code>RETRIEVAL_QUERY</code> na consulta. A assimetria é intencional: documento e pergunta são objetos linguísticos diferentes, e o modelo os projeta em espaços alinhados mas com condicionamento distinto.
Métrica e índice	Cosseno sobre índice HNSW (Malkov; Yashunin, 2020), que oferece busca aproximada com complexidade logarítmica — irrelevante nos 268 <i>chunks</i> atuais, decisivo em escala de milhares de procedimentos.
Lote de requisição	Uma instância por requisição. O serviço aceita lista, mas retorna silenciosamente apenas um vetor; o cliente valida explicitamente que o número de vetores devolvidos é igual ao de textos enviados e levanta exceção caso contrário.

A validação de contagem de *embeddings* parece defensiva em excesso até se considerar a falha que ela previne: um desalinhamento silencioso entre textos e vetores associaria o *embedding* de um trecho ao conteúdo de outro. O resultado seria um sistema que recupera trechos errados com total confiança — e sem nenhuma mensagem de erro.

9.4 O guardrail de cobertura

9.4.1 Definição operacional

Uma família de falha é considerada **documentada** se, e somente se, existe ao menos um *chunk* indexado na base vetorial cujo metadado `family` é igual ao nome canônico dessa família.

Trata-se de uma consulta ao índice, com filtro e limite 1 — não de uma inferência. Três consequências decorrem dessa formulação:

1. **É verificável.** O estado da cobertura é inspecionável a qualquer momento pelos *endpoints* de saúde e de documentos.
2. **É dinâmica.** Cadastrar um procedimento altera a cobertura na requisição seguinte, sem reinício de serviço e sem edição de configuração.
3. **Não pode ser contornada pelo modelo.** A verificação ocorre no fluxo de controle, antes da montagem do *prompt*. Não existe caminho de código em que o modelo seja chamado para uma família descoberta.

9.4.2 Cobertura inicial e as lacunas deliberadas

Tabela 9.3 – Cobertura documental inicial

Documento	Famílias cobertas	Tema
Doc1.pdf	<code>rolamento_inner</code> , <code>rolamento_outer</code> , <code>rolamento_ball</code> , <code>rolamento_combination</code>	Diagnóstico e correção em rolamentos (documento escaneado – exige OCR)
Doc2.pdf	<code>desalinhamento</code>	Correção de desalinhamento em motor elétrico
Doc3.pdf	<code>desbalanceamento</code>	Correção de desbalanceamento em máquinas rotativas
Doc4.pdf	<code>correia</code>	Transmissão por correias
Doc5.pdf	<code>polia</code>	Polias de sistemas rotativos
Doc6.pdf	<code>cocked_rotor</code>	Rotor inclinado
Sem documento – acionam o fluxo de falha não documentada		
—	<code>eccentric_rotor</code>	16.497 ocorrências — maior lacuna documental do acervo
—	<code>ventoinha</code>	12.299 ocorrências
—	<code>falta_fase</code>	800 ocorrências — e a classe que o modelo melhor reconhece

Nove das doze famílias de falha estão cobertas por 268 *chunks*. As três lacunas não são acidentais: elas permitem demonstrar o comportamento exigido pelo enunciado com dados reais, e alimentam a matriz de priorização documental do painel (Seção 11.3.1).

9.5 Anti-alucinação: cinco barreiras

A geração de conteúdo plausível mas não sustentado pela fonte é a falha característica de modelos de linguagem (Ji et al., 2023). O padrão de recuperação aumentada (Lewis et al., 2020) ataca a raiz do problema — substituir conhecimento paramétrico por conhecimento recuperado — mas não é suficiente por si só. O sistema aplica cinco barreiras em série.

Tabela 9.5 – Barreiras anti-alucinação

#	Barreira	Mecanismo
1	<i>Guardrail</i> determinístico	O modelo só é chamado se a família possui trechos indexados. A cobertura nunca é decidida pelo modelo.
2	Contexto fechado	A instrução de sistema exige responder somente com base nos trechos fornecidos e declarar explicitamente quando eles não cobrem o ponto perguntado.
3	Citação determinística	Documento, seção e página vêm dos metadados do trecho recuperado, não do texto gerado. Citação alucinada é estruturalmente impossível.
4	Temperatura baixa	<code>temperature=0.2</code> na geração; <code>0.0</code> na transcrição por OCR, onde qualquer criatividade é defeito.
5	Confiança dupla exposta	Probabilidade × concordância dos vizinhos é apresentada ao usuário; abaixo do limiar, a interface marca o diagnóstico como incerto.

9.5.1 A instrução de sistema

Listing 9.1 – Instrução de sistema do assistente prescritivo — `src/llm/prompts.py`

```

1  SYSTEM_PRESCRIPTIVE = """Voce e um assistente tecnico de manutencao industrial de maquinas rotativas.
   ↳
2  Regras OBRIGATORIAS:
3  1. Responda SOMENTE com base nos trechos de documentos fornecidos no contexto.
4  2. Se a informacao nao estiver no contexto, diga explicitamente que os documentos
5     disponiveis nao cobrem esse ponto.
6  3. Ao usar um trecho, referencie-o pelo marcador [n] correspondente.
7  4. Nunca invente procedimentos, valores de tolerancia, normas ou frequencias que
8     nao estejam no contexto.
9  5. Responda em portugues do Brasil, em Markdown, com passos numerados quando
10     descrever procedimentos.
11  6. Priorize sempre a secao de seguranca antes de qualquer intervencao."""

```

A sexta regra não é anti-alucinação — é segurança operacional. Um procedimento de manutenção que descreve a intervenção antes do bloqueio e etiquetagem do equipamento está tecnicamente correto e operacionalmente perigoso.

9.5.2 Estrutura obrigatória da prescrição

O *prompt* de prescrição exige cinco seções, na ordem em que um técnico as executa: **segurança** (passos obrigatórios antes da intervenção), **diagnóstico** (como confirmar a falha, incluindo as

assinaturas de vibração citadas nos trechos), **correção** (procedimento passo a passo), **validação** (como verificar e quais são os critérios de aceitação) e **prevenção**. A estrutura fixa torna a saída comparável entre falhas e auditável.

9.6 Agente de conversa

9.6.1 Fronteira arquitetural

O agente existe **apenas** no *endpoint* de conversa (**ADR-11**). O caminho crítico de diagnóstico — usado pelo *worker* MQTT, e portanto por toda a planta — permanece determinístico. A razão é direta: um laço agêntico tem número de passos variável, latência imprevisível e liberdade de decisão que enfraqueceria exatamente as garantias descritas na seção anterior. Onde há garantia a preservar, não há agente.

9.6.2 Ferramentas

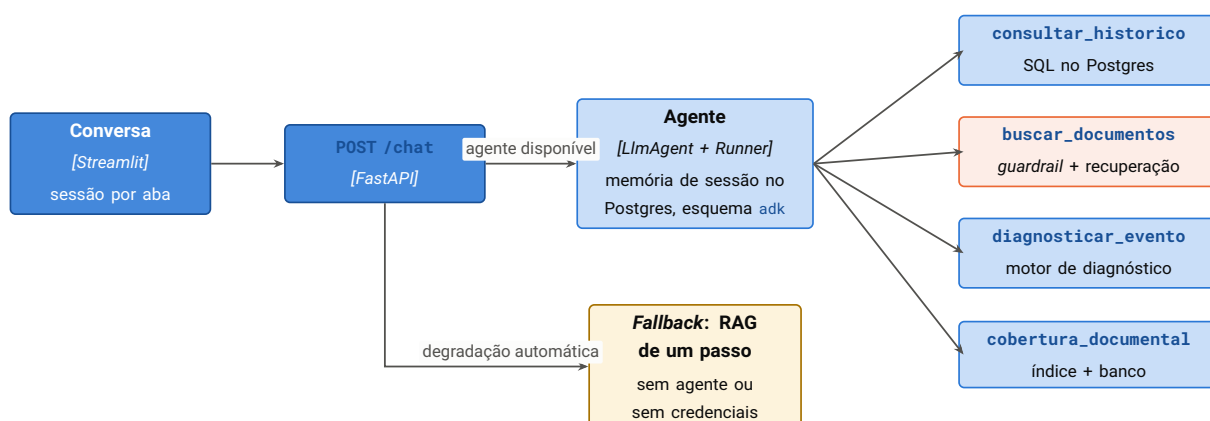


Figura 9.2 – Agente de conversa e suas quatro ferramentas. Em laranja, a ferramenta que carrega o *guardrail*: a verificação de cobertura acontece *dentro* dela, antes da recuperação, de modo que o modelo recebe um aviso estruturado em vez de trechos — e apenas o transmite.

Tabela 9.7 – Ferramentas do agente

Ferramenta	Contrato e comportamento
<code>consultar_historico(dias)</code>	Consulta agregada ao histórico. Valida a família contra a taxonomia e devolve a lista de famílias válidas em caso de erro – o que permite ao modelo se autocorriger. A janela temporal é ancorada no evento <i>mais recente do banco</i> , não na data corrente, o que mantém a resposta coerente em um conjunto de dados histórico.
<code>buscar_documentos(familia)</code>	Obrigatória antes de descrever qualquer procedimento. Aplica o <i>guardrail</i> : família sem cobertura devolve <code>documentado=false</code> e o aviso pronto. Família coberta devolve os trechos numerados com origem. Registra as citações no coletor de contexto.
<code>diagnosticar_evento(json)</code>	Executa o motor de diagnóstico sobre um JSON colado pelo usuário. JSON inválido devolve os erros de validação em forma legível, não uma exceção.
<code>cobertura_documentos()</code>	Lista famílias documentadas, famílias sem documento e documentos cadastrados com data de ingestão.

9.6.3 Citações determinísticas no agente

Em um agente, a dificuldade adicional é que a recuperação acontece dentro de uma ferramenta, várias camadas abaixo do código que monta a resposta HTTP. A solução adotada é um coletor por contexto de execução (*context variable*): a ferramenta registra ali os metadados dos trechos que efetivamente recuperou, e o *endpoint* lê o coletor ao final da execução.

O coletor por contexto de execução é preferível a uma variável global porque é isolado por tarefa assíncrona: duas conversas simultâneas não misturam citações. E é preferível a inspecionar o texto gerado pelo modelo em busca de marcadores, porque preserva a propriedade central – a citação exibida reflete o que foi *recuperado*, não o que o modelo *escreveu*.

9.6.4 Degradação graciosa

A inicialização do agente pode falhar por três motivos: biblioteca ausente, projeto de nuvem não configurado ou credenciais inválidas. Em todos os casos a função de obtenção do agente devolve nulo, e o *endpoint* de conversa recorre ao RAG de um passo. A interface indica o modo em uso por um selo – “agente com ferramentas e memória” ou “recuperação direta” – para que o usuário saiba o que esperar.

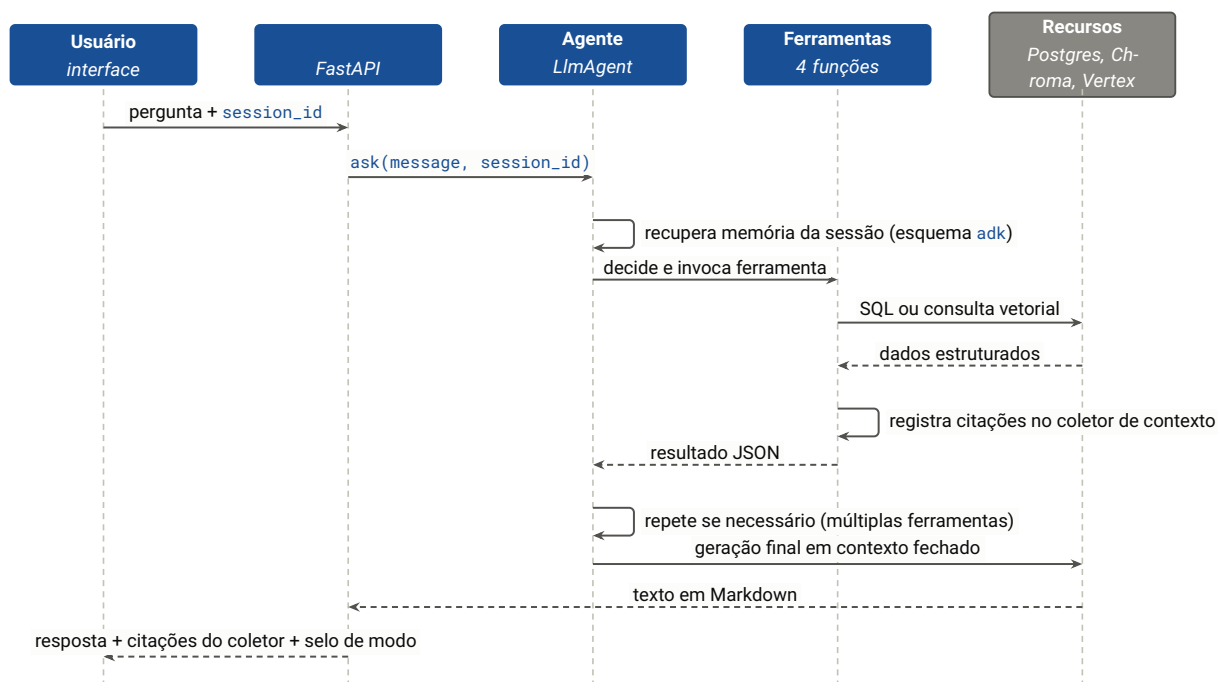


Figura 9.3 – Diagrama de sequência da conversa com agente. As citações devolvidas ao usuário vêm do coletor alimentado pelas ferramentas — não do texto gerado na última etapa.

Integração industrial por MQTT

Um sistema de manutenção que só responde a requisições HTTP disparadas por pessoas resolve metade do problema. No chão de fábrica, o dado nasce em um sensor e precisa chegar ao diagnóstico sem intervenção humana — e o diagnóstico precisa voltar à planta em um formato que o supervisor entenda. Este capítulo documenta esse ciclo.

10.1 Escolha do protocolo

MQTT é o protocolo de publicação e subscrição de fato em telemetria industrial ([OASIS, 2019](#)): cabeçalho mínimo, tolerância a rede instável, desacoplamento entre produtor e consumidor e níveis de garantia de entrega selecionáveis. O sistema o adota tanto para *entrada* de telemetria quanto para *saída* de alertas, o que fecha o ciclo pelo mesmo canal ([ADR-10](#)).

NOTA

OPC UA seria a escolha para comunicação direta com controladores lógicos programáveis, e Sparkplug B para padronizar a semântica da carga útil. Ambos estão registrados como evolução (Seção 17.2): o custo de instalação não se justificaria para demonstrar o ciclo, que MQTT demonstra integralmente.

10.2 Topologia de tópicos

Tabela 10.1 – Tópicos MQTT

Tópico	Direção	Conteúdo
<code>sensors/{maquina}/telemetry</code>	entrada	Evento de sensor no <i>mesmo</i> formato JSON aceito pela API. Assinado pelo <i>worker</i> com curinga: <code>sensors/+/telemetry</code> .
<code>sensors/{maquina}/alerts</code>	saída	Diagnóstico de falha: identificador do evento e da máquina, família prevista, probabilidade, confiança, indicação de cobertura documental e <i>timestamp</i> .
<code>sensors/{maquina}/dlq</code>	saída	<i>Payload</i> rejeitado, com motivo da rejeição (até 2.000 caracteres), conteúdo original (até 5.000) e <i>timestamp</i> .

O identificador da máquina é extraído do próprio tópico, o que permite atender múltiplos equipamentos sem alterar o *payload* nem a configuração — basta o *gateway* publicar no tópico correspondente.

10.3 Fluxo de ingestão

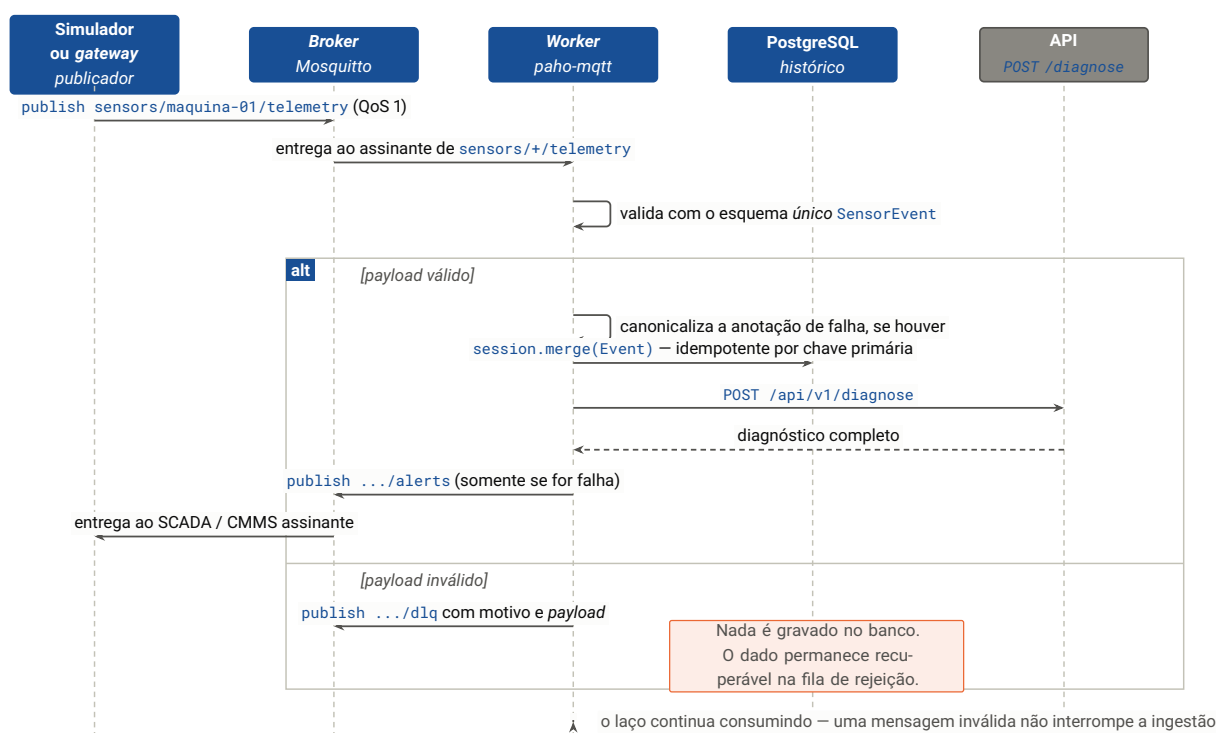


Figura 10.1 – Diagrama de sequência da ingestão industrial. A persistência precede o diagnóstico: se a API estiver indisponível, o evento já está no histórico e pode ser rediagnosticado depois — perder o diagnóstico é recuperável, perder o dado não é.

10.4 Garantias de entrega e processamento

Tabela 10.3 – Garantias da ingestão industrial

Garantia	Mecanismo e consequência
Nenhuma perda	QoS 1 (<i>at-least-once</i>) na subscrição e nas publicações. O <i>broker</i> retransmite até confirmar.
Nenhuma duplicata	A chave primária de <i>events</i> é o identificador de origem, e a persistência usa <i>merge</i> . Reentrega da mesma mensagem atualiza a linha existente em vez de criar outra. É a contrapartida obrigatória de QoS 1: garantia <i>at-least-once</i> no transporte exige idempotência no consumidor.
Nenhum descarte silencioso	<i>Payload</i> inválido vai para a fila de rejeição com o motivo. O problema fica visível e o dado recuperável.
Validação única	O <i>worker</i> usa o <i>mesmo</i> esquema pydantic da API. É impossível um <i>payload</i> ser aceito por um caminho e rejeitado pelo outro.
Tolerância a rótulo desconhecido	Diferente do ETL em lote, aqui um rótulo sem regra gera advertência em <i>log</i> e o evento é persistido <i>sem</i> família. Em ingestão contínua, perder o dado seria pior que perder o rótulo.
Continuidade	Falha de diagnóstico registra erro e o laço prossegue. Um evento problemático não interrompe a ingestão dos seguintes.

A assimetria no tratamento de rótulo desconhecido — interromper no ETL, tolerar no *worker* — é deliberada e reflete a diferença de contexto. No ETL há um operador presente que pode corrigir a regra e reexecutar; a integridade da base é o valor a proteger. Na ingestão contínua não há operador, e o dado bruto de vibração tem valor independentemente da anotação — que, em produção, normalmente não existe.

10.5 Segurança do *broker*

Um *broker* MQTT anônimo exposto à internet é varrido e abusado em questão de horas. A configuração é gerada em tempo de execução a partir de variáveis de ambiente:

- Com usuário e senha definidos: gera o arquivo de senhas, ajusta dono e permissão para `0600`, e escreve `allow_anonymous false`.
- Sem credenciais: registra advertência explícita em *log* e só então cai em modo anônimo — aceitável apenas quando o *broker* não está exposto.

A geração em *runtime* tem uma consequência de segurança relevante: as credenciais nunca entram na imagem de contêiner nem no repositório (**RNF-05**). A evolução natural é habilitar TLS, registrada na Seção 17.2.

10.6 Demonstração e verificação

O simulador reproduz o comportamento de um *gateway* de aquisição publicando eventos *reais* do conjunto de dados — não sintéticos.

Listing 10.1 – Roteiro de verificação da ingestão industrial

```

1 # 1. Infraestrutura local
2 docker compose up -d postgres mosquitto
3
4 # 2. Worker de ingestao (em outro terminal)
5 python -m src.ingestion.mqtt_worker
6
7 # 3. Replay de 5 eventos reais da familia cocked_rotor, 1 a cada 2 s
8 python scripts/simulate_sensor.py --family cocked_rotor --limit 5 --rate 2
9
10 # 4. Demonstracao da fila de rejeicao: publica um payload invalido
11 python scripts/simulate_sensor.py --invalid
12
13 # 5. Observar os topicos de saida
14 mosquitto_sub -h localhost -t 'sensors/+/alerts' -t 'sensors/+/dlq' -v

```

Tabela 10.5 – Resultado esperado da verificação

Passo	Observável	Critério
3	Log do worker	Cinco linhas de evento persistido e, para as falhas, uma linha de alerta publicado.
3	Tópico de alertas	Cinco mensagens JSON com família, probabilidade, confiança e indicação de cobertura.
3	Painel da interface	Os eventos aparecem nas agregações sem recarregar a aplicação — as consultas são executadas sob demanda.
4	Tópico de rejeição	Uma mensagem com o motivo da falha de validação e o <i>payload</i> original.
4	Banco de dados	Nenhuma linha nova em <code>events</code> .
3, repetido	Banco de dados	Republicar os mesmos cinco eventos não altera a contagem de linhas — comprovação da idempotência.

Interface do usuário

A interface é uma camada de apresentação estrita: consome exclusivamente a API e não contém lógica de domínio nem acesso a banco (**ADR-06**). Este capítulo documenta sua arquitetura de informação, os painéis analíticos e as decisões de visualização — que foram tomadas com validação, não por gosto.

11.1 Arquitetura de informação

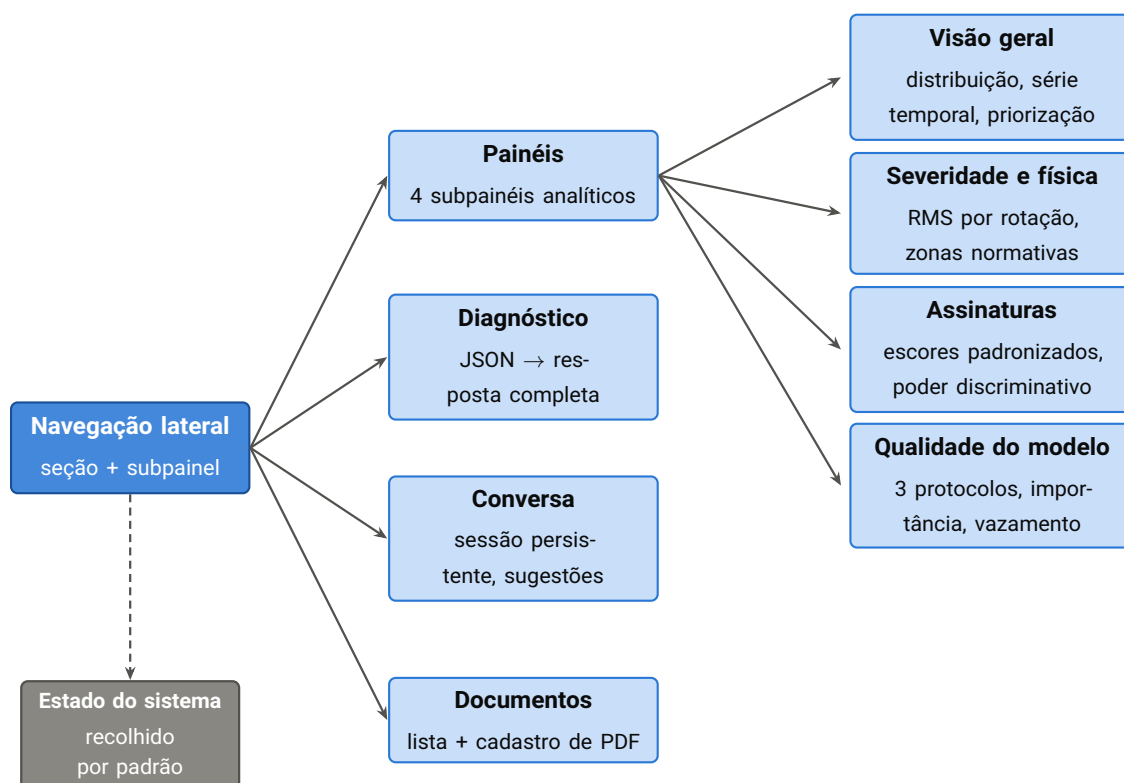


Figura 11.1 – Arquitetura de informação. A navegação renderiza apenas a seção ativa, e cada painel requisita somente os dados de que precisa — não há carga antecipada do conjunto completo.

11.2 Seção de diagnóstico

Recebe o JSON do evento — com botão para carregar o exemplo do enunciado — e apresenta a resposta em quatro blocos:

1. **Cabeçalho de métricas:** tipo de defeito, probabilidade, concordância dos vizinhos e selo de confiança, colorido conforme o nível.
2. **Ocorrências históricas:** total registrado, frequência diária, período de observação e a distribuição temporal em gráfico de barras.
3. **Instruções de correção** em Markdown, com as fontes citadas em seção expansível — documento, seção e página.
4. **Ou** o aviso de falha não documentada com a sugestão de registro, em destaque de atenção.

Quando o diagnóstico aponta estado operacional, a interface informa que não há ação prescritiva a tomar — e não exibe seção de prescrição alguma, evitando a sugestão implícita de que algo precisa ser feito.

11.3 Painéis analíticos

Cada gráfico do painel entrou por responder a uma pergunta operacional específica e por ter passado no teste de não enganar quem o lê. A fundamentação estatística está no Capítulo 8; aqui registra-se a tradução em produto.

11.3.1 Priorização documental

Pergunta. O sistema só prescreve correção para falhas documentadas. Quais lacunas custam mais caro?

Resposta. Uma matriz de bolhas que cruza três dimensões: frequência (contagem por família) no eixo horizontal, severidade (máxima severidade relativa entre regimes de rotação) no vertical, e cobertura documental na cor. O canto superior direito em laranja é a fila de trabalho da engenharia de manutenção.

Família sem documento	Ocorrências	Situação
<code>eccentric_rotor</code>	16.497	Maior lacuna do acervo — segunda família mais frequente do histórico.
<code>ventoinha</code>	12.299	Sem procedimento cadastrado.
<code>falta_fase</code>	800	Sem procedimento, mas é a classe que o modelo reconhece melhor ($F1 = 0,940$).

11.3.2 Severidade e validação física

O painel apresenta a velocidade eficaz **sempre estratificada por regime de rotação**, com dois modos de leitura: absoluto, com as zonas normativas sobrepostas e classe de máquina selecionável; e relativo à linha de base do mesmo regime. O gráfico de validação física destaca o desbalanceamento contra as demais famílias em cinza, evidenciando a dependência quadrática com a rotação prevista pelo manual (Seção 8.8.2).

11.3.3 Assinaturas

Mapa de calor dos escores padronizados por família e por *feature*, **ordenado pelo poder discriminativo** — não pela ordem das colunas no conjunto de dados. Indicadores fracos aparecem em cinza, com legenda explicando o descompasso frente ao que os manuais previam (Seção 8.8.3).

11.3.4 Qualidade do modelo

Apresenta os três protocolos de avaliação lado a lado, o F1 por classe, a importância das *features* com a temperatura destacada e o gráfico de vazamento por *feature*.

Exibir os três protocolos é uma decisão de produto, não de estética. Um painel que mostrasse apenas os 89 % da divisão aleatória comunicaria uma capacidade que o sistema não tem em produção — e a primeira vez que um técnico confiasse nele e errasse, a credibilidade da ferramenta inteira estaria perdida.

11.4 Decisões de visualização

As cores não foram escolhidas por preferência: cada conjunto passou por um validador de paleta que verifica banda de luminosidade, piso de croma, separação para deficiências de visão de cores e contraste contra a superfície, nos modos claro e escuro. A diferença perceptual é medida em CIEDE2000 (Sharma; Wu; Dalal, 2005), com piso de $\Delta E \geq 6$ sob simulação de deuteranopia, conforme a prática recomendada para material acessível (Okabe; Ito, 2008).

Tabela 11.2 – Codificações visuais e sua justificativa

Codificação	Escolha	Justificativa
Documentada × sem documento	Azul #2a78d6 / laranja #eb6834	A escolha intuitiva verde/vermelho falhou o teste: $\Delta E = 4,1$ sob deuteranopia, abaixo do piso de 6. Azul/laranja atinge $\Delta E = 24,7$. A legenda usa ainda ícone como codificação redundante.
Regime de rotação	Rampa ordinal de um único tom (azul, claro para escuro)	Rotação é grandeza ordenada , não identidade. Matizes categóricos sugeririam categorias sem ordem, o que é uma afirmação falsa (Munzner, 2014).
Escore padronizado	Divergente azul ↔ vermelho, cinza no meio	O escore expressa polaridade — acima ou abaixo da média global. Uma escala arco-íris destruiria essa leitura.
Famílias no gráfico de física	Cinza mais um destaque	Doze séries excedem qualquer paleta categórica segura. Destaque sobre referência cinza comunica melhor do que doze cores que ninguém distingue.
Severidade	Percentis (50, 95, 99), nunca média	As distribuições têm cauda longa; a média seria puxada por transientes de partida e daria uma impressão falsa de severidade típica.

NOTA

O registro do *erro evitado* é a parte mais útil desta tabela. A codificação verde/vermelho para “documentado / não documentado” é a primeira escolha de qualquer pessoa, é usada em incontáveis painéis e é inacessível para aproximadamente 8% dos homens. O teste custou minutos; a correção, uma linha de código.

11.5 Estado do sistema

A barra lateral apresenta, em bloco recolhido por padrão, o estado corrente: modelo de linguagem e de *embedding* configurados, presença dos artefatos de aprendizado de máquina e número de famílias documentadas. Se a API estiver inacessível, a interface interrompe a renderização com mensagem acionável contendo o endereço configurado — em vez de exibir painéis vazios que parecem indicar ausência de dados.

Especificação da API

A API é a única porta de entrada de domínio (**ADR-06**). Todos os *endpoints* estão sob o prefixo [/api/v1](#), e a especificação OpenAPI é gerada automaticamente a partir dos esquemas de domínio, disponível em [/docs](#).

12.1 Catálogo de *endpoints*

Tabela 12.1 – Catálogo de *endpoints* da API

Método	Caminho	Função	Requisito
GET	/health	Estado do serviço, artefatos e cobertura	RF-15
POST	/diagnose	Diagnóstico completo de um evento	RF-01, RF-02, RF-03, RF-04
POST	/chat	Conversa prescritiva com agente e ferramentas	RF-07
GET	/stats	Contagem por família e série mensal de falhas	RF-14
GET	/events	Eventos mais recentes, com filtro por família	RF-13
GET	/documents	Documentos ativos e cobertura documental	RF-06
POST	/documents	Cadastro e indexação de novo procedimento	RF-05
GET	/analytics/severity	Severidade por família e regime de rotação	RF-12
GET	/analytics/signatures	Assinaturas e poder discriminativo	RF-12
GET	/analytics/model-quality	Métricas, importância e evidência de vazamento	RF-12

12.2 POST /api/v1/diagnose

É o *endpoint* central do sistema: recebe o evento e devolve as três respostas exigidas pelo enunciado.

12.2.1 Requisição

Corpo em JSON, validado pelo esquema [SensorEvent](#). As 18 grandezas de sensor são **obrigatórias**; `id`, `created_at` e `fault` são opcionais. Campos extras são tolerados e ignorados — o que permite enviar o registro do conjunto de dados original sem remover as colunas redundantes.

Listing 12.1 – Requisição — evento de exemplo do enunciado

```
1 {
2   "id": 114387,
3   "created_at": "2026-06-01 21:32:53.911176+00:00",
4   "z_rms_velocity_mm_s": 1.517,
5   "x_rms_velocity_mm_s": 2.0,
6   "temperature_c": 24.69,
7   "z_peak_acceleration_g": 0.484,
8   "x_peak_acceleration_g": 0.631,
9   "z_peak_vel_comp_freq_hz": 61.0,
10  "x_peak_vel_comp_freq_hz": 61.0,
11  "z_rms_acceleration_g": 0.09,
12  "x_rms_acceleration_g": 0.114,
13  "z_kurtosis": 2.392,
14  "x_kurtosis": 2.77,
15  "z_crest_factor": 3.747,
16  "x_crest_factor": 4.269,
17  "z_peak_velocity_mm_s": 2.146,
18  "x_peak_velocity_mm_s": 2.829,
19  "z_high_freq_rms_accel_g": 0.129,
20  "x_high_freq_rms_accel_g": 0.147,
21  "fault": "cocked_rotor_2",
22  "rpm": 1000.0
23 }
```

12.2.2 Resposta

Campo	Tipo	Significado
<code>predicted_fault</code>	<code>string</code>	Nome canônico da família prevista.
<code>is_fault</code>	<code>boolean</code>	Falso para estados operacionais — e então não há prescrição.
<code>probability</code>	<code>float</code>	Probabilidade da classe predita.
<code>knn_agreement</code>	<code>float</code>	Fração dos 25 vizinhos na família prevista.
<code>confidence</code>	<code>string</code>	<code>alta</code> , <code>media</code> ou <code>baixa</code> .

<code>similar_events.count</code>	<code>int</code>	Total de ocorrências históricas da família.
<code>similar_events.first_seen</code> <code>...last_seen</code>	<code>datetime</code>	Período de observação da família no histórico.
<code>similar_events.freq</code>	<code>float</code>	Frequência média diária no período.
<code>similar_events.timeline</code>	<code>list</code>	Série mensal [{month, count}].
<code>similar_events.neighbor_ids</code>	<code>list[int]</code>	Identificadores dos 25 eventos mais semelhantes — rastreáveis no banco .
<code>documented</code>	<code>boolean</code>	Resultado do <i>guardrail</i> de cobertura.
<code>prescription</code>	<code>object null</code>	<code>instructions_md</code> e <code>citations[{doc, section, page}]</code> . Nulo quando não documentada.
<code>suggestion</code>	<code>string null</code>	Aviso e orientação de registro. Nulo quando documentada.

Listing 12.2 – Resposta — falha documentada (campos de texto abreviados)

```

1  {
2    "predicted_fault": "cocked_rotor",
3    "is_fault": true,
4    "probability": 0.9203,
5    "knn_agreement": 0.96,
6    "confidence": "alta",
7    "similar_events": {
8      "count": 14275,
9      "first_seen": "2026-05-04T11:20:31.402000+00:00",
10     "last_seen": "2026-06-14T08:55:02.117000+00:00",
11     "freq_per_day": 344.58,
12     "timeline": [{ "month": "2026-05", "count": 9312 },
13                  { "month": "2026-06", "count": 4963 }],
14     "neighbor_ids": [114387, 114388, 114362, "..."],
15     "neighbor_agreement": 0.96
16   },
17   "documented": true,
18   "prescription": {
19     "instructions_md": "### 1. Seguranca\n1. Bloqueio e etiquetagem [1] ...",
20     "citations": [
21       { "doc": "Doc6.pdf", "section": "3.1 Verificacao previa", "page": 4 },
22       { "doc": "Doc6.pdf", "section": "4.2 Reassentamento", "page": 6 }
23     ]
24   },
25   "suggestion": null
26 }

```

Listing 12.3 – Resposta — falha **não** documentada (o modelo de linguagem não é chamado)

```

1  {
2    "predicted_fault": "ventoinha",
3    "is_fault": true,
4    "probability": 0.7412,
5    "knn_agreement": 0.72,

```

```

6  "confidence": "media",
7  "similar_events": { "count": 12299, "freq_per_day": 289.4, "...": "..."},
8  "documented": false,
9  "prescription": null,
10 "suggestion": "Ainda nao existe documento orientativo cadastrado para a falha
11  **Falha na ventoinha**. Sugestao: registre um novo documento tecnico para
12  este defeito na aba 'Documentos' (procedimento de diagnostico, correcao e
13  validacao). Assim que o documento for cadastrado, o sistema passara a gerar
14  as instrucoes de correcao automaticamente."
15 }

```

12.2.3 Códigos de estado

Código	Condição
200	Diagnóstico produzido — inclusive no caso de falha não documentada, que é resposta válida e não erro.
422	<i>Payload</i> não satisfaz o esquema. O corpo lista campo a campo o que está inconsistente.
500	Artefatos de aprendizado de máquina ausentes ou corrompidos. Diagnóstico prévio disponível em /health .

12.3 POST /api/v1/chat

Campo (requisição)	Tipo	Uso
message	string	Pergunta do usuário. Pode conter o JSON de um evento colado.
session_id	string null	Identificador da conversa. A memória é persistida por sessão; ausente, uma nova é criada.
fault_family	string null	Contexto do diagnóstico corrente. Usado somente pelo caminho de degradação.
history	list	Histórico da conversa. Usado somente pelo caminho de degradação — com agente, a memória vem do banco.

Resposta

answer_md	string	Resposta em Markdown.
citations	list	Fontes efetivamente recuperadas (doc , section , page).
documented	boolean	Falso quando o <i>guardrail</i> bloqueou a consulta documental.

<code>agent_used</code>	<code>boolean</code>	Verdadeiro se respondido pelo agente; falso se pelo caminho de degradação. A interface exibe o modo.
-------------------------	----------------------	--

Em caso de indisponibilidade do serviço de linguagem, o *endpoint* devolve `503` com mensagem **acionável**: quais variáveis configurar e qual comando de autenticação executar, além do tipo da exceção original.

12.4 POST /api/v1/documents

Requisição `multipart/form-data` com três partes:

Parte	Tipo	Conteúdo
<code>file</code>	arquivo	PDF do procedimento.
<code>title</code>	texto	Título técnico exibido nas citações.
<code>families</code>	texto	Famílias cobertas, separadas por vírgula. Validadas contra a taxonomia canônica.

Listing 12.4 – Cadastro de documento via linha de comando

```
1 curl -X POST http://localhost:8000/api/v1/documents \
2   -F 'file=@procedimento-ventoinha.pdf' \
3   -F 'title=Procedimento para Correcao de Falhas em Ventoinhas' \
4   -F 'families=ventoinha'
5 # -> {"filename":"procedimento-ventoinha.pdf","chunks":37,"families":["ventoinha"]}
```

Código	Condição
<code>200</code>	Documento indexado. A resposta traz a contagem de <i>chunks</i> e as famílias atendidas. A cobertura muda a partir da próxima requisição.
<code>422</code>	Família inexistente (a resposta lista as válidas), lista de famílias vazia, ou nenhum conteúdo extraível do PDF.

NOTA

A indexação é idempotente por nome de arquivo: reenviar o mesmo arquivo remove os *chunks* anteriores antes de reindexar. Isso permite corrigir e recadastrar um procedimento sem duplicar conteúdo na base vetorial — e sem deixar trechos órfãos da versão antiga concorrendo na busca.

12.5 GET /api/v1/health

Listing 12.5 – Resposta de saúde

```
1 {
2   "status": "ok",
3   "artifacts_loaded": true,
4   "documented_families": ["cocked_rotor", "correia", "desalinhamento",
5                           "desbalanceamento", "polia", "rolamento_ball",
6                           "rolamento_combination", "rolamento_inner",
7                           "rolamento_outer"],
8   "chat_model": "gemini-3.6-flash",
9   "embedding_model": "gemini-embedding-2"
10 }
```

O campo `documented_families` vem vazio enquanto a indexação inicial estiver em curso em uma implantação nova — o que é estado *parcial*, não falha, e por isso o *endpoint* continua devolvendo `200`. Se devolvesse erro, o orquestrador reprovaria e reiniciaria a instância indefinidamente, impedindo a indexação de terminar.

12.6 Endpoints de consulta e analítica

Endpoint	Parâmetros e retorno
GET /events	<code>limit</code> (padrão 50, teto 500) e <code>family</code> (nome canônico). Retorna os eventos mais recentes com identificador, instante, rótulo bruto, rotação, velocidades eficazes e temperatura.
GET /stats	Sem parâmetros. Retorna <code>per_family</code> (contagem por família, com indicação de falha), <code>monthly_faults</code> (série mensal apenas de falhas) e <code>documented_families</code> .
GET /documents	Sem parâmetros. Retorna documentos ativos com identificador, arquivo, título e data de ingestão, mais a cobertura corrente.
GET /analytics/severity	Retorna <code>records</code> — uma linha por família e regime de rotação, com percentis 50, 95 e 99 da velocidade eficaz em X, percentis em Z, temperatura mediana, contagem, linha de base e severidade relativa — e <code>baseline_by_rpm</code> .
GET /analytics/signatures	Retorna <code>signatures</code> (escore padronizado de cada família em cada <i>feature</i>), <code>discriminative_power</code> (razão <i>F</i> por <i>feature</i> , ordenada) e a lista de <i>features</i> .
GET /analytics/model-quality	Retorna <code>metrics</code> (conteúdo integral de <code>metrics.json</code>), <code>feature_importance</code> (percentual por <i>feature</i> , ordenado) e <code>leakage</code> (razão entre dispersão inter e intracampanha por <i>feature</i> , com o número de campanhas consideradas).

Os *endpoints* de análise dependem de recursos estatísticos do PostgreSQL — `percentile_cont`, `stddev_samp` e `var_samp` — que não existem em SQLite. Por isso os testes dessas funções exigem um esquema temporário no PostgreSQL e são *pulados* quando o banco não está disponível, em vez de falharem. Interpolação de nome de coluna nas consultas é restrita a uma lista branca derivada da constante de *features* do código; coluna fora da lista levanta exceção antes de a consulta ser montada.

Diagramas de atividade

Este capítulo reúne o diagrama de atividade de **todos** os processos do sistema – os automáticos, os manuais e os de operação. A notação é UML de atividades ([OBJECT MANAGEMENT GROUP, 2017](#)): círculo cheio para início, círculo com anel para término, retângulo arredondado para ação, losango para decisão e barra horizontal para bifurcação ou junção paralela.

Tabela 13.1 – Índice dos diagramas de atividade

ID	Processo	Disparo
AT-01	Carga do histórico corporativo (ETL)	Manual
AT-02	Canonicalização de um rótulo bruto	Interno
AT-03	Treino do motor de diagnóstico	Manual
AT-04	Indexação de um documento orientativo	Manual ou automático
AT-05	Diagnóstico de um novo evento	API
AT-06	<i>Guardrail</i> e geração da prescrição	Interno
AT-07	Conversa com o agente	API
AT-08	Ingestão de telemetria industrial	Mensagem MQTT
AT-09	Cadastro de documento pela interface	Usuário
AT-10	Indexação automática no primeiro <i>boot</i>	<i>Startup</i>
AT-11	Consulta a um painel analítico	Usuário
AT-12	Integração contínua	<i>Push</i> ou <i>pull request</i>
AT-13	Provisionamento e implantação em nuvem	Manual
AT-14	Ciclo de tratamento de lacuna documental	Negócio

13.1 AT-01 – Carga do histórico corporativo

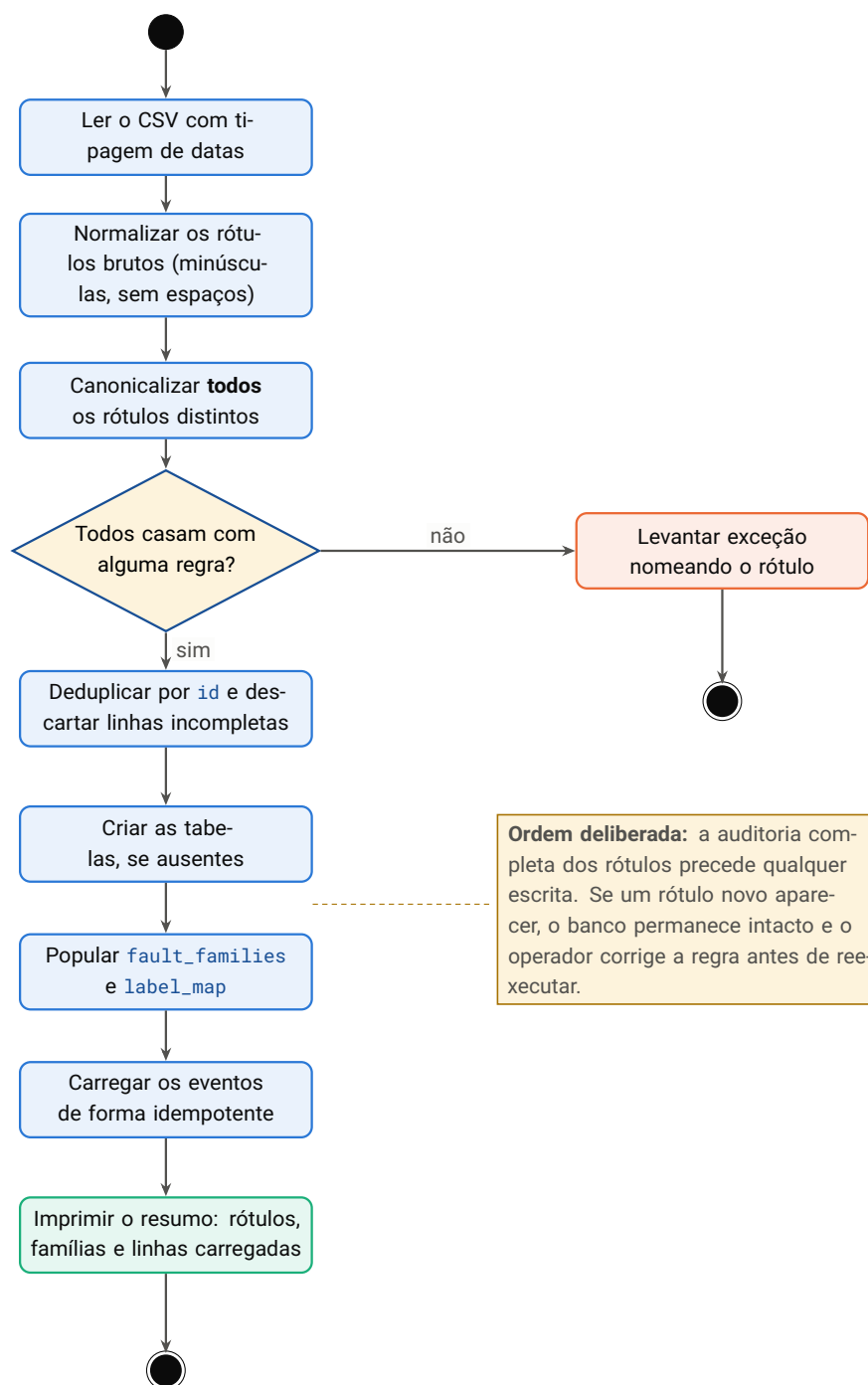


Figura 13.1 – AT-01 – carga do histórico corporativo. O ramo de exceção não é tratamento de erro incidental: é a porta de qualidade de dados do sistema (**RNF-14**).

13.2 AT-02 – Canonicalização de um rótulo bruto

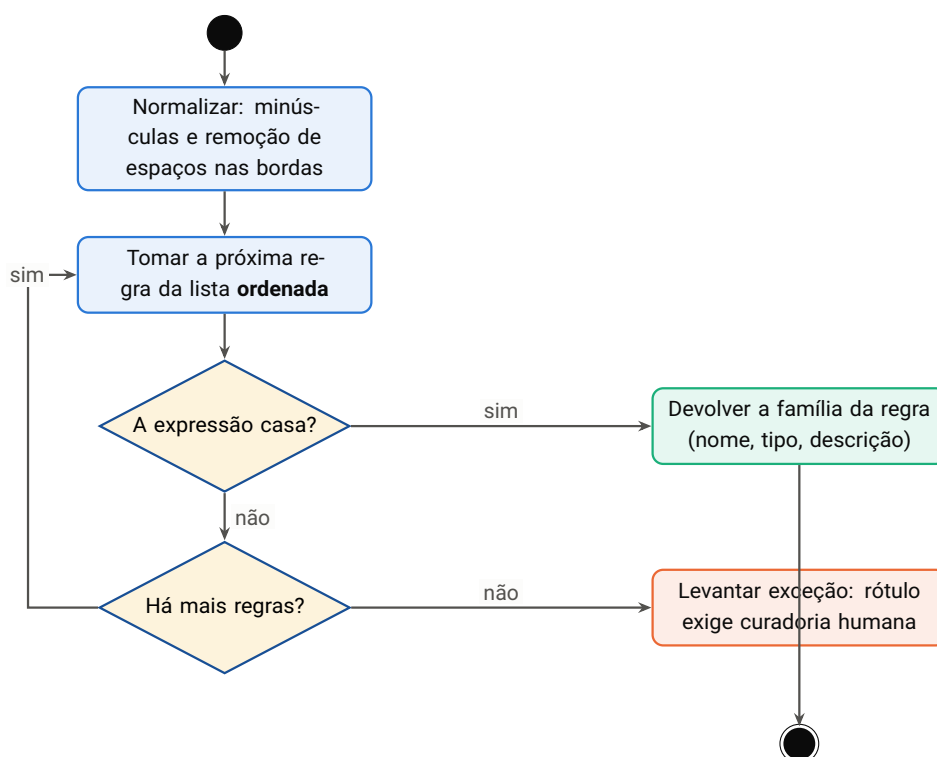


Figura 13.2 – AT-02 – canonicalização de um rótulo bruto. A avaliação é sequencial e a *primeira* regra que casa vence, o que torna a ordem das regras parte da especificação: regras de falha precedem regras de estado operacional.

13.3 AT-03 – Treino do motor de diagnóstico

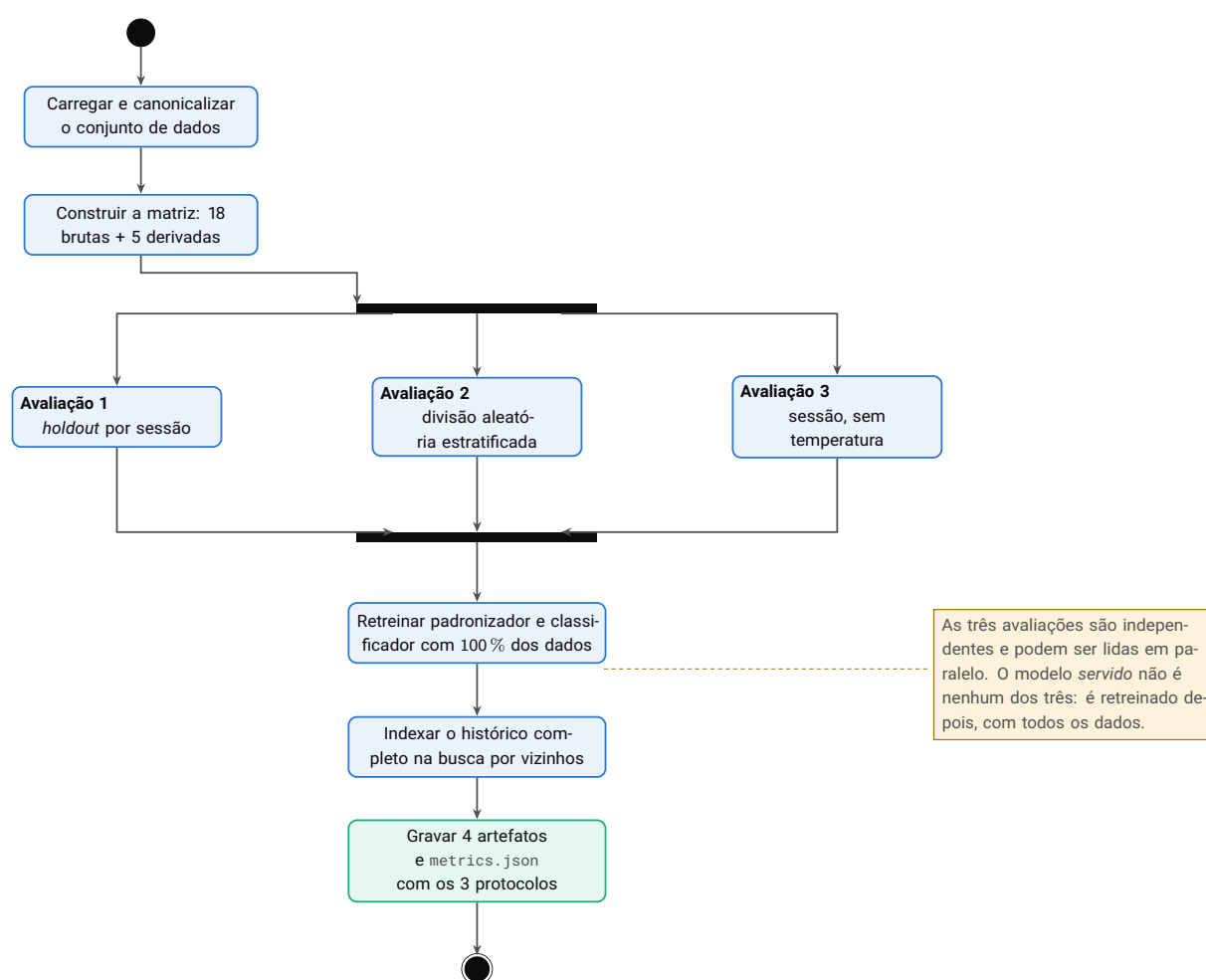


Figura 13.3 – AT-03 – treino do motor de diagnóstico. A bifurcação paralela expressa a independência entre os três protocolos de avaliação; a junção garante que todos concluam antes do treino final.

13.4 AT-04 – Indexação de um documento orientativo

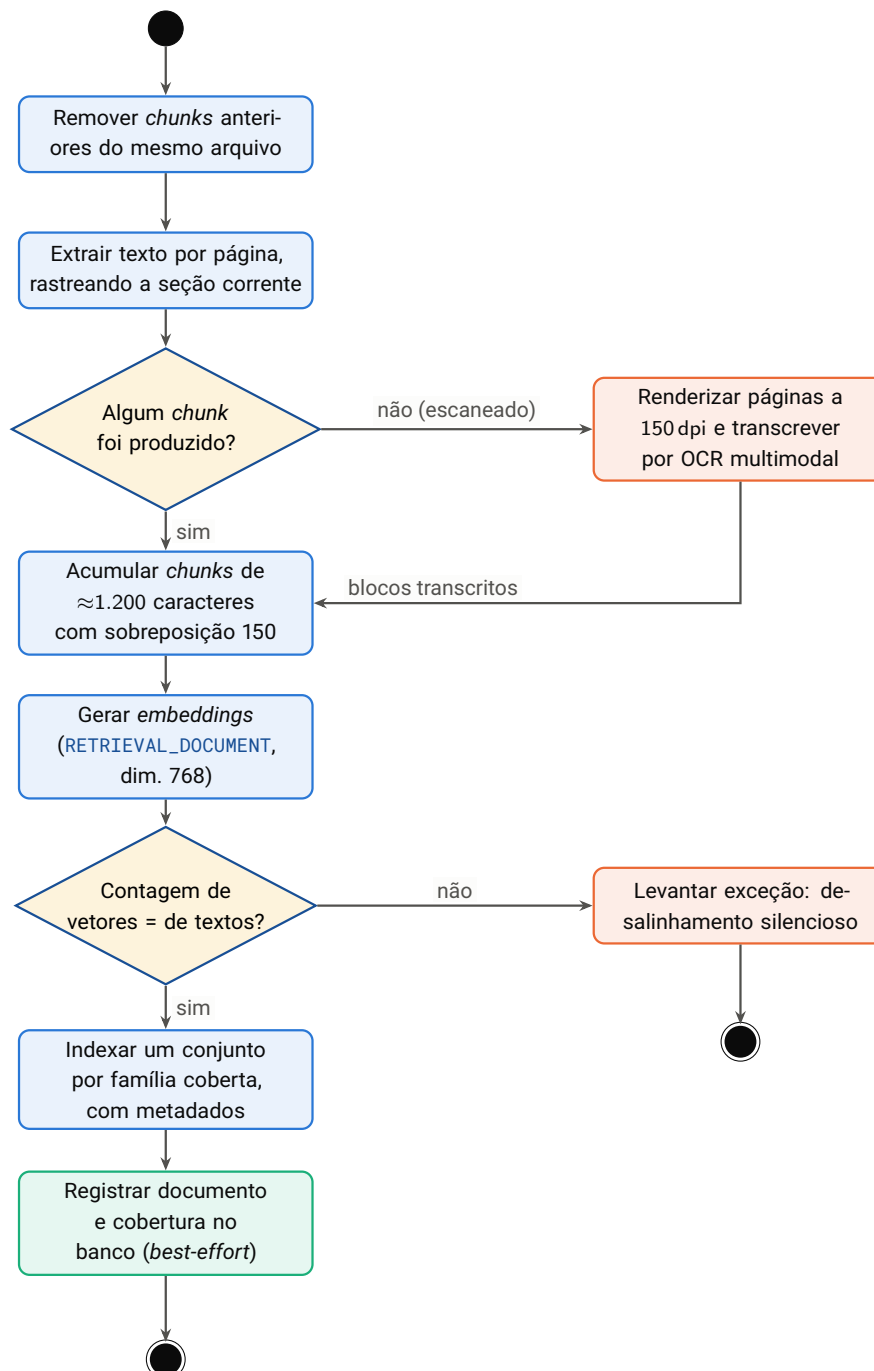


Figura 13.4 – AT-04 – indexação de um documento. O OCR é acionado *apenas* quando a extração direta não produz conteúdo. A verificação de contagem de vetores previne a falha mais perigosa da etapa: associar o vetor de um trecho ao texto de outro.

13.5 AT-05 – Diagnóstico de um novo evento

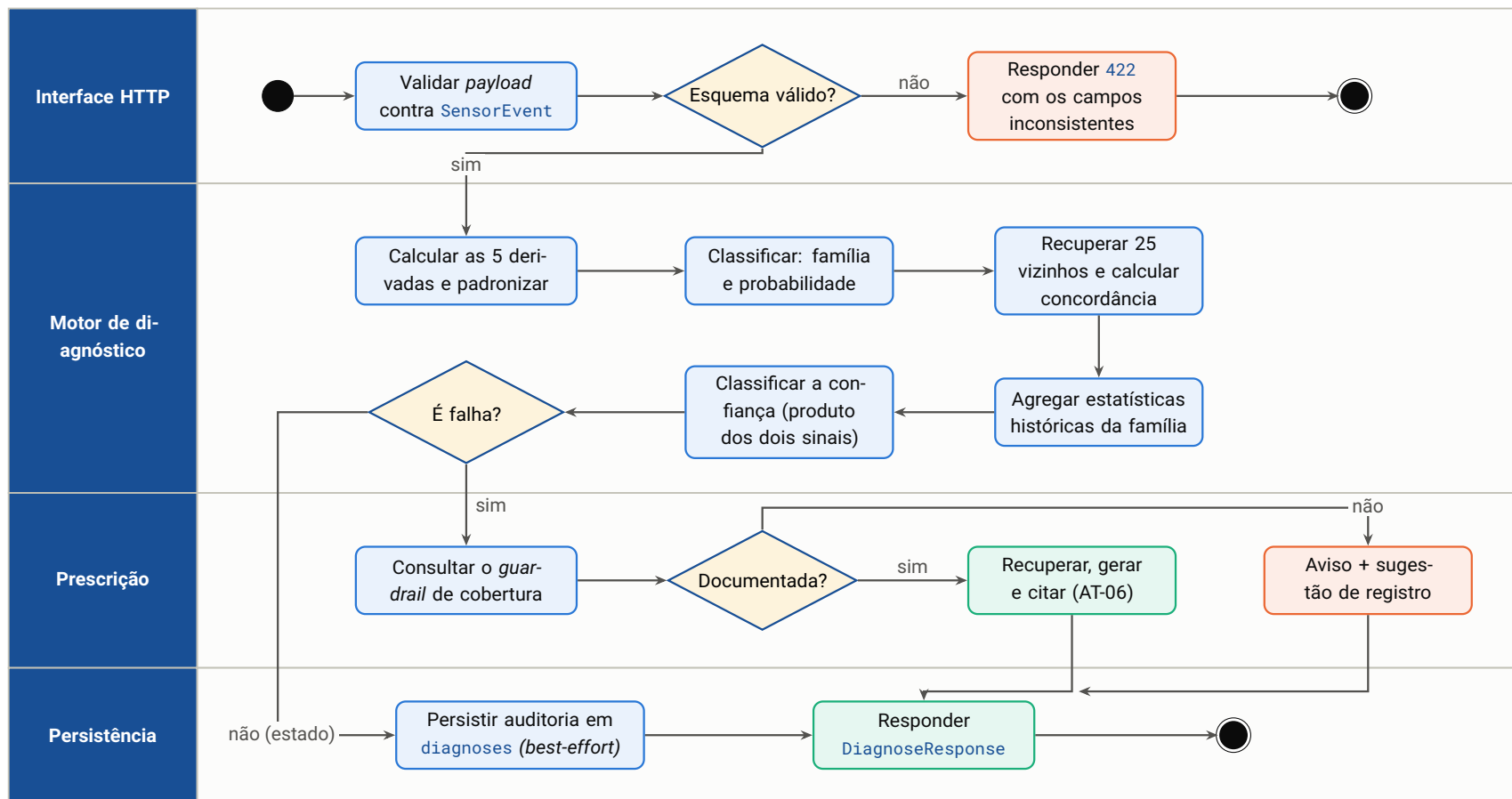


Figura 13.5 – AT-05 – diagnóstico de um novo evento, em raias por responsabilidade. Note que a persistência da auditoria é *best-effort* e paralela ao caminho de resposta: uma falha de banco não impede o técnico de receber o diagnóstico.

13.6 AT-06 – *Guardrail* e geração da prescrição

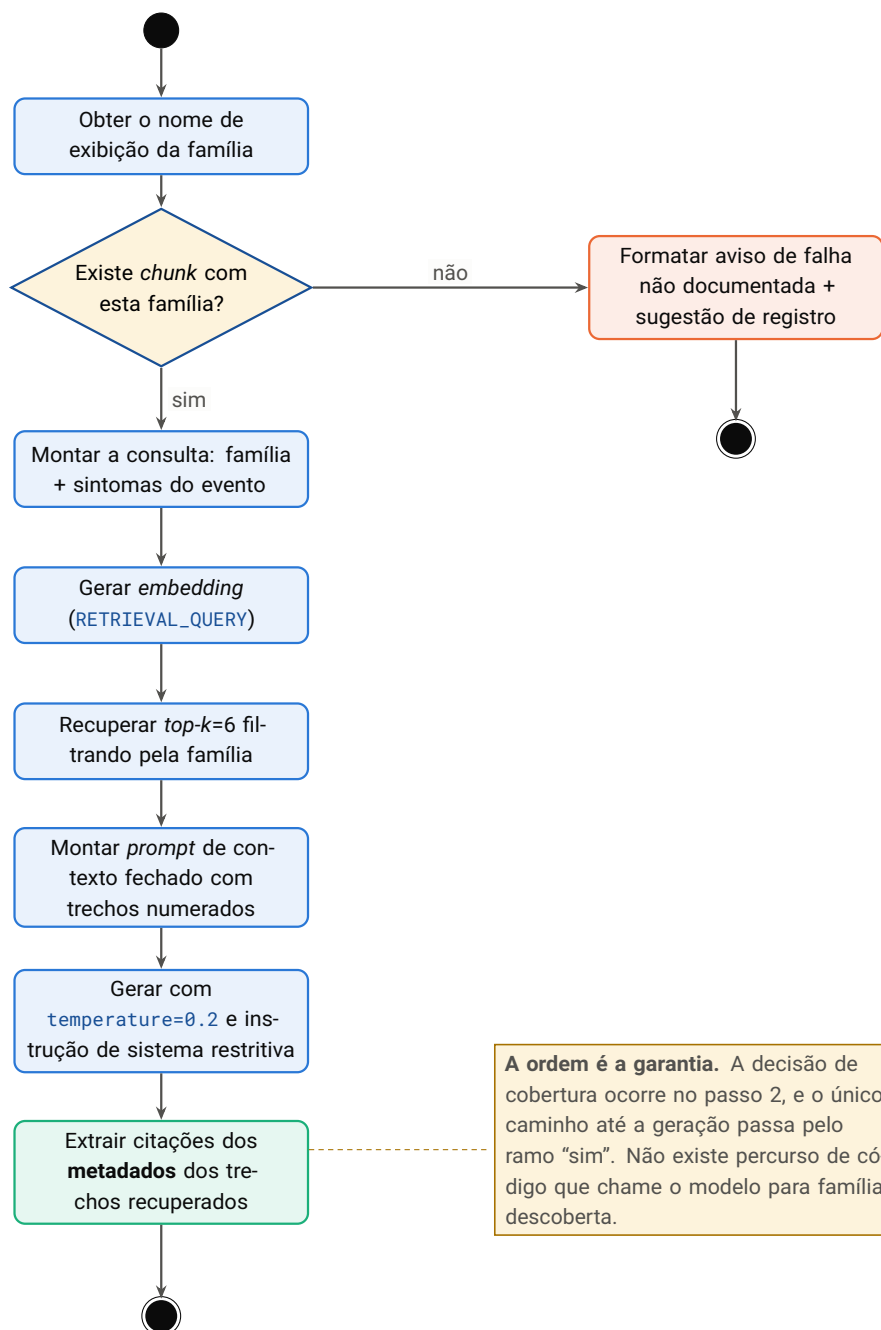


Figura 13.6 – AT-06 – *guardrail* de cobertura e geração da prescrição. As citações são extraídas no penúltimo passo, a partir dos metadados do *retrieval* – nunca do texto gerado.

13.7 AT-07 – Conversa com o agente

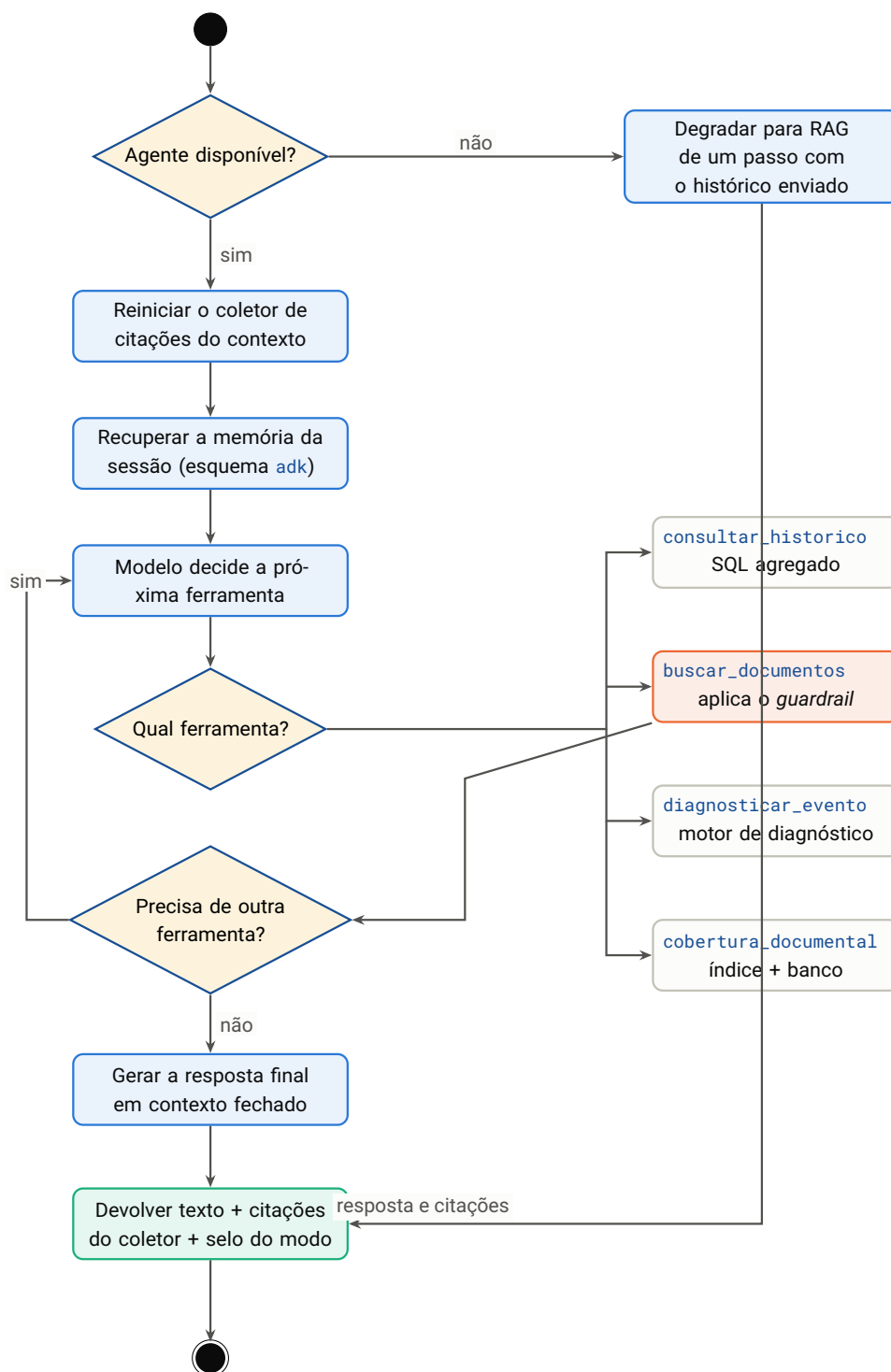


Figura 13.7 – AT-07 – conversa com o agente. A ferramenta de busca documental aparece em laranja porque é ela que carrega o *guardrail*: o modelo recebe um aviso estruturado, não a liberdade de decidir se existe procedimento.

13.8 AT-08 – Ingestão de telemetria industrial

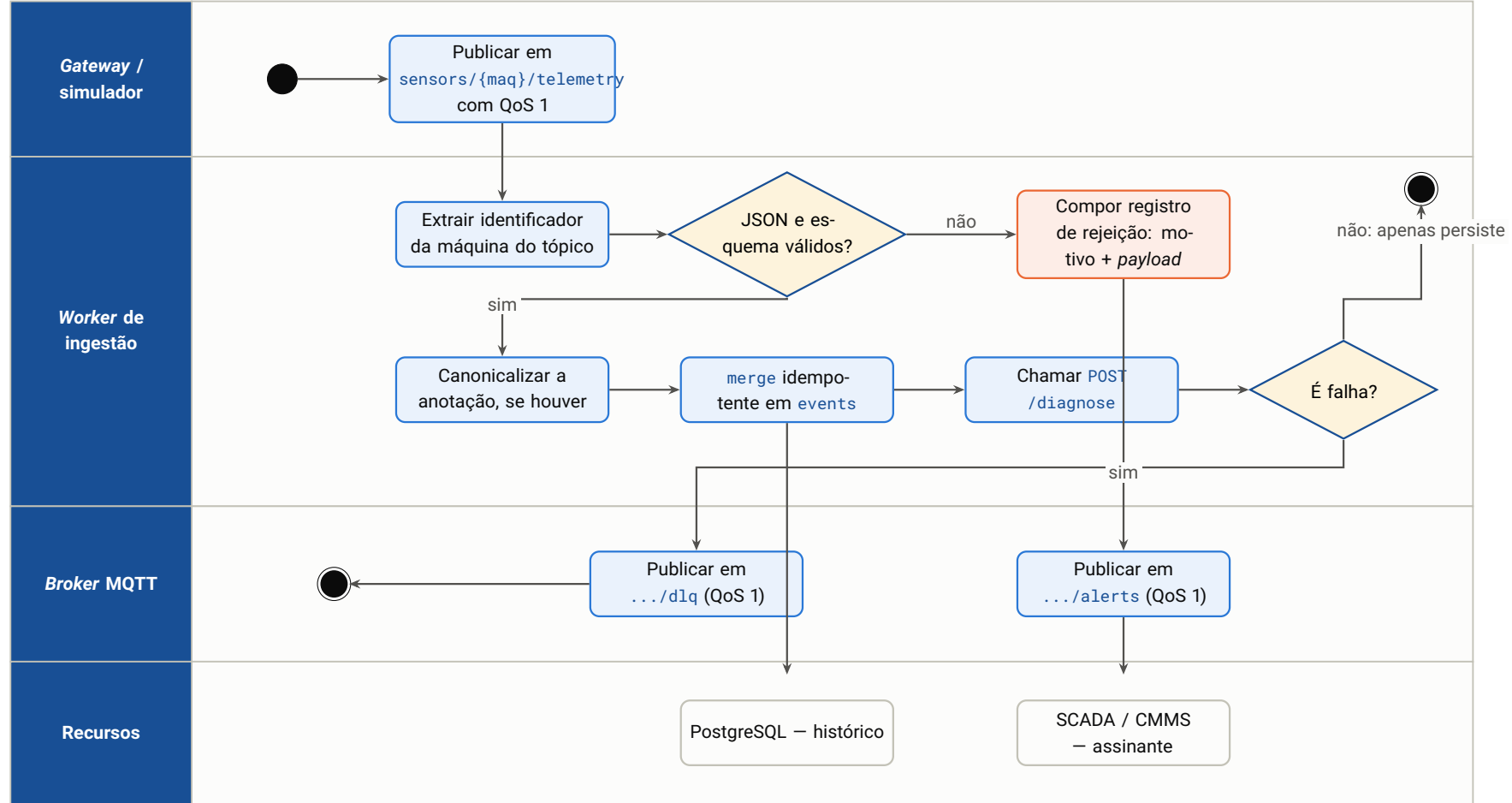


Figura 13.8 – AT-08 – ingestão de telemetria industrial. A persistência precede o diagnóstico: perder o diagnóstico é recuperável, perder o dado bruto não é. O ramo de rejeição nunca descarta em silêncio.

13.9 AT-09 – Cadastro de documento pela interface

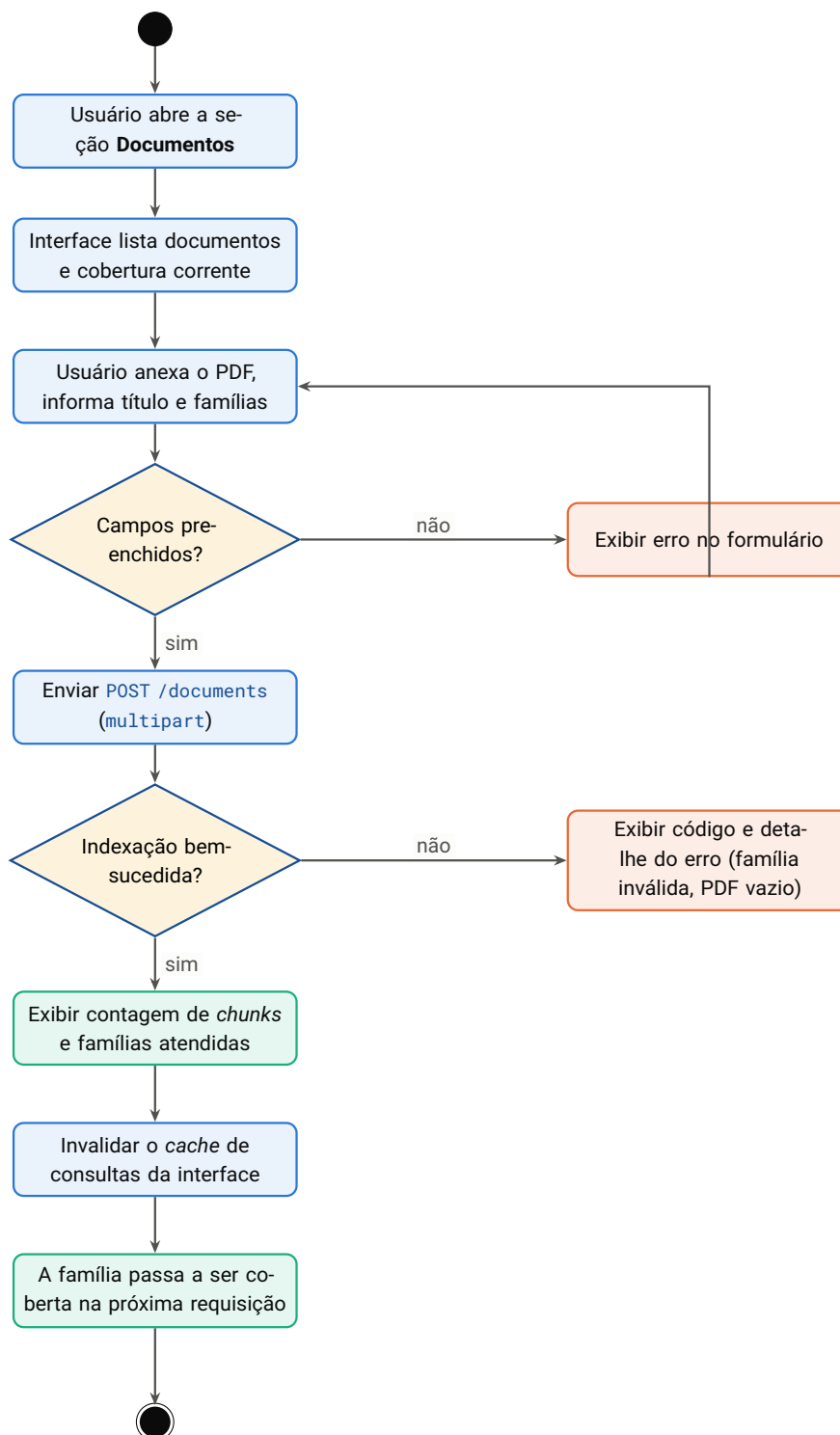


Figura 13.9 – AT-09 – cadastro de documento pela interface. O ciclo se fecha em uma única sessão de trabalho: o engenheiro descobre a lacuna no diagnóstico, escreve o procedimento, cadastra e o sistema passa a prescrever.

13.10 AT-10 – Indexação automática no primeiro *boot*

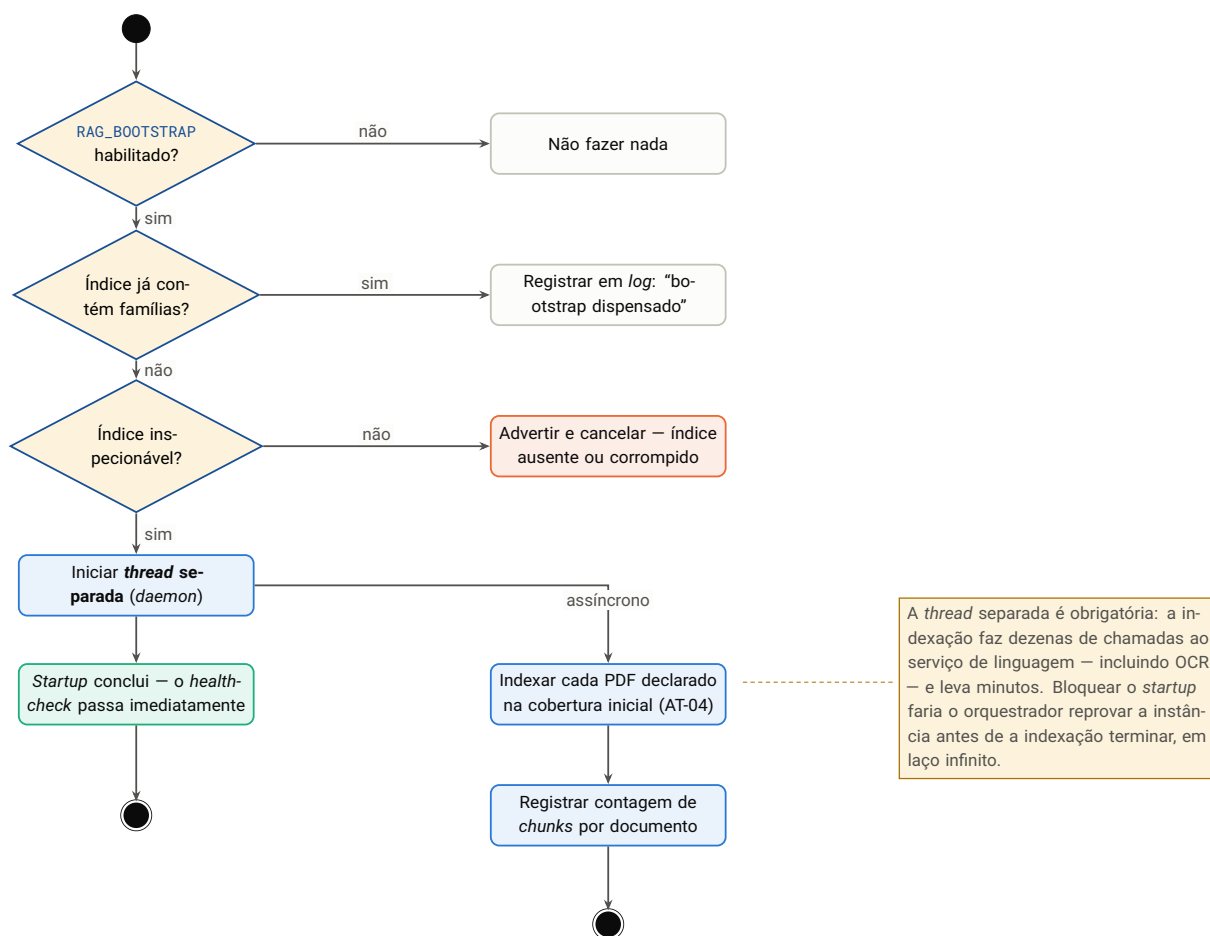


Figura 13.10 – AT-10 – indexação automática no primeiro *boot*. Três guardas antecedem a execução, e a indexação ocorre fora do caminho de *startup*.

13.11 AT-11 – Consulta a um painel analítico

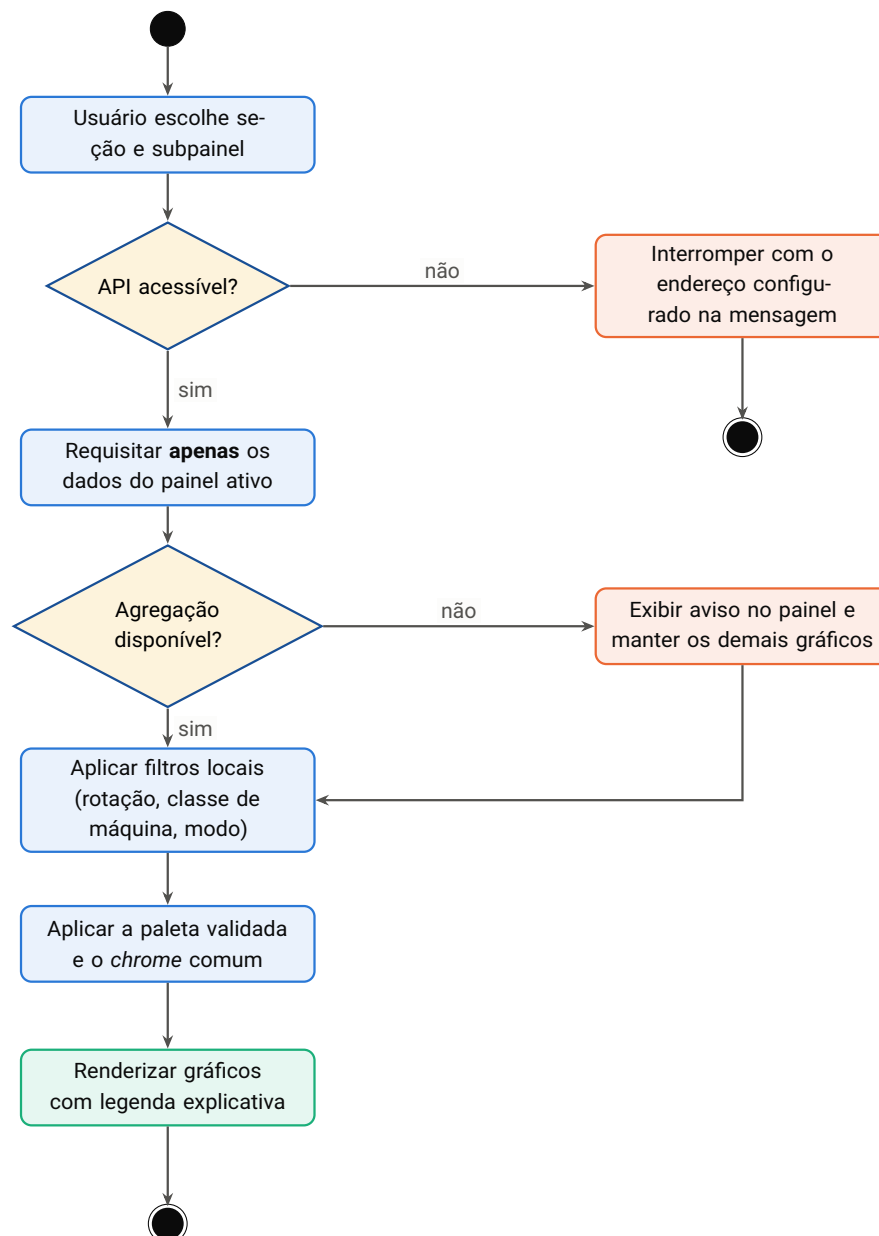


Figura 13.11 – AT-11 – consulta a um painel analítico. A indisponibilidade de uma agregação degrada apenas o gráfico afetado; o painel continua útil.

13.12 AT-12 – Integração contínua

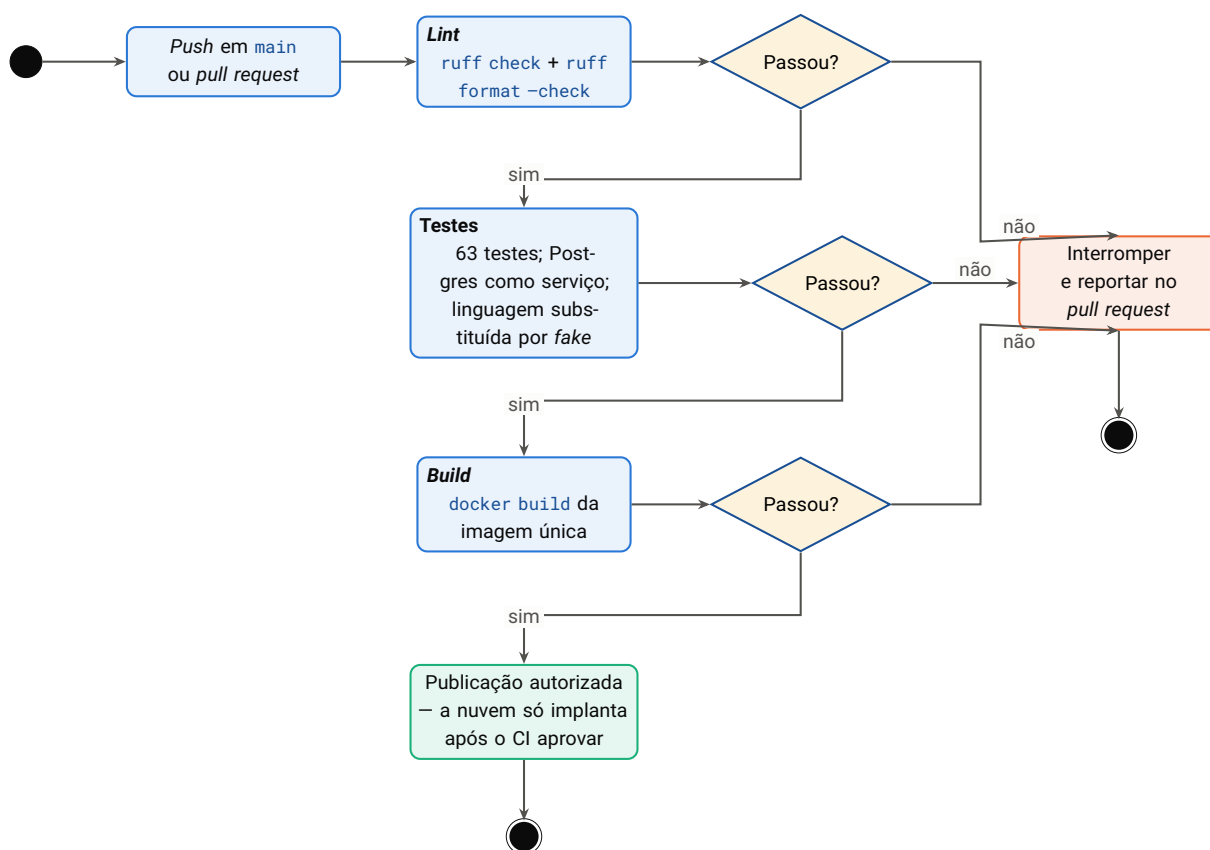


Figura 13.12 – AT-12 – integração contínua. As etapas são sequenciais e bloqueantes: um apontamento de estilo impede a execução dos testes, que por sua vez precedem a construção da imagem. Nenhum teste do fluxo chama o serviço de linguagem real.

13.13 AT-13 – Provisionamento e implantação em nuvem

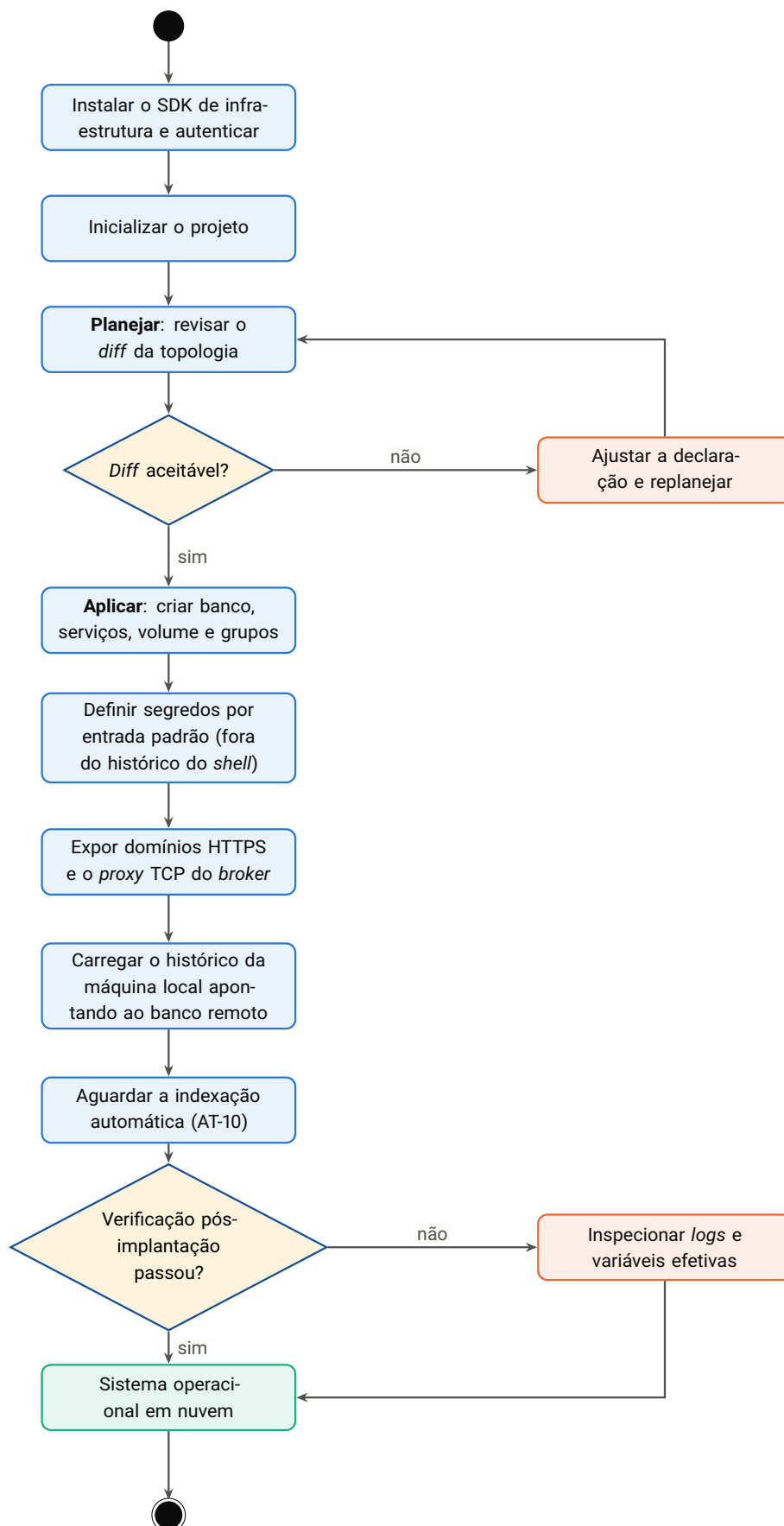


Figura 13.13 – AT-13 – provisionamento e implantação. O passo de planejamento é obrigatório e revisável: a topologia é declarada em código, e nenhuma mudança é aplicada sem que o *diff* tenha sido lido. 111 / 187

13.14 AT-14 – Ciclo de tratamento de lacuna documental

Este é o único diagrama de nível de negócio: ele descreve o ciclo completo que o sistema habilita, atravessando pessoas e *software*. É a razão de ser da solução.

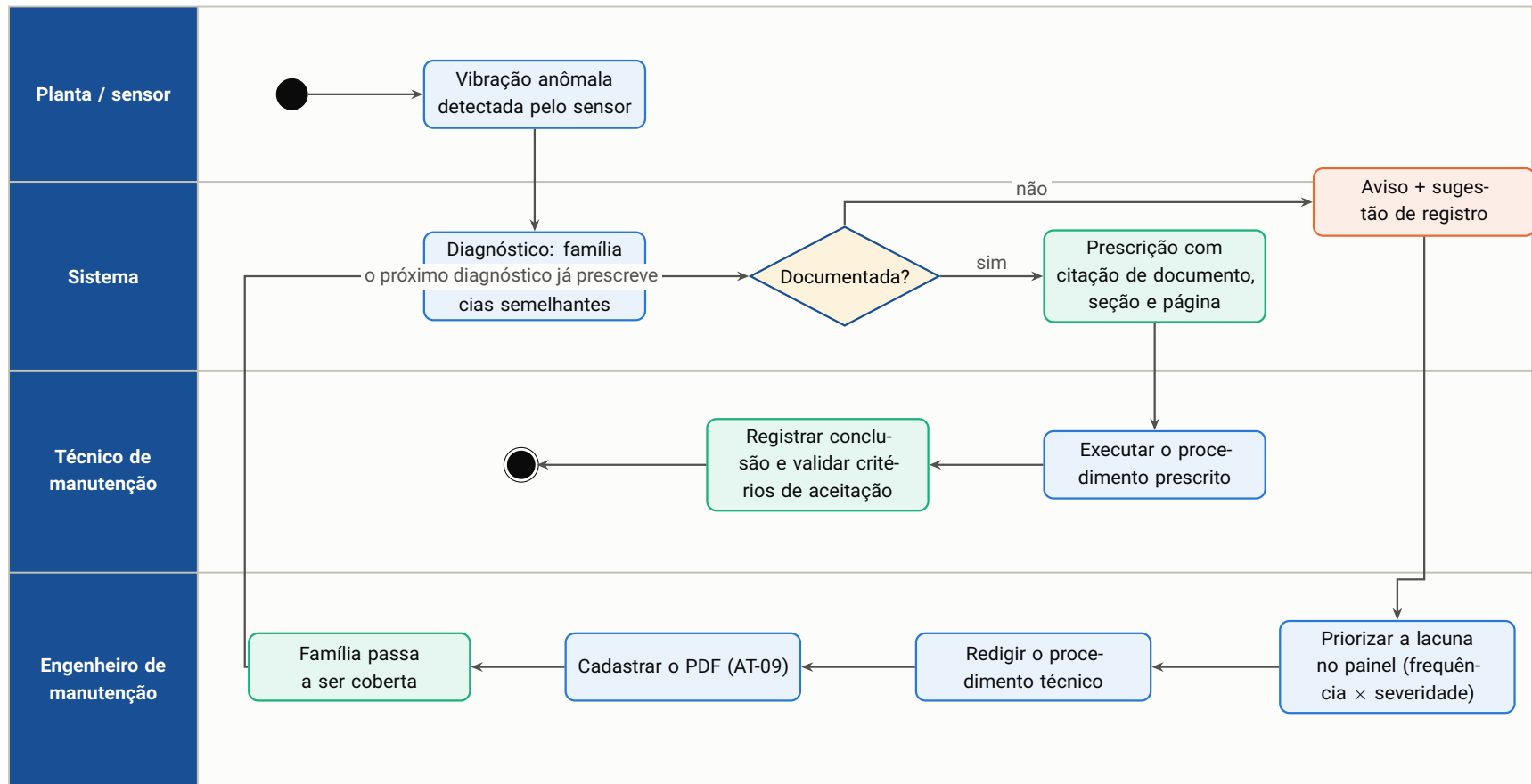


Figura 13.14 – AT-14 – ciclo de tratamento de lacuna documental. O laço de retorno é o valor central da solução: uma falha sem procedimento deixa de ser um beco sem saída e passa a ser uma tarefa priorizada, que ao ser concluída realimenta o sistema.

Infraestrutura e implantação

O sistema roda em dois ambientes: a estação de trabalho local, orquestrada por *docker-compose*, e a nuvem, declarada em código. Este capítulo documenta os dois, as decisões de segurança e o roteiro de operação.

14.1 Imagem de contêiner

Três dos cinco serviços — API, interface e *worker* — compartilham **a mesma imagem** ([ADR-12](#)). O papel de cada um é definido pelo comando de inicialização, não por imagens distintas.

Tabela 14.1 – Decisões de construção da imagem

Decisão	Justificativa
Base <code>python:3.12-slim</code>	Reduz a superfície de imagem sem exigir compilação de dependências binárias.
<code>libgomp1</code> instalada explicitamente	O classificador exige a biblioteca de paralelismo em tempo de execução; sem ela a importação falha — e falha <i>no primeiro diagnóstico</i> , não na construção.
Camada de dependências separada da de código	As dependências são instaladas primeiro, a partir de um pacote-esqueleto. Alterar código-fonte não reinstala aproximadamente 1 GiB de dependências a cada implantação.
Usuário sem privilégios (UID 10.001)	O processo do serviço não roda como <i>root</i> .
Troca de usuário no <i>entrypoint</i>	O volume persistente é montado pelo orquestrador com dono <i>root</i> . O <i>entrypoint</i> inicia como <i>root</i> apenas para ceder o ponto de montagem ao usuário da aplicação e então baixa privilégios antes de executar o comando do serviço. Sem isso, a base vetorial não conseguiria criar o índice.

14.2 Ambiente local

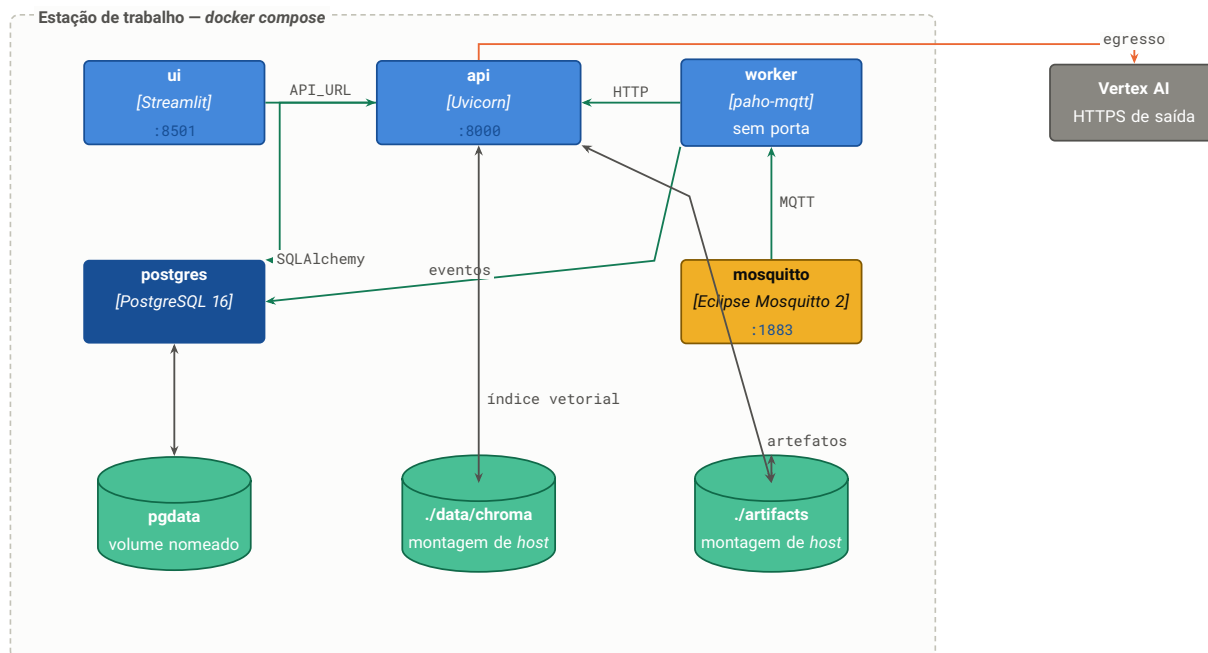


Figura 14.1 – Ambiente local orquestrado por *docker-compose*. As portas expostas no *host* permitem inspecionar cada serviço isoladamente; a comunicação entre serviços usa os nomes de rede internos. Os artefatos de ML e o índice vetorial são montagens do *host*, o que permite treinar fora do contêiner e servir dentro dele.

O arquivo de composição declara dependências com condição de saúde: a API só inicia depois de o banco responder à verificação de prontidão, e o *worker* só inicia depois da API. Isso elimina a classe de falha em que um serviço sobe, encontra a dependência indisponível e entra em laço de reinício.

14.3 Ambiente em nuvem

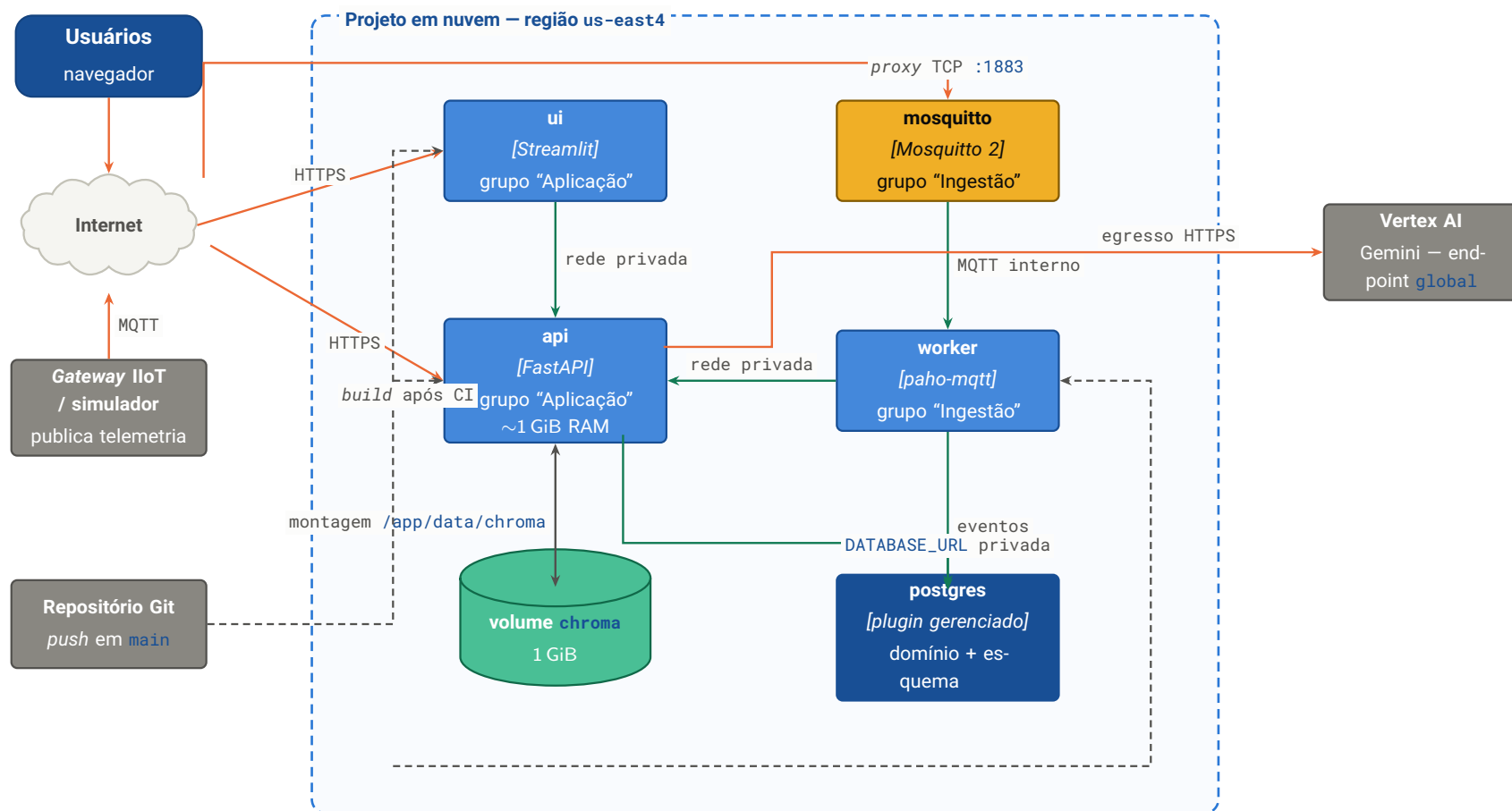


Figura 14.2 – Implantação em nuvem. Em verde, a rede privada: interface, *worker* e API conversam por domínio interno, sem egresso e sem exposição à internet. Em laranja, o que é público por necessidade: os dois domínios HTTPS, o *proxy* TCP do *broker* e o egresso para o serviço de linguagem. Em tracejado, a construção automática de imagem disparada por *push*, condicionada à aprovação do CI.

14.3.1 Topologia declarada

Tabela 14.3 – Serviços em nuvem

Serviço	Origem	Papel	Exposição
<code>postgres</code>	Plugin gerenciado	Histórico de eventos e memória de conversa do agente	Interna
<code>api</code>	Dockerfile da raiz	API, motor de ML e recuperação documental; monta o volume do índice	HTTPS público
<code>ui</code>	Dockerfile da raiz	Interface; consome a API pela rede privada	HTTPS público
<code>worker</code>	Dockerfile da raiz	Ingestão MQTT → banco → diagnóstico → alertas	Nenhuma
<code>mosquitto</code>	mosquitto/Dockerfile	Broker com autenticação obrigatória	Proxy TCP

14.3.2 Decisões de infraestrutura

Tabela 14.5 – Decisões de infraestrutura em nuvem

Decisão	Justificativa
Rede privada entre serviços	O endereço da API para a interface e para o <i>worker</i> aponta para o domínio <i>interno</i> , não para o público. Isso evita egresso cobrado, reduz latência e mantém o tráfego fora da internet.
Volume persistente para o índice vetorial	O sistema de arquivos do contêiner é efêmero. Sem o volume, cada nova implantação apagaria os 268 <i>chunks</i> indexados e todo documento cadastrado pela interface. Volumes pertencem a um único serviço — só a API toca a base vetorial, portanto a restrição não atrapalha.
Região do volume fixada explicitamente	Omitir a região faria o planejamento zerar a região do volume existente, o que é operação destrutiva .
Porta declarada como variável	A porta injetada em tempo de execução não é referenciável entre serviços; declará-la explicitamente permite que a interface e o <i>worker</i> componham o endereço interno da API.
Comando envolvido em <i>shell</i>	O orquestrador executa o comando de inicialização diretamente, sem <i>shell</i> ; sem o invólucro, a variável de porta não seria expandida.
Publicação condicionada ao CI	A construção de imagem só ocorre após a aprovação do fluxo de integração contínua — código que não passa no <i>lint</i> ou nos testes não chega a ser implantado.

14.3.3 Extrato da declaração

Listing 14.1 – Extrato de `.railway/railway.ts` — serviço de API com volume e segredos preservados

```

1  const api = service("api", {
2    source: github(REPO, SOURCE),
3    // Envolvido em `sh -c` de proposito: o orquestrador executa o start command
4    // diretamente (sem shell), entao $PORT nao seria expandido.
5    start: 'sh -c "uvicorn src.api.main:app --host 0.0.0.0 --port ${PORT:-8000}"',
6    env: {
7      PORT: "8000", // declarado para ser referenciavel
8      DATABASE_URL: db.env.DATABASE_URL, // referencia ao plugin gerenciado
9      GOOGLE_CLOUD_PROJECT: "manutencao-prescritiva",
10     GOOGLE_CLOUD_LOCATION: "global",
11     GOOGLE_CREDENTIALS_B64: preserve(), // nome versionado, valor nunca
12     GEMINI_CHAT_MODEL: "gemini-3.6-flash",
13     GEMINI_EMBEDDING_MODEL: "gemini-embedding-2",
14     EMBEDDING_DIM: "768",
15     ARTIFACTS_DIR: "/app/artifacts",
16     CHROMA_DIR: "/app/data/chroma",

```

```

17 DOCS_DIR: "/app/documentos",
18 RAG_BOOTSTRAP: "true",           // indexa no primeiro boot
19 MQTT_HOST: mosquito.env.RAILWAY_PRIVATE_DOMAIN,
20 },
21 volumeMounts: {
22   // A regioao e fixada: omiti-la zeraria a regioao do volume existente,
23   // o que e uma operacao destrutiva.
24   "/app/data/chroma": volume("chroma", { sizeMB: 1024, region: "us-east4-eqdc4a" }),
25 },
26 });

```

14.4 Gestão de segredos

Segredo	Tratamento
Credencial do serviço de nuvem	Transportada em base64 em variável de ambiente. Na inicialização é decodificada, validada como JSON e escrita em arquivo temporário com permissão <code>0600</code> . O arquivo nunca entra no repositório nem na imagem. Conteúdo inválido é registrado como erro e o serviço de linguagem fica indisponível — sem interromper o restante.
Usuário e senha do <i>broker</i>	Definidos como variáveis do serviço. A configuração do <i>broker</i> é gerada no <i>boot</i> ; sem credenciais, registra advertência e só então cai em modo anônimo.
Credencial do banco	Referência direta ao plugin gerenciado — o valor nunca transita pelo arquivo de declaração.
Declaração em código	Variáveis sensíveis usam o marcador de preservação: o <i>nome</i> é versionado, o <i>valor</i> nunca. Isso mantém a declaração auditável sem expor segredo.
Definição por entrada padrão	Segredos são definidos por <i>pipe</i> da entrada padrão, não como argumento de linha de comando — o que evita que fiquem no histórico do <i>shell</i> .

14.5 Carga inicial em ambiente novo

Dado	Como é carregado
Histórico (166.796 eventos)	Da máquina local, apontando para o banco remoto. O arquivo de 31 MB não precisa ir para a imagem nem para o repositório.
Artefatos de ML (74 MiB)	Copiados para a imagem em tempo de construção. São determinísticos e versionados junto ao código que os produziu.

Base documental **Não exige passo manual:** a API detecta o índice vazio no *startup* e indexa os PDFs sozinha, em *thread* separada ([AT-10](#)).

A indexação automática existe por uma razão prática: o volume nasce vazio e não há *shell* dentro do contêiner para executar o *script* de ingestão. A alternativa — exigir um passo manual — transformaria cada nova implantação em procedimento com etapa esquecível, e o sintoma do esquecimento seria o sistema afirmar que *nenhuma* falha está documentada.

14.6 Verificação pós-implantação

Listing 14.2 – Roteiro de verificação pós-implantação

```

1 # 1. Saúde: artefatos carregados e famílias documentadas
2 curl -s https://<api>/api/v1/health | jq
3
4 # 2. Diagnostico de ponta a ponta com o evento do enunciado
5 curl -s -X POST https://<api>/api/v1/diagnose \
6     -H 'Content-Type: application/json' -d @evento.json \
7     | jq '.predicted_fault, .probability, .documented, .similar_events.count'
8
9 # 3. Acompanhar a indexacao inicial (leva minutos por causa do OCR)
10 railway logs --service api | grep bootstrap
11
12 # 4. Interface e broker
13 curl -s -o /dev/null -w '%{http_code}\n' https://<ui>/
14 mosquitto_pub -h <host-proxy> -p <porta> -u "$MQTT_USERNAME" -P "$MQTT_PASSWORD" \
15     -t 'sensors/maquina-01/telemetry' -m "$(cat evento.json)"

```

#	Verificação	Critério de aprovação
1	Saúde	<code>artifacts_loaded=true</code> . O campo <code>documented_families</code> só vem preenchido após a indexação inicial — lista vazia no primeiro minuto é estado esperado, não falha.
2	Diagnóstico	Família prevista coerente, probabilidade acima de 0,9 para o evento do enunciado, <code>documented=true</code> e contagem de ocorrências semelhantes na ordem de 14.000.
3	Indexação inicial	Linhas de <i>log</i> com a contagem de <i>chunks</i> por documento, até a mensagem de conclusão.
4	Interface e <i>broker</i>	Interface responde <code>200</code> ; publicação no <i>broker</i> aceita a autenticação e o <i>worker</i> registra a persistência do evento.

14.7 Operação

Objetivo	Comando
Logs de execução	<code>railway logs --service api --lines 100</code>
Histórico de implantações	<code>railway deployment list --service api</code>
Reimplantar sem reconstruir	<code>railway redeploy --service api</code>
Variáveis efetivas	<code>railway variable list --service api</code>
Visão do projeto	<code>railway status</code>
Planejar mudança de topologia	<code>railway config plan</code>
Aplicar mudança de topologia	<code>railway config apply</code>

14.8 Custo e topologia reduzida

Cinco serviços ativos continuamente excedem o crédito de planos de entrada. A API é o serviço mais pesado: carrega 74 MiB de artefatos e mantém o índice de vizinhos em memória, o que resulta em aproximadamente 1 GiB de RAM.

Topologia	Capacidades entregues
Reduzida — banco, API e interface	Diagnóstico, quantificação histórica, prescrição documental, conversa com agente e todos os painéis analíticos. Atende integralmente ao enunciado.
Completa — acrescenta <i>worker</i> e <i>broker</i>	Demonstra a integração industrial de ponta a ponta. Pode ser mantida apenas localmente, via composição de contêineres.

14.9 Adequação à restrição de hardware

A estação de trabalho alvo (32 GiB de RAM, GPU de 16 GiB) executa a solução com folga (**RE-01**):

- Artefatos de ML somam 74 MiB em disco; em memória, o componente dominante é o índice de vizinhos.
- A base vetorial local contém centenas de *chunks* — ordens de magnitude abaixo do que exigiria um servidor vetorial dedicado.

- Os modelos pesados — linguagem e *embeddings* — rodam como serviço gerenciado. **Nenhuma inferência local de modelo de linguagem é necessária**, e a GPU permanece integralmente disponível.

Qualidade, testes e integração contínua

15.1 Estratégia de testes

A suíte tem **63 testes** e uma propriedade que determinou todo o desenho da camada de linguagem: ela executa **sem rede e sem credenciais**. Nenhum teste chama o serviço de linguagem real.

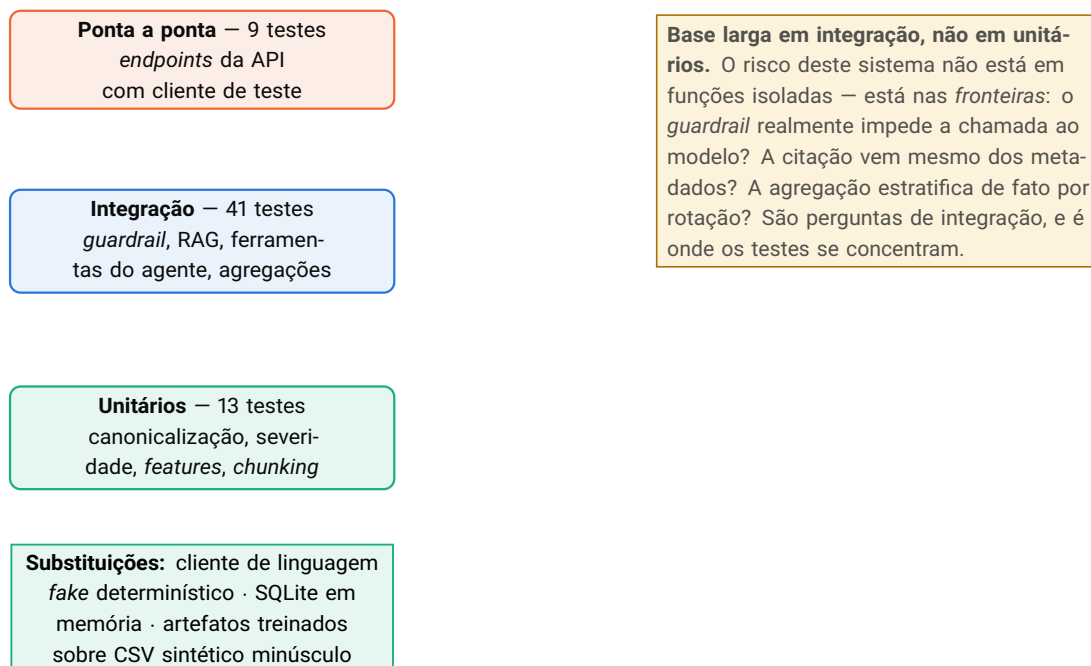


Figura 15.1 – Distribuição dos 63 testes. A forma é deliberadamente diferente da pirâmide canônica (Cohn, 2009): o volume está na camada de integração, porque é ali que residem as garantias que este sistema promete.

15.1.1 Substituições que tornam a suíte offline

Tabela 15.1 – Substituições de dependência nos testes

Dependência real	Substituição e propriedade preservada
Cliente de linguagem	<i>Fake</i> que implementa o mesmo protocolo. <i>Embeddings</i> são derivados de <i>hash</i> SHA-256 do texto — determinísticos e, portanto, testáveis quanto a ordenação de similaridade. A geração devolve texto fixo com marcadores de citação. O <i>fake</i> registra as chamadas, o que permite testar a propriedade mais importante do sistema: <i>que ele não foi chamado</i> .
Banco de dados	SQLite em memória, injetado por sobrescrita de dependência. Suficiente para o modelo de domínio; insuficiente para as agregações analíticas, que exigem recursos estatísticos do PostgreSQL.
Artefatos de ML	Treinados durante a inicialização da suíte sobre um CSV sintético minúsculo. Isso cobre, de graça, o <i>pipeline</i> de treino de ponta a ponta — e mantém a suíte independente dos artefatos de 74 MiB.
Base vetorial	Diretório temporário por sessão de teste, apontado por variável de ambiente antes dos <i>imports</i> com memoização.

O cliente de linguagem foi definido como **protocolo** (interface estrutural), não como classe concreta a herdar. A consequência: o *fake* não precisa importar nem conhecer a implementação real — basta ter os três métodos com as assinaturas corretas. Isso mantém o código de teste livre de dependência sobre a biblioteca de nuvem, que assim não precisa nem estar instalada para a suíte rodar.

15.2 Suíte de testes

Tabela 15.3 – Arquivos de teste e o que cada um garante

Arquivo	Testes	Garantia verificada
test_canonize.py	4	Rótulos reais — inclusive os com erro de digitação — caem na família correta; rótulo desconhecido levanta exceção; o sinalizador de falha está correto para as 17 famílias; toda família tem descrição de exibição.
test_rag_guardrail.py	9	O núcleo do requisito RF-04. Inclui <code>test_undocumented_never_calls_llm</code> , que verifica que o registro de chamadas do <i>fake</i> permanece <i>vazio</i> para família sem documento. Cobre também o <i>fallback</i> de OCR, o filtro por família na recuperação e a sugestão de registro.
test_api.py	7	Contratos dos <i>endpoints</i> : saúde, diagnóstico documentado, diagnóstico não documentado, estado operacional, <i>payload</i> inválido, conversa e rejeição de família inexistente no cadastro de documento.
test_agent_tools.py	11	Cada uma das quatro ferramentas do agente, incluindo o <i>guardrail</i> dentro da busca documental, JSON inválido no diagnóstico e família inexistente na consulta ao histórico. Verifica também que o <i>endpoint</i> usa o agente quando disponível e degrada quando não.
test_analytics.py	5	As agregações: estratificação por rotação, linha de base do mesmo regime, escores padronizados com poder discriminativo, detecção da <i>feature</i> -carimbo de sessão e rejeição de coluna fora da lista branca.
Total	63	Coletados por parametrização dos casos acima.

15.2.1 O teste mais importante da suíte

Listing 15.1 – O teste que verifica a garantia central: o modelo não é chamado para família sem documento

```

1 def test_undocumented_never_calls_llm(fake_llm, undocumented_family, event):
2     """Família sem chunk indexado não deve produzir NENHUMA chamada ao modelo."""
3     fake_llm.generate_calls.clear()
4
5     prescription, suggestion = rag_service.prescribe(
6         undocumented_family, event, fake_llm
7     )

```

```
8
9  assert prescription is None           # nao ha prescricao
10 assert suggestion is not None        # ha aviso e sugestao de registro
11 assert fake_llm.generate_calls == [] # <- a garantia: zero chamadas
```

A última asserção é o que distingue uma garantia de uma intenção. Sem ela, uma refatoração futura poderia mover a verificação de cobertura para *depois* da montagem do *prompt* — o comportamento visível continuaria correto, mas o modelo passaria a ser chamado, e o custo e o risco de alucinação voltariam sem que ninguém percebesse.

15.3 Integração contínua

O fluxo tem três etapas sequenciais e bloqueantes (AT-12):

Tabela 15.5 – Etapas do fluxo de integração contínua

Etapa	O que executa	Por que bloqueia
Lint	Verificação de regras e de formatação em código, testes e scripts	Estilo inconsistente polui o <i>diff</i> de revisões futuras e esconde mudanças reais.
Testes	Suíte completa, com PostgreSQL como serviço de apoio	É a porta de qualidade funcional. As credenciais de nuvem não são fornecidas — e não são necessárias.
Construção	Construção da imagem única	Garante que a imagem que será implantada realmente constrói, antes de qualquer implantação.

NOTA

As credenciais de nuvem existem apenas como segredos de repositório e como arquivo local não versionado. O fluxo de CI **não** as recebe, porque não precisa: o *fake* implementa a mesma interface do cliente real e é injetado por sobrescrita de dependência. Um fluxo de CI que exige segredos para rodar é um fluxo que ninguém consegue executar em um *fork*.

15.4 Padrões de código

Padrão	Aplicação e motivo
Linha de 100 colunas	Compatível com revisão em tela dividida sem quebra de linha artificial.
Conjuntos de regras habilitados	Erros, <i>imports</i> desordenados, construções obsoletas para a versão de linguagem alvo e armadilhas comuns.
Exceção documentada	Argumento padrão mutável é permitido porque é o idioma do <i>framework</i> web usado — a exceção está registrada na configuração, com o motivo.
Tipagem em fronteiras	Anotações de tipo em todas as funções públicas. Os esquemas de domínio são a fonte de verdade de tipo, e a especificação OpenAPI é derivada deles.
<i>Docstrings</i> explicando <i>por quê</i>	As <i>docstrings</i> do projeto registram a <i>razão</i> das decisões, não a repetição do nome da função. Boa parte deste documento foi consolidada a partir delas.
Configuração externalizada	Toda configuração vem de variáveis de ambiente com valores padrão sensatos, conforme o padrão de doze fatores (Wiggins, 2017).

15.5 Lacunas conhecidas de teste

- **O worker MQTT não tem teste automatizado.** Sua verificação é o roteiro manual da Seção 10.6. A ausência é consciente: um teste real exigiria *broker* embarcado no fluxo de CI. A lógica testável — validação e canonicalização — é compartilhada com a API e está coberta por lá.
- **A interface não tem teste automatizado.** É camada de apresentação sem lógica de domínio; o risco de regressão silenciosa é baixo e o custo de instrumentação seria alto.
- **Os testes de analítica são pulados sem PostgreSQL.** Eles exigem recursos estatísticos ausentes no SQLite. São executados no fluxo de CI, que provê o banco como serviço.
- **Não há teste de carga.** Os números de latência citados neste documento vêm de medição pontual em ambiente local, não de teste de desempenho sistemático.

Documentação de código

Este capítulo é a referência de código do projeto: para cada módulo, seu propósito, sua interface pública, os invariantes que ele mantém e os pontos por onde estendê-lo. São 3.514 linhas de Python em 24 módulos.

16.1 Estrutura do repositório

Listing 16.1 – Estrutura do repositório

```

1  FIESC/
2  |-- src/
3  |   |-- core/           configuracao, esquemas de dominio, ORM, severidade
4  |   |   |-- config.py   (111) Settings + ponte para o ambiente das libs de nuvem
5  |   |   |-- schemas.py  (111) esquemas pydantic UNICOS (API + MQTT + ETL)
6  |   |   |-- db.py       (110) modelos SQLAlchemy 2.0 e fabrica de sessao
7  |   |   +-- severity.py  (78) zonas ISO 10816 e severidade relativa
8  |   |-- etl/
9  |   |   |-- canonize.py  (59) canonicalizacao auditavel de rotulos
10 |   |   +-- label_map.yaml 17 familias + regras regex ordenadas
11 |   |-- ml/
12 |   |   |-- features.py  (43) features fisicas derivadas (treino + inferencia)
13 |   |   |-- train.py     (178) tres protocolos de avaliacao + modelo final
14 |   |   +-- diagnose.py  (108) motor de inferencia (LightGBM + KNN + confianca)
15 |   |-- rag/
16 |   |   |-- chunking.py  (84) extracao de PDF, secoes, fallback de OCR
17 |   |   |-- store.py     (92) ChromaDB + GUARDRAIL de cobertura
18 |   |   |-- service.py   (86) orquestracao RAG e citacoes deterministicas
19 |   |   |-- bootstrap.py (78) indexacao automatica no primeiro boot
20 |   |   +-- coverage.yaml cobertura documental inicial
21 |   |-- llm/
22 |   |   |-- client.py    (89) protocolo do cliente + implementacao Gemini
23 |   |   |-- prompts.py   (77) instrucoes de sistema e montagem de prompt
24 |   |   |-- agent.py     (112) agente ADK, sessoes no Postgres, fallback
25 |   |   +-- agent_tools.py (213) as 4 ferramentas + coletor de citacoes
26 |   |-- api/
27 |   |   |-- main.py      (280) roteadores REST e ciclo de vida
28 |   |   |-- deps.py      (28) dependencias injetaveis (substituiveis em teste)
29 |   |   +-- analytics.py  (240) agregacoes SQL sob demanda
30 |   |-- ingestion/
31 |   |   +-- mqtt_worker.py (145) worker MQTT: valida, persiste, diagnostica, alerta
32 |   +-- ui/

```

```
33 |      |-- app.py          (321) navegacao, diagnostico, conversa, documentos
34 |      |-- dashboard.py    (730) quatro paineis analiticos
35 |      |-- theme.py        (121) paleta validada e chrome dos graficos
36 |      +-- api_client.py   (20) cliente HTTP com cache
37 |-- scripts/              ingest_data, ingest_docs, simulate_sensor
38 |-- tests/                conftest + 5 arquivos, 63 testes
39 |-- artifacts/            modelos treinados (gerados, fora do controle de versao)
40 |-- documentos/          Doc1..Doc6.pdf + banner.csv
41 |-- .railway/railway.ts   infraestrutura como codigo
42 |-- Dockerfile            imagem unica (api / worker / ui via start command)
43 |-- docker-compose.yml    topologia local
44 +-- .github/workflows/ci.yml lint -> testes -> build
```

16.2 Grafo de dependências

As dependências apontam sempre *para baixo*. Nenhum módulo de camada inferior importa de camada superior — propriedade que permite testar o domínio sem levantar a aplicação.

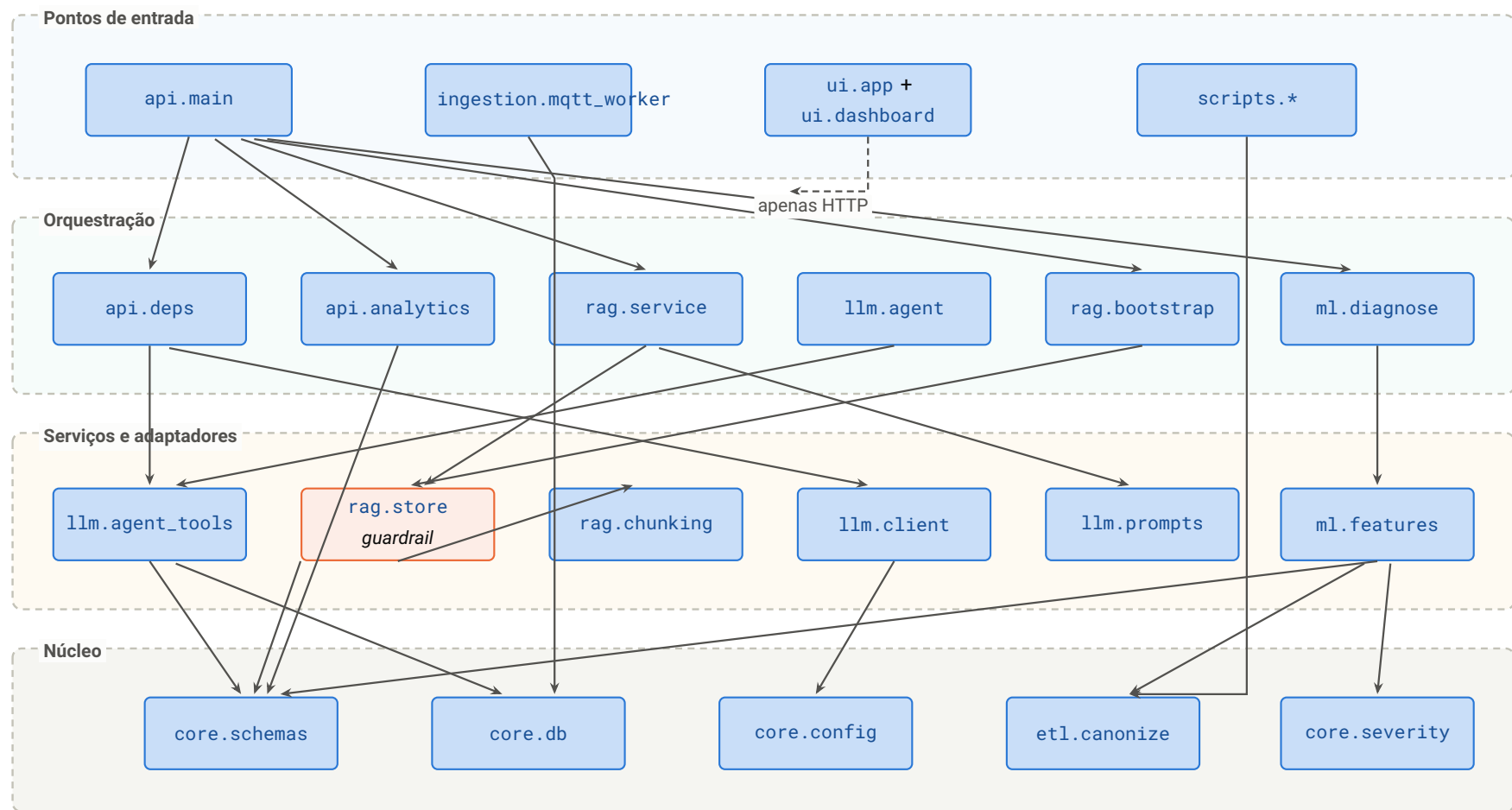


Figura 16.1 – Grafo de dependências entre módulos. O *guardrail* (em laranja) reside na camada de adaptadores, e é alcançado tanto pela orquestração RAG quanto pelas ferramentas do agente – uma única implementação, dois caminhos de uso. A interface depende do sistema apenas por HTTP.

16.3 Pacote `src/core` — núcleo

16.3.1 `config.py`

Aspecto	Descrição
Propósito	Fonte única de configuração, carregada de variáveis de ambiente com valores padrão sensatos.
Interface pública	<code>Settings</code> (modelo de configuração); <code>get_settings()</code> memorizado; <code>ensure_google_env()</code> .
Invariante	<code>get_settings()</code> devolve sempre a mesma instância. Alterar configuração exige reiniciar o processo — o que é a propriedade desejada: configuração não muda em tempo de execução.
Detalhe notável 1	Normalização de URL de banco: provedores gerenciados expõem <code>postgresql://</code> ou o legado <code>postgres://</code> ; o validador converte para o esquema com driver explícito. Sem isso, seria necessário reescrever a variável no painel de cada ambiente.
Detalhe notável 2	Materialização de credencial: a credencial em base64 é decodificada, <i>validada como JSON</i> e escrita em arquivo temporário com permissão <code>0600</code> . A validação antecipada é o que transforma “o serviço falha misteriosamente na primeira chamada” em “o <i>log</i> diz que a variável está inválida”.
Detalhe notável 3	<code>ensure_google_env()</code> faz a ponte entre o objeto de configuração e as variáveis de ambiente que as bibliotecas de nuvem leem diretamente. É idempotente e usa atribuição condicional, respeitando valores já presentes no ambiente.

16.3.2 `schemas.py`

Aspecto	Descrição
Propósito	Esquemas de domínio únicos , compartilhados por API, <i>worker</i> MQTT e ETL.
Interface pública	<code>FEATURE_COLUMNS</code> (lista ordenada de 18 nomes); <code>SensorEvent</code> ; <code>SimilarEvents</code> ; <code>Citation</code> ; <code>Prescription</code> ; <code>DiagnoseResponse</code> ; <code>ChatRequest</code> ; <code>ChatResponse</code> .
Invariante crítico	A ordem de <code>FEATURE_COLUMNS</code> é significativa: ela define a ordem das colunas do vetor de <i>features</i> , e portanto a ordem esperada pelos artefatos treinados. Reordenar a lista sem retreinar produz previsões silenciosamente erradas.

Decisão	SensorEvent aceita campos extras e os ignora. Isso permite enviar o registro original do conjunto de dados — com as colunas redundantes em unidades imperiais — sem pré-processamento.
Separação de contratos	DiagnoseResponse é o contrato <i>público</i> ; DiagnosisResult (em <code>ml/diagnose.py</code>) é a estrutura <i>interna</i> . A separação permite evoluir um sem quebrar o outro.

16.3.3 db.py e severity.py

Módulo	Conteúdo e observações
<code>db.py</code>	Modelos declarativos: FaultFamily , LabelMap , Event , Document , DocCoverage , Diagnosis . Fábricas get_engine , get_session_factory e create_all . A conexão usa verificação prévia de vitalidade, o que evita a falha clássica de conexão reciclada pelo provedor. Event.id não tem geração automática — é a chave de idempotência da ingestão.
<code>severity.py</code>	Domínio de severidade vibracional reutilizável: limites das zonas por classe de máquina, faixas contíguas para desenho de bandas, classify_zone e relative_severity . A classe da máquina é <i>parâmetro</i> , não constante — os limites dependem de potência e rigidez da fundação (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1995). Classe inválida levanta exceção listando as válidas.

16.4 Pacote `src/etl`

Aspecto	Descrição
Propósito	Canonicalização auditável de rótulos brutos em famílias canônicas.
Interface pública	Family (registro imutável); LabelCanonizer.canonize(raw) -> Family ; LabelCanonizer.fault_families() ; get_canonizer() memorizado; UnknownLabelError .
Invariantes	(i) As regras são avaliadas na ordem de declaração e a primeira que casa vence; (ii) toda regra aponta para família existente — validado na construção; (iii) rótulo sem regra levanta exceção , nunca retorna valor padrão.
Ponto de extensão	Acrescentar família ou regra é edição de YAML, sem alteração de código. A ordem da lista de regras é parte da especificação: a regra de estado operacional deve vir <i>depois</i> das de falha.

16.5 Pacote src/ml

16.5.1 features.py

Aspecto	Descrição
Propósito	Engenharia de <i>features</i> física, compartilhada entre treino e inferência.
Interface pública	<code>DERIVED_COLUMNS</code> ; <code>MODEL_COLUMNS</code> (as 23); <code>add_derived_features(df)</code> ; <code>feature_matrix(df)</code> .
Invariante crítico	Este é o único lugar onde as derivadas são calculadas. Treino e inferência chamam a mesma função, o que torna a divergência entre transformações estruturalmente impossível.
Proteção numérica	$\varepsilon = 10^{-6}$ nos denominadores e limite superior de 50 na ordem de rotação. Trata os 658 eventos com rotação nula sem produzir infinitos nem valores extremos que distorceriam a padronização.

Listing 16.2 – Cálculo das *features* derivadas — `src/ml/features.py`

```

1 def add_derived_features(df: pd.DataFrame) -> pd.DataFrame:
2     """Adiciona as features derivadas (recebe DataFrame com FEATURE_COLUMNS)."""
3     out = df.copy()
4     rot_hz = np.maximum(out["rpm"].to_numpy(dtype=float) / 60.0, _EPS)
5
6     # Ordem do pico: os manuais descrevem assinaturas em multiplos da rotacao
7     out["x_peak_order"] = np.clip(out["x_peak_vel_comp_freq_hz"] / rot_hz, 0, _MAX_ORDER)
8     out["z_peak_order"] = np.clip(out["z_peak_vel_comp_freq_hz"] / rot_hz, 0, _MAX_ORDER)
9     # Radial x axial: desbalanceamento e radial; desalinhamento tem axial
10    out["radial_axial_ratio"] = out["x_rms_velocity_mm_s"] / (out["z_rms_velocity_mm_s"] + _EPS)
11    # Alta x baixa frequencia: defeitos de rolamento excitam alta frequencia
12    out["hf_lf_ratio_x"] = out["x_high_freq_rms_accel_g"] / (out["x_rms_acceleration_g"] + _EPS)
13    out["hf_lf_ratio_z"] = out["z_high_freq_rms_accel_g"] / (out["z_rms_acceleration_g"] + _EPS)
14    return out

```

16.5.2 train.py

Aspecto	Descrição
Propósito	Treinar e avaliar o motor de diagnóstico, produzindo os artefatos servidos e o registro de métricas.
Interface pública	<code>load_dataset(csv)</code> ; <code>session_holdout_mask(df, rng)</code> ; <code>train(csv, out_dir) -> dict</code> . Executável como módulo.
Constantes	<code>KNN_NEIGHBORS=25</code> ; <code>SESSION_TEST_FRACTION=0.2</code> ; <code>RANDOM_STATE=42</code> .

Função notável	<code>session_holdout_mask</code> implementa o protocolo principal: para cada família, separa cerca de 20 % das sessões <i>inteiras</i> (mínimo de uma, quando há pelo menos duas). Família com sessão única fica integralmente no treino — não há como avaliar generalização entre campanhas com uma só campanha.
Saída	Quatro artefatos e o registro de métricas com os três protocolos, as classes, a lista de <i>features</i> e o instante do treino. O modelo servido é retreinado com todos os dados após as avaliações.

16.5.3 `diagnose.py`

Aspecto	Descrição
Propósito	Inferência: classe, probabilidade, vizinhos semelhantes e nível de confiança.
Interface pública	<code>DiagnosisEngine(artifacts_dir);</code> <code>DiagnosisEngine.diagnose(event) -> DiagnosisResult;</code> <code>DiagnosisResult; get_engine_singleton().</code>
Invariantes	(i) Os quatro artefatos são carregados na construção; (ii) o <i>singleton</i> memorizado garante uma única carga por processo — crítico porque o índice de vizinhos ocupa a maior parte da memória; (iii) a ordem das colunas do vetor segue <code>MODEL_COLUMNS</code> .
Método notável	<code>_similar_events</code> agrega o histórico completo da família prevista — não apenas os 25 vizinhos — para produzir contagem, período, frequência diária e série mensal. Os vizinhos entram na resposta como identificadores rastreáveis.
Regra de confiança	<code>_confidence</code> usa o <i>produto</i> de probabilidade e concordância, com limiares em 0,70 e 0,40 (Seção 8.7).

16.6 Pacote `src/rag`

16.6.1 `chunking.py`

Aspecto	Descrição
Propósito	Extrair texto de PDF e produzir <i>chunks</i> com metadados de rastreio.
Interface pública	<code>Chunk</code> (texto, doc, título, seção, página); <code>chunk_pdf(pdf_bytes, doc_name, title, transcribe=None).</code>

Constantes	<code>CHUNK_SIZE=1200</code> ; <code>CHUNK_OVERLAP=150</code> ; expressão regular de título de seção.
Estrutura	Duas fontes de blocos: extração direta e transcrição por OCR. Uma função interna consome qualquer uma das duas, o que mantém a lógica de <i>chunking</i> e de rastreo de seção idêntica nos dois caminhos.
Detalhe	O parâmetro <code>transcribe</code> é uma função injetada, não uma dependência importada. Isso mantém o módulo testável sem cliente de linguagem e sem conhecer o provedor.

16.6.2 `store.py` — onde vive o *guardrail*

Aspecto	Descrição
Propósito	Acesso à base vetorial e fonte de verdade da cobertura documental .
Interface pública	<code>COLLECTION</code> ; <code>get_collection()</code> ; <code>add_document(...) -> int</code> ; <code>is_documented(family) -> bool</code> ; <code>documented_families() -> set[str]</code> ; <code>retrieve(query, family, llm, top_k) -> list[dict]</code> .
Invariante do <i>guardrail</i>	<code>is_documented</code> é uma consulta ao índice com filtro por família e limite 1. Não há inferência, não há <i>cache</i> e não há configuração intermediária: o estado do índice é a resposta.
Idempotência	<code>add_document</code> remove os <i>chunks</i> do mesmo arquivo antes de indexar. Recadastrar um procedimento corrigido não duplica conteúdo nem deixa trechos órfãos concorrendo na busca.
Identificador de <i>chunk</i>	Chave composta <code>{arquivo}::{família}::{índice}</code> — determinística, o que torna a reindexação previsível.
Filtro na consulta	<code>retrieve</code> aplica o filtro por família <i>dentro</i> da consulta ao índice, não como pós-filtragem. Isso preserva o número de resultados relevantes para um mesmo <i>top-k</i> .

16.6.3 `service.py`

Aspecto	Descrição
Propósito	Orquestrar o <i>guardrail</i> , a recuperação, a geração e as citações.
Interface pública	<code>UNDOCUMENTED_MSG</code> (modelo da mensagem); <code>prescribe(family, event, llm) -> (Prescription None, str None)</code> ; <code>chat(message, family, history, llm) -> ChatResponse</code> .

Contrato de retorno	<code>prescribe</code> devolve exatamente um dos dois: <i>prescrição</i> ou <i>sugestão</i> . O tipo de retorno torna o caso não documentado explícito na assinatura, em vez de escondido em um campo nulo.
Citações	A função interna de citações deduplica por (documento, seção, página) preservando a ordem de relevância da recuperação, e lê <i>apenas</i> os metadados.
Conversa sem contexto	Quando não há família de referência, a busca percorre todas as famílias documentadas com <i>top-k</i> reduzido e mantém os seis trechos globalmente mais próximos. Custa mais chamadas de <i>embedding</i> , mas evita exigir que o usuário informe a família antes de poder perguntar.

16.6.4 `bootstrap.py`

Aspecto	Descrição
Propósito	Indexar a base documental automaticamente em ambiente novo.
Interface pública	<code>COVERAGE_PATH; ingest_initial_documents(docs_dir, llm=None) -> dict[str, int]; bootstrap_if_empty()</code> .
Três guardas	(i) habilitação por configuração; (ii) índice já povoado dispensa; (iii) índice não inspecionável cancela com advertência. Só depois a <i>thread</i> é iniciada.
Decisão	A <i>thread</i> é do tipo <i>daemon</i> e a indexação ocorre fora do caminho de <i>startup</i> . Bloquear o <i>startup</i> faria o orquestrador reprovar a instância antes de a indexação terminar — em laço infinito (AT-10).
Tratamento de erro	O corpo da <i>thread</i> captura qualquer exceção e a registra com <i>traceback</i> , orientando o uso do <i>endpoint</i> de cadastro. Uma <i>thread</i> que morre em silêncio produziria um sistema que afirma não ter documento algum, sem explicação.

16.7 Pacote `src/llm`

16.7.1 `client.py`

Aspecto	Descrição
Propósito	Interface fina e substituível sobre o serviço de linguagem.
Interface pública	<code>LLMClient</code> (protocolo com <code>generate</code> , <code>embed</code> e <code>transcribe</code>); <code>GeminiClient</code> (implementação); <code>get_llm_client()</code> memorizado.

Decisão central	O contrato é um protocolo estrutural , não uma classe base. O <i>fake</i> de teste não importa nada deste módulo — basta implementar os três métodos. É o que permite à suíte rodar sem a biblioteca de nuvem instalada.
<i>Imports</i> adiados	As bibliotecas de nuvem são importadas <i>dentro</i> dos métodos. Importar o módulo não exige que elas estejam presentes.
Validação de <i>embeddings</i>	O laço envia um texto por requisição — o serviço aceita lista mas devolve silenciosamente um único vetor — e valida a contagem final, levantando exceção em caso de divergência.
Temperaturas	0,2 na geração; 0,0 na transcrição por OCR, onde qualquer criatividade é defeito.

16.7.2 `prompts.py`

Aspecto	Descrição
Propósito	Centralizar as instruções de sistema e a montagem de <i>prompt</i> .
Interface pública	<code>SYSTEM_PRESCRIPTIVE; SYSTEM_AGENT; build_context_block(chunks); build_prescription_prompt(...); build_chat_prompt(...)</code> .
Duas instruções distintas	A instrução do agente é mais longa porque precisa reger o <i>uso de ferramentas</i> : proíbe responder de memória, torna a busca documental obrigatória antes de qualquer procedimento e enumera as famílias canônicas válidas, para que o modelo traduza o vocabulário do usuário (“rotor inclinado”) no identificador canônico.
Bloco de contexto	Numera os trechos e prefixa cada um com sua origem legível. A numeração é o que torna a citação por marcador verificável — o marcador aponta para um trecho concreto no <i>prompt</i> .
Histórico limitado	O <i>prompt</i> de conversa inclui no máximo as seis mensagens mais recentes, limitando o crescimento do contexto no caminho de degradação.

16.7.3 `agent.py`

Aspecto	Descrição
Propósito	Agente de conversa com ferramentas e memória de sessão persistente.
Interface pública	<code>MaintenanceAgent; MaintenanceAgent.ask(message, session_id, user_id) -> ChatResponse; get_agent() -> MaintenanceAgent None</code> .

Isolamento de esquema	O <i>framework</i> de agentes cria uma tabela chamada <code>events</code> , que colidiria com a do domínio. A engine de sessões é criada com o caminho de busca fixado no esquema <code>adk</code> , criado na inicialização.
Conversão de driver	As sessões exigem driver assíncrono; a URL é convertida em tempo de inicialização, cobrindo também o caso de SQLite.
Degradação em três casos	Biblioteca ausente, projeto não configurado ou credencial inválida: em todos, <code>get_agent()</code> devolve nulo, registra advertência e o <i>endpoint</i> recorre ao RAG de um passo. O <i>singleton</i> usa um valor sentinela distinto de nulo para diferenciar “ainda não tentado” de “tentado e indisponível” — sem isso, cada requisição repetiria a inicialização falha.

16.7.4 `agent_tools.py`

Aspecto	Descrição
Propósito	As quatro ferramentas do agente e o coletor de citações.
Interface pública	<code>AGENT_TOOLS</code> ; <code>consultar_historico</code> ; <code>buscar_documentos</code> ; <code>diagnosticar_evento</code> ; <code>cobertura_documental</code> ; <code>reset_citations()</code> ; <code>get_citations()</code> .
<i>Docstrings</i> como contrato	As <i>docstrings</i> em português são a descrição apresentada ao modelo. Elas não são comentário: são especificação de comportamento — por isso declaram explicitamente que a busca documental é obrigatória antes de descrever procedimento.
Coletor por contexto	Uma variável de contexto acumula os metadados dos trechos recuperados, deduplicando por (documento, seção, página). É isolada por tarefa assíncrona, de modo que conversas simultâneas não misturam citações.
Erros como dados	Família inexistente devolve um dicionário com a lista de famílias válidas, em vez de exceção. Isso permite ao modelo se autocorriger na iteração seguinte, o que uma exceção impediria.
Janela temporal ancorada	A consulta ao histórico ancora a janela no evento <i>mais recente do banco</i> , não na data corrente — caso contrário, um conjunto de dados histórico devolveria sempre zero ocorrências.

16.8 Pacote `src/api`

16.8.1 `main.py`

Aspecto	Descrição
Propósito	Definir os roteadores REST e o ciclo de vida da aplicação.
Ciclo de vida	O gerenciador de contexto de inicialização dispara a indexação automática condicional e devolve o controle imediatamente.
Funções	<code>health</code> ; <code>diagnose</code> ; <code>_persist_diagnosis</code> ; <code>chat</code> ; <code>stats</code> ; <code>analytics_severity</code> ; <code>analytics_signatures</code> ; <code>analytics_model_quality</code> ; <code>events</code> ; <code>list_documents</code> ; <code>upload_document</code> .
Auditoria em <i>best-effort</i>	<code>_persist_diagnosis</code> captura qualquer exceção, reverte a transação e retorna. A resposta ao cliente não depende da gravação — um problema de banco não deve impedir o técnico de receber o diagnóstico.
Mensagem de erro acionável	A indisponibilidade do serviço de linguagem devolve <code>503</code> informando quais variáveis configurar e qual comando de autenticação executar, além do tipo da exceção original. Mensagem de erro é interface com o operador.
Validação no cadastro	Famílias inválidas produzem <code>422</code> com a lista das válidas; a indexação vetorial ocorre <i>antes</i> do registro relacional, porque é ela que determina a cobertura.

16.8.2 `deps.py` e `analytics.py`

Módulo	Conteúdo e observações
<code>deps.py</code>	Três dependências injetáveis: sessão de banco (com fechamento garantido), cliente de linguagem e agente de conversa. São o ponto de substituição usado pelos testes — por isso o módulo é minúsculo e sem lógica.
<code>analytics.py</code>	Funções puras que recebem uma engine e devolvem dicionários: <code>severity_by_rpm</code> , <code>fault_signatures</code> , <code>leakage_evidence</code> , <code>model_quality</code> . Proteção de segurança: <code>_safe_columns</code> valida os nomes de coluna contra a constante de <i>features</i> antes de qualquer interpolação em SQL; coluna fora da lista levanta exceção. O módulo documenta explicitamente sua dependência de dialeto — percentis e variâncias amostrais não existem em SQLite.

16.9 Pacote `src/ingestion`

Aspecto	Descrição
Propósito	Consumir telemetria MQTT, validar, persistir, diagnosticar e alertar.
Interface pública	<code>IngestionWorker</code> ; <code>IngestionWorker.run()</code> . Executável como módulo.
Métodos	<code>_on_connect</code> , <code>_on_message</code> , <code>_persist</code> , <code>_diagnose_and_alert</code> , <code>_to_dlq</code> , <code>_machine_from_topic</code> .
Idempotência	<code>_persist</code> usa fusão pela chave primária. É a contrapartida obrigatória de QoS 1.
Ordem deliberada	Persistir precede diagnosticar. Se a API estiver indisponível, o evento já está no histórico — perder o diagnóstico é recuperável, perder o dado não é.
Truncamento na rejeição	Motivo limitado a 2.000 caracteres e <i>payload</i> a 5.000. Um <i>payload</i> malformado gigantesco não deve derrubar o <i>broker</i> nem o assinante da fila de rejeição.
Autenticação condicional	Credenciais são configuradas apenas quando presentes, o que mantém o desenvolvimento local anônimo e a produção autenticada com o mesmo código.

16.10 Pacote `src/ui`

Módulo	Conteúdo e observações
<code>app.py</code>	Navegação lateral e as três seções não analíticas. <code>render_sidebar</code> devolve seção e subpainel; apenas a seção ativa é renderizada. O evento de exemplo do enunciado está embutido como constante, para demonstração em um clique. O caminho do logotipo é resolvido de forma absoluta, para que a aplicação possa ser iniciada de qualquer diretório.
<code>dashboard.py</code>	O maior módulo do projeto (730 linhas): os quatro painéis analíticos, a matriz de priorização, as bandas normativas selecionáveis, o gráfico de validação física, o mapa de assinaturas e os painéis de qualidade do modelo e de vazamento. Cada painel busca apenas os dados de que precisa.
<code>theme.py</code>	Paleta validada e <i>chrome</i> comum dos gráficos. As <i>docstrings</i> registram as decisões cromáticas e o <i>erro evitado</i> : a codificação verde/vermelho reprovada na separação para deuteranopia. Contém a rampa ordinal por regime de rotação e a escala divergente para escores padronizados.

<code>api_client.py</code>	Cliente HTTP com <i>cache</i> de curta duração e invalidação explícita após cadastro de documento — sem isso, a interface continuaria exibindo a cobertura anterior por um minuto.
----------------------------	--

16.11 Scripts de operação

Script	Função e observações
<code>ingest_data.py</code>	Carga do histórico. Audita 100 % dos rótulos antes de gravar; imprime o resumo da carga. Parâmetro: <code>-csv</code> .
<code>ingest_docs.py</code>	Indexação da base documental inicial a partir da declaração de cobertura. Imprime a contagem de <i>chunks</i> por documento e as famílias resultantes. Parâmetro: <code>-docs-dir</code> .
<code>simulate_sensor.py</code>	Simulador de <i>gateway</i> : republica eventos <i>reais</i> do conjunto de dados. Parâmetros: <code>-machine</code> , <code>-rate</code> , <code>-limit</code> , <code>-family</code> e <code>-invalid</code> (para demonstrar a fila de rejeição).

16.12 Pontos de extensão

Tabela 16.21 – Como estender o sistema

Objetivo	Onde alterar
Acrescentar família de falha	Declarar a família e sua regra em <code>src/etl/label_map.yaml</code> , recarregar o histórico e retreinar. Nenhuma alteração de código.
Cobrir uma família com documento	Nenhuma alteração: cadastrar o PDF pela interface ou pela API. Para carga inicial, declarar em <code>src/rag/coverage.yaml</code> .
Trocar o provedor de linguagem	Implementar os três métodos do protocolo em um novo cliente e alterar a fábrica em <code>src/llm/client.py</code> . Nenhum outro módulo muda.
Acrescentar <i>feature</i> derivada	Editar <code>src/ml/features.py</code> (duas listas e uma função) e retreinar. Treino e inferência acompanham automaticamente.
Acrescentar ferramenta ao agente	Escrever a função com <i>docstring</i> descritiva em <code>src/llm/agent_tools.py</code> e incluí-la na lista de ferramentas.
Acrescentar agregação analítica	Escrever a função pura em <code>src/api/analytics.py</code> (usando a lista branca de colunas), expor a rota em <code>src/api/main.py</code> e consumir em <code>src/ui/dashboard.py</code> .
Suportar múltiplos equipamentos	Acrescentar <code>machine_id</code> a <code>events</code> e ao esquema de evento; o identificador já é extraído do tópico MQTT pelo <i>worker</i> .
Trocar o <i>broker</i> ou o protocolo de campo	Substituir <code>src/ingestion/mqtt_worker.py</code> . Ele consome os mesmos esquemas e a mesma API, portanto nenhuma outra parte do sistema é afetada.

Riscos e plano de evolução

17.1 Matriz de riscos

Cada risco está registrado com probabilidade, impacto, a mitigação já implementada e a ação recomendada. A honestidade aqui é deliberada: os dois riscos de severidade mais alta são *conhecidos e não resolvidos*, e ambos decorrem de limitações do dado, não de escolhas de engenharia.

Tabela 17.1 – Matriz de riscos

ID	Risco	Prob.	Impacto	Mitigação atual e ação recomendada
RI-01	O classificador não generaliza para novas campanhas de coleta (F1 = 0,195)	Alta	Alto	Mitigado, não resolvido. A busca por similaridade é a resposta operacional principal; o gate de confiança sinaliza incerteza; o painel expõe os três protocolos. Ação: coletar múltiplas campanhas por condição (Seção 8.6).
RI-02	Ausência da forma de onda impede as assinaturas espectrais de rolamento	Alta	Alto	Não mitigável com o dado atual. As <i>features</i> derivadas aproximam o que é possível. Ação: negociar acesso ao sinal bruto do sensor.
RI-03	Indisponibilidade do serviço de linguagem	Média	Médio	Diagnóstico e quantificação seguem funcionando; a conversa degrada para RAG de um passo; a mensagem de erro é acionável. Ação: avaliar <i>cache</i> de prescrições por família.
RI-04	Perda do índice vetorial em nova implantação	Baixa	Alto	Volume persistente dedicado e indexação automática quando o índice está vazio. Ação: incluir a contagem de <i>chunks</i> em monitoramento ativo.
RI-05	Broker exposto sem autenticação	Baixa	Alto	Autenticação obrigatória quando as credenciais existem; advertência explícita em <i>log</i> quando não. Ação: habilitar TLS e tornar a autenticação incondicional em produção.
RI-06	Ausência de autenticação de usuário na interface e na API	Média	Médio	Fora do escopo acordado. Ação: autenticação com perfis (leitura, operação, curadoria documental) antes de qualquer uso produtivo.
RI-07	Custo de nuvem excede o crédito disponível	Alta	Baixo	Topologia reduzível a três serviços sem perda de capacidade exigida. Ação: manter o par de ingestão industrial apenas localmente.
RI-08	Prescrição gerada contém passo perigoso	Baixa	Alto	Contexto fechado com citação verificável; a instrução de sistema exige a seção de segurança primeiro. Ação: revisão humana obrigatória de prescrição para intervenção de risco — o sistema é apoio à decisão, não autoridade.
RI-09	Rótulo novo em	Média	Baixo	Comportamento desejado , não defeito: a

17.2 Plano de evolução

17.2.1 Curto prazo — dados e modelo

1. **Coletar múltiplas campanhas por condição.** É a única ação que ataca a causa de **RI-01**. Sem ela, qualquer métrica de generalização permanece limitada por construção.
2. **Normalizar contra a linha de base do próprio equipamento.** Substituir valores absolutos pelo desvio contra a referência recente da máquina. A referência existe em produção — ao contrário da identidade da campanha — de modo que a transformação remove o deslocamento de sessão preservando o sinal físico.
3. **Adotar *features* invariantes à condição operacional:** razões entre eixos e entre bandas de frequência, e a derivada da amplitude em relação a ω^2 , indicada pela análise de validação física.
4. **Instrumentar explicabilidade local** por atribuição de contribuição ([Lundberg; Lee, 2017](#)), expondo ao técnico *quais* grandezas sustentaram cada diagnóstico. Hoje o painel mostra apenas a importância global.

17.2.2 Médio prazo — produto e integração

1. **Autenticação com perfis (**RI-06**):** leitura, operação e curadoria documental, com registro de quem cadastrou cada procedimento.
2. **Suporte a múltiplos equipamentos:** acrescentar identificador de máquina ao esquema de evento e à tabela de histórico — o identificador já é extraído do tópico MQTT.
3. **Priorização por custo:** substituir a *proxy* de frequência $\times$ severidade vibracional por custo de parada e criticidade do ativo. A matriz de priorização documental ganharia significado econômico.
4. **Ciclo de *feedback* do técnico:** registrar se a prescrição resolveu o problema. É o dado que hoje falta para avaliar a utilidade da prescrição — e não apenas a acurácia da classificação.

17.2.3 Longo prazo — plataforma

1. **Acesso à forma de onda** e extração de envelope e espectro. É o que desbloqueia as frequências características de rolamento ([Randall; Antoni, 2011](#)) e, com elas, o diagnóstico de defeito incipiente.
2. **Prognóstico de vida útil remanescente** conforme a estrutura normativa de prognóstico ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2015](#)), viável quando houver histórico de degradação até a falha.
3. **OPC UA e Sparkplug B** para integração direta com controladores e padronização semântica da carga útil.
4. **TLS no *broker*** e autenticação incondicional (**RI-05**).

5. **Monitoramento ativo** com métricas de negócio: distribuição de confiança dos diagnósticos ao longo do tempo, cobertura documental, profundidade da fila de rejeição e latência da geração.

17.3 Dívida técnica reconhecida

- **Ausência de teste automatizado do *worker* MQTT** — verificado por roteiro manual (Seção 10.6).
- **Ausência de teste da interface** — camada de apresentação sem lógica de domínio.
- **Cobertura documental espelhada em duas estruturas** — tabela relacional e índice vetorial. A fonte de verdade é o índice; a tabela é espelho para consulta. É risco controlado, mas é duplicação.
- **Um documento indexado *por família*** multiplica o armazenamento. Irrelevante em 268 *chunks*, revisável em escala de milhares.
- **Ausência de teste de carga** — os números de latência citados vêm de medição pontual, não de teste sistemático.
- **Migração de esquema não versionada** — o esquema é criado por reflexão dos modelos. Uso produtivo exigiria ferramenta de migração versionada.

Conclusão

18.1 Síntese da solução

O sistema documentado nestas páginas transforma um evento de sensores de vibração em três respostas acionáveis — o tipo de defeito, o histórico de ocorrências semelhantes e o procedimento de correção fundamentado na base documental da planta — e sabe, com certeza verificável, quando *não* tem procedimento a oferecer.

A arquitetura repousa sobre uma fronteira única e explícita: tudo que constitui garantia é determinístico e testável; o modelo generativo atua apenas na redação de um texto cujo conteúdo ele não escolheu e não pode ampliar.

Tabela 18.1 – Aderência ao enunciado

Exigência do enunciado	Estado	Como é atendida
Identificar o tipo de defeito a partir de um evento JSON	✓ Atendido	Classificador <i>gradient boosting</i> sobre 23 <i>features</i> , com probabilidade e gate de confiança cruzando duas evidências independentes.
Quantificar ocorrências históricas semelhantes	✓ Atendido	Busca de vizinhos sobre o histórico completo (166.796 eventos), com contagem, período, frequência diária, série mensal e identificadores rastreáveis.
Gerar instruções de correção a partir da base documental	✓ Atendido	Recuperação aumentada com contexto fechado e citação de documento, seção e página derivada dos metadados do <i>retrieval</i> .
Restringir-se a falhas documentadas; caso contrário, avisar e sugerir registro	✓ Atendido	<i>Guardrail</i> determinístico anterior à chamada do modelo. Verificado por teste que assegura zero chamadas ao modelo para família sem documento.
Executar na estação de trabalho especificada	✓ Atendido	74 MiB de artefatos, base vetorial embarcada e nenhuma inferência local de modelo de linguagem.
Python como linguagem principal	✓ Atendido	Solução integral em Python 3.12 — API, interface, <i>worker</i> e <i>scripts</i> .

18.2 Contribuições além do exigido

Contribuição	Por que agrega valor
Integração industrial completa por MQTT	Fecha o ciclo sensor → diagnóstico → alerta → planta, com garantias de entrega, idempotência e fila de rejeição. É como um supervisorio real consumiria a solução.
Agente com ferramentas na conversa	Permite perguntas exploratórias sobre histórico e procedimentos, com memória de sessão — mantendo o caminho crítico determinístico.
Diagnóstico honesto do vazamento de dados	A investigação identificou o mecanismo, testou a hipótese, registrou sua refutação parcial e propôs a correção correta — que é de coleta, não de modelagem.

Análise exploratória que mudou o produto	Três achados alteraram decisões concretas: a severidade agregada é enganosa, a curtose não discrimina neste conjunto de dados, e a validação física confirma a lei do desbalanceamento.
Acessibilidade cromática validada	A codificação intuitiva verde/vermelho foi reprovada em teste de separação para deuteranopia e substituída. O registro do erro evitado está no código.
Infraestrutura como código	Topologia versionada e revisável, com segredos declarados por nome e nunca por valor.
Indexação documental automática	Uma implantação nova fica funcional sem passo manual — inclusive com OCR do documento escaneado.

18.3 O que este documento afirma e o que não afirma

Afirma, com verificação: que o sistema nunca prescreve procedimento para falha sem documento; que as citações apontam para trechos reais; que os números históricos vêm de consulta ao banco; que a ingestão não perde nem duplica evento; e que a suíte de 63 testes roda sem rede.

Não afirma: que o classificador reconhece falhas confiavelmente em uma campanha de coleta nova — o F1 macro de 0,195 no protocolo honesto diz o contrário, e a causa está diagnosticada na Seção 8.6. Também não afirma que a bancada representa uma planta industrial real, nem que a prescrição gerada substitui o julgamento do técnico.

18.4 Consideração final

O maior risco de um sistema de manutenção prescritiva não é errar o diagnóstico — é errar com aparência de certeza. Um painel que anunciasse 89 % de acurácia e um assistente que redigisse procedimentos plausíveis para falhas sem documentação produziriam uma ferramenta convincente e inútil, cuja credibilidade se perderia na primeira intervenção equivocada.

As decisões estruturais deste projeto foram tomadas contra esse risco: o *guardrail* determinístico, a citação derivada de metadados, o gate de confiança duplo, os três protocolos de avaliação expostos no painel e o registro explícito das limitações. O resultado é um sistema que responde o que sabe, declara o que não sabe e mostra, em cada caso, de onde veio a resposta — o que é a condição para que um técnico possa confiar nele.

Apêndice A

Fontes Mermaid dos diagramas

Os diagramas do corpo do documento são vetoriais e desenhados nativamente em $\text{T}_{\text{E}}\text{X}$, o que garante tipografia uniforme e nitidez em qualquer ampliação. Este apêndice fornece o código-fonte **Mermaid** equivalente de cada um, para uso direto em `README.md`, plataformas de repositório, *wikis* e ferramentas de documentação que renderizam Mermaid nativamente.

NOTA

Duas notações do corpo do documento não têm equivalente exato em Mermaid: o diagrama de casos de uso (Figura 4.1) — Mermaid não implementa a notação UML de casos de uso, e a aproximação abaixo usa um grafo com agrupamentos — e as raias dos diagramas de atividade, expressas aqui como subgrafos rotulados.

A.1 Visão geral do sistema

Listing A.1 – Visão geral — fluxo completo entre entrada, aplicação e nuvem

```
1 flowchart TB
2     subgraph ENTRADA["Entrada"]
3         SENS["Sensores IIoT / simulador<br/>(replay do dataset)"]
4         EV["Novo evento JSON<br/>(REST direto)"]
5         USR["Usuario<br/>(chat / novos documentos)"]
6     end
7
8     subgraph APP["Estacao de trabalho (docker-compose)"]
9         MQTT["Broker MQTT<br/>Eclipse Mosquitto<br/>topic: sensors/+/telemetry"]
10        WRK["Ingestion Worker<br/>(paho-mqtt -> valida -> Postgres<br/>-> dispara /diagnose ->
11        ↪ alerts)"]
12        subgraph UI["Frontend - Streamlit"]
13            DASH["Dashboard"]
14            CHAT["Chat prescritivo"]
15            UPL["Upload de documentos"]
16        end
17        subgraph API["Backend - FastAPI"]
18            RT["/diagnose /chat /events<br/>/stats /documents /health"]
19            DIAG["Motor de Diagnostico<br/>KNN + LightGBM"]
20            GUARD["Guardrail de cobertura<br/>documental (deterministico)"]
21            RAGS["Servico RAG<br/>(retrieve -> prompt -> cite)"]
22        end
23    end
```

```

22     PG[("PostgreSQL<br/>historico de eventos")]
23     CH[("ChromaDB<br/>chunks dos documentos")]
24     ART[("Artefatos ML<br/>scaler + knn + lgbm")]
25 end
26
27 subgraph GCP["Google Cloud – Vertex AI"]
28     EMB["gemini-embedding-2"]
29     LLM["gemini-3.6-flash"]
30 end
31
32 SENS -->|publish JSON| MQTT
33 MQTT -->|subscribe| WRK
34 WRK --> PG
35 WRK --> RT
36 EV --> RT
37 USR --> CHAT & UPL
38 DASH & CHAT & UPL --> RT
39 RT --> DIAG --> GUARD
40 GUARD -->|familia documentada| RAGS
41 GUARD -->|sem documento| RT
42 DIAG <--> PG
43 DIAG <--> ART
44 RAGS <--> CH
45 RAGS --> LLM
46 UPL -->|ingestao| CH
47 RAGS & UPL --> EMB

```

A.2 C4 nível 1 – Contexto (Figura 5.1)

Listing A.2 – C4 nível 1 – contexto do sistema

```

1  C4Context
2      title Sistema de Manutencao Prescritiva – Contexto
3
4      Person(tec, "Tecnico de manutencao", "Diagnostica e executa o procedimento")
5      Person(eng, "Engenheiro de manutencao", "Analisa historico e documenta")
6      Person(dat, "Engenheiro de dados", "Carrega, treina e monitora")
7
8      System(pmx, "Sistema de Manutencao Prescritiva",
9          "Identifica o defeito, quantifica ocorrencias semelhantes e prescreve a
10         correcao fundamentada na base documental – restrito a falhas documentadas")
11
12      System_Ext(gw, "Gateway de aquisicao IIoT", "Sensores de vibracao")
13      System_Ext(scada, "SCADA / CMMS", "Ordens de servico")
14      System_Ext(vertex, "Vertex AI – Gemini", "Embeddings, geracao e OCR")
15
16      Rel(tec, pmx, "Diagnostica, consulta")
17      Rel(eng, pmx, "Analisa, cadastra documento")
18      Rel(dat, pmx, "Carrega, treina")
19      Rel(gw, pmx, "Telemetria", "MQTT")
20      Rel(pmx, scada, "Alertas, DLQ", "MQTT")
21      BiRel(pmx, vertex, "Embeddings, geracao, OCR", "HTTPS")
22
23      UpdateLayoutConfig($c4ShapeInRow="3", $c4BoundaryInRow="1")

```

A.3 C4 nível 2 – Contêineres (Figuras 5.2 a 5.4)

Listing A.3 – C4 nível 2 – contêineres (os três planos em uma única vista)

```

1  C4Container
2      title Sistema de Manutencao Prescritiva - Containeres
3
4      Person(tec, "Tecnico de manutencao", "")
5      Person(eng, "Engenheiro de manutencao", "")
6      Person(dat, "Engenheiro de dados / MLOps", "")
7
8      System_Boundary(pmx, "Sistema de Manutencao Prescritiva") {
9          Container(ui, "Interface", "Streamlit + Plotly", "Paineis, diagnostico, conversa,
10         ↪ documentos")
11         Container(api, "API de aplicacao", "FastAPI + Uvicorn", "Diagnostico, prescricao,
12         ↪ conversa, analitica")
13         Container(wk, "Worker de ingestao", "paho-mqtt", "Valida, persiste, diagnostica,
14         ↪ alerta")
15         ContainerQueue(mq, "Broker MQTT", "Eclipse Mosquitto 2", "telemetry / alerts / dlq")
16         Container(scr, "Scripts de preparacao", "Python 3.12 CLI", "ETL, treino, ingestao
17         ↪ documental, simulador")
18         ContainerDb(pg, "Historico", "PostgreSQL 16", "Eventos, taxonomia, documentos,
19         ↪ auditoria")
20         ContainerDb(ch, "Base vetorial", "ChromaDB (HNSW)", "Chunks + metadados; fonte da
21         ↪ cobertura")
22         Container(art, "Artefatos de ML", "joblib", "scaler, lgbm, knn, index_meta
23         ↪ (74 MB)")
24     }
25
26      System_Ext(gw, "Gateway IIoT", "Publica telemetria")
27      System_Ext(scada, "SCADA / CMMS", "Consume alertas")
28      System_Ext(vertex, "Vertex AI", "Gemini")
29
30      Rel(tec, ui, "Usa", "HTTPS")
31      Rel(eng, ui, "Usa", "HTTPS")
32      Rel(dat, scr, "Executa", "CLI")
33      Rel(ui, api, "Consume", "JSON/HTTP (rede privada)")
34      Rel(api, pg, "Le e escreve", "SQLAlchemy")
35      Rel(api, ch, "Consulta e indexa", "")
36      Rel(api, art, "Carrega em memoria", "joblib")
37      Rel(api, vertex, "Embeddings e geracao", "HTTPS")
38      Rel(gw, mq, "Publica telemetria", "MQTT QoS 1")
39      Rel(mq, wk, "Entrega", "MQTT QoS 1")
40      Rel(wk, pg, "merge idempotente", "")
41      Rel(wk, api, "POST /diagnose", "HTTP")
42      Rel(wk, mq, "Publica alerts / dlq", "MQTT QoS 1")
43      Rel(mq, scada, "Entrega alertas", "MQTT")
44      Rel(scr, pg, "Carga do historico", "")
45      Rel(scr, ch, "Indexacao inicial", "")
46      Rel(scr, art, "Gravacao do treino", "")

```

A.4 C4 nível 3 – Componentes da API (Figura 5.5)

Listing A.4 – C4 nível 3 – componentes do contêiner de API

```

1  C4Component
2      title API de aplicacao – Componentes
3
4      Container_Boundary(api, "API de aplicacao (FastAPI)") {
5          Component(rt, "Roteadores REST", "src/api/main.py", "/diagnose /chat /stats /
↳ events /documents /health /analytics/*")
6          Component(dep, "Injecao de dependencias", "src/api/deps.py", "Sessao, cliente de
↳ linguagem, agente – substituiuveis em teste")
7          Component(mot, "Motor de diagnostico", "src/ml/diagnose.py", "LightGBM + KNN + gate de
↳ confianca")
8          Component(rag, "Servico RAG", "src/rag/service.py", "Recupera, monta prompt,
↳ cita")
9          Component(gd, "Guardrail de cobertura", "src/rag/store.py", "Decisao deterministica")
10         Component(ag, "Agente e ferramentas", "src/llm/agent*.py", "4 ferramentas, memoria de
↳ sessao")
11         Component(anl, "Agregacoes analiticas", "src/api/analytics.py", "SQL sob demanda")
12         Component(orm, "Modelos ORM", "src/core/db.py", "SQLAlchemy 2.0")
13         Component(vs, "Adaptador vetorial", "src/rag/store.py", "ChromaDB")
14         Component(cli, "Cliente de linguagem", "src/llm/client.py", "Protocolo substituiuvel")
15         Component(cn, "Canonizador", "src/etl/canonize.py", "Regras regex ordenadas")
16         Component(bs, "Bootstrap documental", "src/rag/bootstrap.py", "Indexacao no primeiro boot
↳ ")
17         Component(cfg, "Configuracao", "src/core/config.py", "pydantic-settings")
18         Component(sch, "Esquemas de dominio", "src/core/schemas.py", "pydantic v2 – UNICOS")
19     }
20
21     ContainerDb(pg, "PostgreSQL", "", "")
22     ContainerDb(ch, "ChromaDB", "", "")
23     System_Ext(vx, "Vertex AI", "")
24
25     Rel(rt, mot, "Diagnostica")
26     Rel(rt, rag, "Prescreve")
27     Rel(rt, ag, "Conversa")
28     Rel(rt, anl, "Agrega")
29     Rel(rt, bs, "Dispara no startup")
30     Rel(dep, cli, "Fornece")
31     Rel(mot, rag, "Familia prevista")
32     Rel(rag, gd, "Consulta cobertura")
33     Rel(ag, gd, "Reutiliza (dentro da ferramenta)")
34     Rel(mot, orm, "")
35     Rel(rag, vs, "")
36     Rel(gd, vs, "")
37     Rel(vs, ch, "HNSW / cosseno")
38     Rel(orm, pg, "")
39     Rel(anl, pg, "SQL")
40     Rel(cli, vx, "HTTPS")
41     Rel(mot, cn, "")

```

A.5 C4 nível 4 – Classes (Figura 5.6)

Listing A.5 – C4 nível 4 – classes do motor de diagnóstico

```

1  classDiagram
2      class SensorEvent {
3          +int id

```

```

4      +datetime created_at
5      +str fault
6      +float z_rms_velocity_mm_s
7      +float x_rms_velocity_mm_s
8      +float temperature_c
9      +float ...15 demais grandezas
10     +float rpm
11     +feature_vector() list~float~
12 }
13 class DiagnosisEngine {
14     -StandardScaler scaler
15     -LGBMClassifier clf
16     -NearestNeighbors knn
17     -DataFrame index_meta
18     -LabelCanonizer canonizer
19     +diagnose(event) DiagnosisResult
20     -_similar_events(family, neighbors)
21     -_confidence(probability, agreement) str
22 }
23 class DiagnosisResult {
24     +str family
25     +bool is_fault
26     +float probability
27     +float knn_agreement
28     +str confidence
29     +SimilarEvents similar
30 }
31 class SimilarEvents {
32     +int count
33     +datetime first_seen
34     +datetime last_seen
35     +float freq_per_day
36     +list timeline
37     +list neighbor_ids
38     +float neighbor_agreement
39 }
40 class LabelCanonizer {
41     -dict families
42     -list _rules
43     +canonize(raw_label) Family
44     +fault_families() list~str~
45 }
46 class DiagnoseResponse {
47     +str predicted_fault
48     +bool is_fault
49     +float probability
50     +float knn_agreement
51     +str confidence
52     +SimilarEvents similar_events
53     +bool documented
54     +Prescription prescription
55     +str suggestion
56 }
57
58 DiagnosisEngine ..> SensorEvent : usa
59 DiagnosisEngine ..> DiagnosisResult : produz
60 DiagnosisEngine *-- LabelCanonizer
61 DiagnosisResult *-- SimilarEvents
62 DiagnosisResult ..> DiagnoseResponse : mapeia para
63 DiagnoseResponse *-- SimilarEvents

```


A.6 Modelo entidade-relacionamento (Figura 6.1)

Listing A.6 – Modelo entidade-relacionamento

```

1  erDiagram
2      FAULT_FAMILIES ||--o{ LABEL_MAP : canonicaliza
3      FAULT_FAMILIES ||--o{ EVENTS : classifica
4      FAULT_FAMILIES ||--o{ DOC_COVERAGE : cobre
5      FAULT_FAMILIES ||--o{ DIAGNOSES : preve
6      DOCUMENTS ||--o{ DOC_COVERAGE : referencia
7      EVENTS ||--o{ DIAGNOSES : gera
8
9      FAULT_FAMILIES {
10         int id PK
11         text name UK
12         bool is_fault
13     }
14     LABEL_MAP {
15         text raw_label PK
16         int family_id FK
17     }
18     EVENTS {
19         bigint id PK "identificador de origem, sem autoincrement"
20         timestampz created_at "indexado"
21         text raw_fault "identifica tambem a campanha de coleta"
22         int family_id FK "indexado, pode ser nulo"
23         float z_rms_velocity_mm_s
24         float x_rms_velocity_mm_s
25         float temperature_c
26         float demais_15_grandezas
27         float rpm
28     }
29     DOCUMENTS {
30         int id PK
31         text filename
32         text title
33         timestampz ingested_at
34         text status
35     }
36     DOC_COVERAGE {
37         int family_id PK
38         int document_id PK
39     }
40     DIAGNOSES {
41         bigint id PK
42         bigint event_id "sem FK: evento ad hoc"
43         int predicted_family_id FK
44         float probability
45         json neighbors
46         json llm_response
47         timestampz created_at
48     }

```

A.7 Sequência do diagnóstico (Figura 5.7)

Listing A.7 – Sequência do diagnóstico completo

```

1 sequenceDiagram
2     participant U as Solicitante (tecnico ou worker)
3     participant A as API (FastAPI)
4     participant D as Motor (LightGBM + KNN)
5     participant P as Dados (Postgres + artefatos)
6     participant G as Guardrail + RAG (Chroma)
7     participant V as Vertex AI (Gemini)
8
9     U->>A: POST /api/v1/diagnose (JSON do evento)
10    A->>A: valida esquema SensorEvent
11    A->>D: engine.diagnose(event)
12    D->>D: 18 brutas + 5 derivadas, padronizadas
13    D->>P: predict_proba + kneighbors (k=25)
14    P-->>D: familia, probabilidade, 25 vizinhos
15    D->>D: concordancia e gate de confianca
16    D->>P: estatisticas historicas da familia
17    P-->>D: contagem, periodo, freq/dia, serie mensal
18    D-->>A: DiagnosisResult
19
20    alt familia de falha COM documento indexado
21        A->>G: prescribe(family, event)
22        G->>V: embedding + geracao em contexto fechado
23        V-->>G: instrucoes em Markdown
24        G-->>A: prescricao + citacoes dos metadados
25    else familia de falha SEM documento indexado
26        A->>G: is_documented(family) -> falso
27        Note over G,V: O Vertex AI NAO e chamado.<br/>Aviso + sugestao de registro.
28        G-->>A: suggestion
29    end
30
31    A-->>U: DiagnoseResponse + auditoria persistida

```

A.8 Sequência da ingestão MQTT (Figura 10.1)**Listing A.8 – Sequência da ingestão industrial**

```

1 sequenceDiagram
2     participant S as Simulador / gateway
3     participant B as Broker (Mosquitto)
4     participant W as Worker (paho-mqtt)
5     participant P as PostgreSQL
6     participant A as API /diagnose
7
8     S->>B: publish sensors/maquina-01/telemetry (QoS 1)
9     B->>W: entrega (subscribe sensors/+/telemetry)
10    W->>W: valida com o schema pydantic UNICO da API
11
12    alt payload valido
13        W->>W: canonicaliza a anotacao, se houver
14        W->>P: session.merge(Event) - idempotente por PK
15        W->>A: POST /api/v1/diagnose
16        A-->>W: diagnostico completo
17        W->>B: publish sensors/maquina-01/alerts (se for falha)
18        B->>S: entrega ao SCADA / CMMS assinante

```

```

19     else payload invalido
20         W->>B: publish sensors/maquina-01/dlq (motivo + payload)
21         Note over W,P: Nada e gravado no banco.<br/>O dado permanece recuperavel na DLQ.
22     end

```

A.9 Sequência da conversa com agente (Figura 9.3)

Listing A.9 – Sequência da conversa com o agente

```

1  sequenceDiagram
2      participant U as Usuario (interface)
3      participant E as POST /chat (FastAPI)
4      participant G as Agente (LlmAgent)
5      participant T as Ferramentas (4 funcoes)
6      participant R as Recursos (Postgres, Chroma, Vertex)
7
8      U->>E: pergunta + session_id
9      alt agente disponivel
10         E->>G: ask(message, session_id)
11         G->>G: recupera memoria da sessao (schema adk)
12         loop enquanto precisar de ferramenta
13             G->>T: decide e invoca ferramenta
14             T->>R: SQL ou consulta vetorial
15             R-->>T: dados estruturados
16             T->>T: registra citacoes no coletor de contexto
17             T-->>G: resultado JSON
18         end
19         G->>R: geracao final em contexto fechado
20         R-->>E: texto em Markdown
21     else agente indisponivel (sem lib ou credenciais)
22         E->>R: RAG de um passo (rag_service.chat)
23         R-->>E: texto em Markdown
24     end
25     E-->>U: resposta + citacoes do coletor + selo de modo

```

A.10 Pipeline de dados e documentos (Figuras 7.1 e 9.1)

Listing A.10 – Pipeline de dados e pipeline documental

```

1  flowchart LR
2      subgraph ETL ["ETL de dados (scripts/ingest_data.py)"]
3          CSV["banner.csv<br/>166.796 linhas"] --> CLEAN["Limpeza + dedup +<br/>validacao"]
4          CLEAN --> CANON["Canonicalizacao<br/>label_map.yaml<br/>151 rotulos -> 17 familias"]
5          CANON -->|rotulo desconhecido| ERR["Excecao: nada e gravado"]
6          CANON --> PGL["PostgreSQL<br/>events + fault_families<br/>+ label_map"]
7          PGL --> FEAT["Treino (src/ml/train.py):<br/>StandardScaler + KNN + LightGBM"]
8          FEAT --> ARTS["artifacts/<br/>joblib + metrics.json"]
9      end
10     subgraph DOCPipe ["Ingestao documental (scripts/ingest_docs.py)"]
11         PDFS["Doc1..Doc6.pdf<br/>+ novos uploads via API"] --> PARSE["Extracao pymupdf<br/>por secas"]
12         PARSE -->|sem camada de texto| OCR["OCR multimodal<br/>(Gemini, 150 dpi)"]

```

```

13     OCR --> CHUNK
14     PARSE --> CHUNK["Chunks ~1200 chars c/ overlap<br/>metadados: doc, secão, página, família"]
15     CHUNK --> GEMB["gemini-embedding-2<br/>(dim 768)"]
16     GEMB --> CHR["ChromaDB<br/>collection: manuals"]
17     end

```

A.11 Casos de uso (Figura 4.1)

Listing A.11 – Casos de uso – aproximação em grafo (Mermaid não implementa a notação UML de casos de uso)

```

1  flowchart LR
2      tec(["Técnico de manutenção"])
3      eng(["Engenheiro de manutenção"])
4      dat(["Engenheiro de dados / MLOps"])
5      gw(["Gateway de aquisição IIoT"])
6      scada(["SCADA / CMMS"])
7      vx(["Vertex AI (Gemini)"])
8
9      subgraph SIS["Sistema de Manutenção Prescritiva"]
10         UC01["UC-01 Diagnosticar novo evento"]
11         UC02["UC-02 Obter instruções de correção"]
12         UC03["UC-03 Tratar falha não documentada"]
13         UC04["UC-04 Registrar documento orientativo"]
14         UC05["UC-05 Consultar cobertura documental"]
15         UC06["UC-06 Conversar com assistente"]
16         UC07["UC-07 Carregar histórico de eventos"]
17         UC08["UC-08 Treinar motor de diagnóstico"]
18         UC09["UC-09 Ingerir telemetria industrial"]
19         UC10["UC-10 Tratar telemetria inválida"]
20         UC11["UC-11 Analisar histórico no painel"]
21         UC12["UC-12 Verificar saúde do sistema"]
22     end
23
24     tec --- UC01
25     tec --- UC06
26     eng --- UC01
27     eng --- UC04
28     eng --- UC11
29     dat --- UC07
30     dat --- UC08
31     dat --- UC12
32     gw --- UC09
33     UC09 --- scada
34     UC10 --- scada
35     vx --- UC02
36     vx --- UC04
37     vx --- UC06
38
39     UC01 -.->|include| UC02
40     UC03 -.->|extend| UC02
41     UC09 -.->|include| UC01
42     UC10 -.->|extend| UC09
43     UC06 -.->|include| UC05
44
45     classDef sec fill:#f5f5f1,stroke:#c3c2b7;

```

```
46 class UC03,UC10 sec;
```

A.12 Diagramas de atividade (Capítulo 13)

Listing A.12 – AT-01 – carga do histórico corporativo

```
1 flowchart TD
2   I(( )) --> A1["Ler o CSV com tipagem de datas"]
3   A1 --> A2["Normalizar os rotulos brutos"]
4   A2 --> A3["Canonicalizar TODOS os rotulos distintos"]
5   A3 --> D1{"Todos casam com alguma regra?"}
6   D1 -->|nao| E1["Levantar excecao nomeando o rotulo"]
7   E1 --> FE((( )))
8   D1 -->|sim| A4["Deduplicar por id e descartar linhas incompletas"]
9   A4 --> A5["Criar as tabelas, se ausentes"]
10  A5 --> A6["Popular fault_families e label_map"]
11  A6 --> A7["Carregar os eventos de forma idempotente"]
12  A7 --> A8["Imprimir o resumo da carga"]
13  A8 --> F((( )))
```

Listing A.13 – AT-02 – canonicalização de um rótulo bruto

```
1 flowchart TD
2   I(( )) --> A1["Normalizar: minusculas, sem espacos nas bordas"]
3   A1 --> A2["Tomar a proxima regra da lista ORDENADA"]
4   A2 --> D1{"A expressao casa?"}
5   D1 -->|sim| OK["Devolver a familia da regra"]
6   OK --> F((( )))
7   D1 -->|nao| D2{"Ha mais regras?"}
8   D2 -->|sim| A2
9   D2 -->|nao| ER["Excecao: rotulo exige curadoria humana"]
10  ER --> FE((( )))
```

Listing A.14 – AT-03 – treino do motor de diagnóstico

```
1 flowchart TD
2   I(( )) --> A1["Carregar e canonicalizar o dataset"]
3   A1 --> A2["Construir a matriz: 18 brutas + 5 derivadas"]
4   A2 --> FORK[" "]
5   FORK --> V1["Avaliacao 1<br/>holdout por sessao"]
6   FORK --> V2["Avaliacao 2<br/>divisao aleatoria estratificada"]
7   FORK --> V3["Avaliacao 3<br/>sessao, sem temperatura"]
8   V1 --> JOIN[" "]
9   V2 --> JOIN
10  V3 --> JOIN
11  JOIN --> A3["Retreinar com 100% dos dados"]
12  A3 --> A4["Indexar o historico completo no KNN"]
13  A4 --> A5["Gravar 4 artefatos + metrics.json"]
14  A5 --> F((( )))
```

Listing A.15 – AT-04 – indexação de um documento orientativo

```

1 flowchart TD
2   I(( )) --> A1["Remover chunks anteriores do mesmo arquivo"]
3   A1 --> A2["Extrair texto por pagina, rastreando a secao"]
4   A2 --> D1{"Algum chunk foi produzido?"}
5   D1 -->|nao: escaneado| OCR["Renderizar a 150 dpi e transcrever por OCR"]
6   OCR --> A3
7   D1 -->|sim| A3["Acumular chunks ~1200 chars, overlap 150"]
8   A3 --> A4["Gerar embeddings (RETRIEVAL_DOCUMENT, dim 768)"]
9   A4 --> D2{"Contagem de vetores = de textos?"}
10  D2 -->|nao| ER["Excecao: desalinhamento silencioso"]
11  ER --> FE((( )))
12  D2 -->|sim| A5["Indexar um conjunto por familia, com metadados"]
13  A5 --> A6["Registrar documento e cobertura no banco"]
14  A6 --> F((( )))

```

Listing A.16 – AT-05 – diagnóstico de um novo evento

```

1 flowchart TD
2   I(( )) --> A1["Validar payload contra SensorEvent"]
3   A1 --> D0{"Esquema valido?"}
4   D0 -->|nao| E0["Responder 422 com os campos inconsistentes"]
5   E0 --> F0((( )))
6   D0 -->|sim| A2["Calcular as 5 derivadas e padronizar"]
7   A2 --> A3["Classificar: familia e probabilidade"]
8   A3 --> A4["Recuperar 25 vizinhos e calcular concordancia"]
9   A4 --> A5["Agregar estatisticas historicas da familia"]
10  A5 --> A6["Classificar a confianca (produto dos dois sinais)"]
11  A6 --> D1{"E falha?"}
12  D1 -->|nao: estado| A10
13  D1 -->|sim| A7["Consultar o guardrail de cobertura"]
14  A7 --> D2{"Documentada?"}
15  D2 -->|sim| A8["Recuperar, gerar e citar (AT-06)"]
16  D2 -->|nao| A9["Aviso + sugestao de registro"]
17  A8 --> A11
18  A9 --> A11
19  A10["Persistir auditoria em diagnoses (best-effort)"] --> A11
20  A11["Responder DiagnoseResponse"] --> F((( )))

```

Listing A.17 – AT-06 – *guardrail* e geração da prescrição

```

1 flowchart TD
2   I(( )) --> A1["Obter o nome de exibicao da familia"]
3   A1 --> D1{"Existe chunk com esta familia?"}
4   D1 -->|nao| N1["Formatar aviso + sugestao de registro"]
5   N1 --> FE((( )))
6   D1 -->|sim| A2["Montar a consulta: familia + sintomas do evento"]
7   A2 --> A3["Gerar embedding (RETRIEVAL_QUERY)"]
8   A3 --> A4["Recuperar top-k=6 filtrando pela familia"]
9   A4 --> A5["Montar prompt de contexto fechado, trechos numerados"]
10  A5 --> A6["Gerar com temperature=0.2 e instrucao restritiva"]
11  A6 --> A7["Extrair citacoes dos METADADOS dos trechos"]
12  A7 --> F((( )))

```

Listing A.18 – AT-07 – conversa com o agente

```

1 flowchart TD

```

```

2      I(( )) --> D0{"Agente disponivel?"}
3      D0 -->|nao| FB["Degradar para RAG de um passo"]
4      FB --> A5
5      D0 -->|sim| A1["Reiniciar o coletor de citacoes"]
6      A1 --> A2["Recuperar a memoria da sessao (schema adk)"]
7      A2 --> A3["Modelo decide a proxima ferramenta"]
8      A3 --> D1{"Qual ferramenta?"}
9      D1 --> T1["consultar_historico"]
10     D1 --> T2["buscar_documentos (aplica o guardrail)"]
11     D1 --> T3["diagnosticar_evento"]
12     D1 --> T4["cobertura_documental"]
13     T1 --> D2
14     T2 --> D2
15     T3 --> D2
16     T4 --> D2{"Precisa de outra ferramenta?"}
17     D2 -->|sim| A3
18     D2 -->|nao| A4["Gerar a resposta final em contexto fechado"]
19     A4 --> A5["Devolver texto + citacoes + selo do modo"]
20     A5 --> F((( )))

```

Listing A.19 – AT-08 – ingestão de telemetria industrial

```

1  flowchart TD
2      I(( )) --> P1["Gateway publica em sensors/{maq}/telemetry (QoS 1)"]
3      P1 --> W1["Worker extrai o identificador da maquina do topico"]
4      W1 --> D1{"JSON e esquema validos?"}
5      D1 -->|nao| DLQ["Compore registro de rejeicao: motivo + payload"]
6      DLQ --> B2["Publicar em .../dlq (QoS 1)"]
7      B2 --> F1((( )))
8      D1 -->|sim| W2["Canonicalizar a anotacao, se houver"]
9      W2 --> W3["merge idempotente em events"]
10     W3 --> W4["Chamar POST /diagnose"]
11     W4 --> D2{"E falha?"}
12     D2 -->|sim| B1["Publicar em .../alerts (QoS 1)"]
13     B1 --> F2((( )))
14     D2 -->|nao| F3((( )))

```

Listing A.20 – AT-09 e AT-10 – cadastro de documento e indexação automática

```

1  flowchart TD
2      subgraph AT09["AT-09 – cadastro pela interface"]
3          I1(( )) --> C1["Usuario abre a secao Documentos"]
4          C1 --> C2["Interface lista documentos e cobertura"]
5          C2 --> C3["Usuario anexa PDF, titulo e familias"]
6          C3 --> CD1{"Campos preenchidos?"}
7          CD1 -->|nao| CE1["Erro no formulario"] --> C3
8          CD1 -->|sim| C4["POST /documents (multipart)"]
9          C4 --> CD2{"Indexacao bem-sucedida?"}
10         CD2 -->|nao| CE2["Exibir codigo e detalhe do erro"]
11         CD2 -->|sim| C5["Exibir chunks e familias atendidas"]
12         C5 --> C6["Invalidar o cache da interface"]
13         C6 --> C7["Familia coberta na proxima requisicao"] --> F1((( )))
14     end
15
16     subgraph AT10["AT-10 – indexacao automatica no primeiro boot"]
17         I2(( )) --> BD0{"RAG_BOOTSTRAP habilitado?"}
18         BD0 -->|nao| BN0["Nao fazer nada"] --> F2((( )))
19         BD0 -->|sim| BD1{"Indice ja contem familias?"}

```

```

20 BD1 -->|sim| BN1["Log: bootstrap dispensado"] --> F2
21 BD1 -->|nao| BD2{"Indice inspecionavel?"}
22 BD2 -->|nao| BN2["Advertir e cancelar"] --> F2
23 BD2 -->|sim| B1["Iniciar thread daemon separada"]
24 B1 --> B2["Startup conclui: healthcheck passa"]
25 B1 --> B3["Indexar cada PDF da cobertura inicial (AT-04)"]
26 B3 --> B4["Registrar chunks por documento"] --> F3((( )))
27 end

```

Listing A.21 – AT-11, AT-12 e AT-13 – painel, integração contínua e implantação

```

1 flowchart TD
2   subgraph AT11["AT-11 - consulta a um painel"]
3     I1(( )) --> P1["Usuario escolhe secao e subpainel"]
4     P1 --> PD0{"API acessivel?"}
5     PD0 -->|nao| PE0["Interromper com o endereco configurado"] --> F1((( )))
6     PD0 -->|sim| P2["Requisitar APENAS os dados do painel ativo"]
7     P2 --> PD1{"Agregacao disponivel?"}
8     PD1 -->|nao| PE1["Aviso no painel; demais graficos seguem"] --> P3
9     PD1 -->|sim| P3["Aplicar filtros locais"]
10    P3 --> P4["Aplicar a paleta validada e o chrome comum"]
11    P4 --> P5["Renderizar com legenda explicativa"] --> F2((( )))
12  end
13
14  subgraph AT12["AT-12 - integracao continua"]
15    I2(( )) --> C1["push em main ou pull request"]
16    C1 --> C2["Lint: ruff check + format --check"]
17    C2 --> CD1{"Passou?"}
18    CD1 -->|nao| CE["Interromper e reportar no PR"] --> F3((( )))
19    CD1 -->|sim| C3["Testes: 63, Postgres de servico, LLM fake"]
20    C3 --> CD2{"Passou?"}
21    CD2 -->|nao| CE
22    CD2 -->|sim| C4["Build: docker build da imagem unica"]
23    C4 --> CD3{"Passou?"}
24    CD3 -->|nao| CE
25    CD3 -->|sim| C5["Publicacao autorizada"] --> F4((( )))
26  end
27
28  subgraph AT13["AT-13 - provisionamento e implantacao"]
29    I3(( )) --> D1["Instalar SDK de IaC e autenticar"]
30    D1 --> D2["Inicializar o projeto"]
31    D2 --> D3["Planejar: revisar o diff da topologia"]
32    D3 --> DD1{"Diff aceitavel?"}
33    DD1 -->|nao| DE1["Ajustar a declaracao"] --> D3
34    DD1 -->|sim| D4["Aplicar: banco, servicos, volume, grupos"]
35    D4 --> D5["Definir segredos por entrada padrao"]
36    D5 --> D6["Expor dominios HTTPS e proxy TCP"]
37    D6 --> D7["Carregar o historico da maquina local"]
38    D7 --> D8["Aguardar a indexacao automatica (AT-10)"]
39    D8 --> DD2{"Verificacao pos-implantacao passou?"}
40    DD2 -->|nao| DE2["Inspecionar logs e variaveis efetivas"] --> D8
41    DD2 -->|sim| D9["Sistema operacional em nuvem"] --> F5((( )))
42  end

```

Listing A.22 – AT-14 – ciclo de tratamento de lacuna documental

```

1 flowchart TD
2   I(( )) --> P1["Planta: vibracao anomala detectada"]

```



```

3      P1 --> S1["Sistema: familia prevista + ocorrencias semelhantes"]
4      S1 --> D1{"Documentada?"}
5      D1 -->|sim| S2["Prescricao com citacao de doc, secao e pagina"]
6      S2 --> T1["Tecnico executa o procedimento"]
7      T1 --> T2["Tecnico registra conclusao e valida critérios"]
8      T2 --> F1((( )))
9      D1 -->|nao| S3["Aviso + sugestao de registro"]
10     S3 --> E1["Engenheiro prioriza a lacuna no painel"]
11     E1 --> E2["Engenheiro redige o procedimento tecnico"]
12     E2 --> E3["Engenheiro cadastra o PDF (AT-09)"]
13     E3 --> E4["Familia passa a ser coberta"]
14     E4 -->|o proximo diagnostico ja prescreve| S1

```

A.13 Infraestrutura (Figuras 14.1 e 14.2)

Listing A.23 – Implantação local e em nuvem

```

1  flowchart TB
2      subgraph LOCAL["Estacao de trabalho - docker compose"]
3          LUI["ui :8501<br/>Streamlit"] --> LAPI["api :8000<br/>Uvicorn"]
4          LWK["worker<br/>paho-mqtt"] --> LAPI
5          LMQ["mosquitto :1883"] --> LWK
6          LAPI --> LPG["postgres :5432"]
7          LWK --> LPG
8          LAPI --> LVOL["./data/chroma"]
9          LAPI --> LART["./artifacts"]
10         LPG --> LPGD["volume pgdata"]
11     end
12
13     subgraph CLOUD["Projeto em nuvem - regioao us-east4"]
14         CUI["ui<br/>HTTPS publico"] -->|rede privada| CAPI["api<br/>HTTPS publico"]
15         CWK["worker<br/>sem exposicao"] -->|rede privada| CAPI
16         CMQ["mosquitto<br/>proxy TCP :1883"] -->|rede privada| CWK
17         CAPI -->|rede privada| CPG["postgres<br/>plugin gerenciado"]
18         CWK -->|rede privada| CPG
19         CAPI --> CVOL["volume chroma<br/>1 GB, /app/data/chroma"]
20     end
21
22     NET(("Internet")) -->|HTTPS| CUI
23     NET -->|HTTPS| CAPI
24     NET -->|TCP| CMQ
25     CAPI -->|egresso HTTPS| VX["Vertex AI - Gemini"]
26     GH["Repositorio Git<br/>push em main"] -.->|build apos CI| CUI
27     GH -.->|build apos CI| CAPI
28     GH -.->|build apos CI| CWK

```

A.14 Agente e ferramentas (Figura 9.2)

Listing A.24 – Agente de conversa e suas quatro ferramentas

```

1  flowchart LR
2      UI["Streamlit Chat<br/>(session_id por aba)"] --> EP["POST /api/v1/chat"]

```

```
3 EP -->|ADK disponível| AG["LlmAgent (gemini-3.6-flash)<br/>Runner + sessoes no Postgres"]
4 EP -->|fallback automatico| RAGQ["RAG de um passo<br/>(rag_service.chat)"]
5 AG --> T1["consultar_historico<br/>(SQL no Postgres)"]
6 AG --> T2["buscar_documentos<br/>(guardrail + ChromaDB)"]
7 AG --> T3["diagnosticar_evento<br/>(DiagnosisEngine)"]
8 AG --> T4["cobertura_documental"]
9 classDef guard fill:#fdede7,stroke:#eb6834;
10 class T2 guard;
```

A.15 Integração contínua (Figura 13.12)

Listing A.25 – Fluxo de integração contínua, forma compacta

```
1 flowchart LR
2   PUSH["push / PR"] --> LINT["ruff check + format"]
3   LINT --> TEST["pytest - 63 testes<br/>(Postgres de servico,<br/>LLM/embeddings mockados)"]
4   TEST --> BUILD["docker build"]
5   BUILD --> DEPLOY["publicacao autorizada"]
```

Apêndice B

Dicionário completo de *features*

O vetor de entrada do modelo tem 23 dimensões, na ordem exata abaixo. A **ordem é significativa**: ela é a ordem esperada pelos artefatos treinados. Reordenar a lista sem retreinar produz previsões silenciosamente erradas.

B.1 Grandezas brutas (18)

#	Nome	Unidade	Significado físico
1	<code>z_rms_velocity_mm_s</code>	mm/s	Velocidade eficaz de vibração no eixo Z (axial).
2	<code>x_rms_velocity_mm_s</code>	mm/s	Velocidade eficaz no eixo X (radial). É o indicador normativo de severidade global (INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 1995).
3	<code>temperature_c</code>	°C	Temperatura medida no ponto do sensor. <i>Feature</i> mais influente do classificador — e a mais suspeita de funcionar como carimbo de campanha (Seção 8.6).
4	<code>z_peak_acceleration_g</code>	<i>g</i>	Aceleração de pico no eixo Z. Sensível a impactos.
5	<code>x_peak_acceleration_g</code>	<i>g</i>	Aceleração de pico no eixo X.
6	<code>z_peak_vel_comp_freq_hz</code>	Hz	Frequência da componente dominante de velocidade em Z.
7	<code>x_peak_vel_comp_freq_hz</code>	Hz	Idem, eixo X. Base da <i>feature</i> derivada de ordem. 61 Hz responde por 48,7 % dos registros — distribuição quase degenerada.
8	<code>z_rms_acceleration_g</code>	<i>g</i>	Aceleração eficaz em Z (banda completa).

#	Nome	Unidade	Significado físico
9	<code>x_rms_acceleration_g</code>	<i>g</i>	Aceleração eficaz em X (banda completa).
10	<code>z_kurtosis</code>	—	Curtose do sinal em Z. Teoricamente indicativa de impactos (Randall; Antoni, 2011); neste conjunto de dados varia entre 2,65 e 2,75 em <i>todas</i> as famílias.
11	<code>x_kurtosis</code>	—	Curtose em X, com a mesma limitação.
12	<code>z_crest_factor</code>	—	Razão pico/eficaz em Z. Varia entre 3,73 e 3,85 em <i>todas</i> as famílias.
13	<code>x_crest_factor</code>	—	Fator de crista em X.
14	<code>z_peak_velocity_mm_s</code>	mm/s	Velocidade de pico em Z.
15	<code>x_peak_velocity_mm_s</code>	mm/s	Velocidade de pico em X. Colinear com a velocidade eficaz em X — razão <i>F</i> idêntica (1.193,6), porque o fator de crista é quase constante.
16	<code>z_high_freq_rms_accel_g</code>	<i>g</i>	Aceleração eficaz de alta frequência em Z.
17	<code>x_high_freq_rms_accel_gg</code>		Idem, eixo X. Base da razão de alta/baixa frequência.
18	<code>rpm</code>	rpm	Rotação do eixo. Variável de <i>condição operacional</i> : estratificar por ela é obrigatório em qualquer leitura de severidade (Seção 8.8.1).

B.2 *Features* derivadas (5)

#	Nome	Definição	Motivação
19	<code>x_peak_order</code>	$\min\left(\frac{f_{\text{pico},X}}{\max(\text{rpm}/60, 10^{-6})}, 0,01\right)$	Ordem do pico no eixo X. Os manuais descrevem assinaturas em múltiplos da rotação, não em Hz absoluto.
20	<code>z_peak_order</code>	Idem, eixo Z	Ordem do pico no eixo axial.

21	<code>radial_axial_ratio</code>	$\frac{v_{\text{RMS},X}}{v_{\text{RMS},Z} + 10^{-6}}$	Desbalanceamento é predominantemente radial; desalinhamento e rotor inclinado produzem componente axial apreciável.
22	<code>hf_lf_ratio_x</code>	$\frac{a_{\text{HFRMS},X}}{a_{\text{RMS},X} + 10^{-6}}$	Defeitos localizados em rolamentos excitam preferencialmente alta frequência.
23	<code>hf_lf_ratio_z</code>	Idem, eixo Z	Mesma motivação, eixo axial.

B.3 Colunas descartadas

Coluna descartada	Motivo
Velocidades em in/s	Transformação linear exata das contrapartes em mm/s (fator 25,4). Inclusão produziria colinearidade perfeita sem acrescentar informação.
Temperatura em °F	Transformação afim exata da temperatura em °C.
<code>fault</code> (rótulo bruto)	É o <i>alvo</i> , não <i>feature</i> . Preservado em <code>events.raw_fault</code> para auditoria e para identificar a campanha de coleta na análise de vazamento.
<code>id</code> e <code>created_at</code>	Identificador e instante não são sintomas mecânicos. Incluí-los seria vazamento explícito, dado que o instante identifica a campanha.

B.4 Poder discriminativo observado

Razão entre a variância inter-famílias e a variância intra-família (estatística no espírito do F de análise de variância), calculada sobre as famílias com ao menos 30 eventos.

Feature	Razão F	Leitura
velocidade RMS X	1.193,6	Melhor discriminador do conjunto.
velocidade de pico X	1.193,6	Colinear com a anterior.
frequência de pico X	1.106,8	Alto valor, mas distribuição quase degenerada.

temperatura	885,3	Discrimina – e também carimba a campanha.
rotação	208,3	Condição operacional, não sintoma.
curtose Z	189,2	Baixo: contraria a previsão dos manuais.
aceleração de pico Z	188,2	Baixo.

Apêndice C

Taxonomia canônica de falhas

C.1 As 17 famílias

Nome canônico	É falha	Descrição de exibição
Famílias de falha (12)		
<code>rolamento_inner</code>	✓	Defeito na pista interna do rolamento (BPFI)
<code>rolamento_outer</code>	✓	Defeito na pista externa do rolamento (BPFO)
<code>rolamento_ball</code>	✓	Defeito nos elementos rolantes (BSF)
<code>rolamento_combinatio</code>	✓	Defeito combinado no rolamento
<code>desbalanceamento</code>	✓	Desbalanceamento do rotor
<code>desalinhamento</code>	✓	Desalinhamento de eixos
<code>correia</code>	✓	Falha na transmissão por correia
<code>polia</code>	✓	Falha na polia
<code>cocked_rotor</code>	✓	Rotor inclinado (<i>cocked rotor</i>)
<code>eccentric_rotor</code>	✓	Rotor excêntrico
<code>ventoinha</code>	✓	Falha na ventoinha
<code>falta_fase</code>	✓	Falta de fase no motor
Estados operacionais (5) – não geram prescrição		
<code>normal</code>	—	Operação normal
<code>baseline</code>	—	Coleta de linha de base
<code>teste</code>	—	Registro de teste
<code>acelerando</code>	—	Máquina acelerando (transiente)
<code>motor_desligado</code>	—	Motor desligado

C.2 Regras de canonicalização

Avaliadas **na ordem**; a primeira que casa vence. Regras de falha precedem regras de estado operacional.

#	Expressão	Família	Rótulos reais cobertos
1	inner	rolamento_inner	rolamento_inner, rolamento_inner_2, new_rolamento_inner_0, rolamento_inner_carga
2	outer	rolamento_outer	rolamento_outer, rolamento_outer_novo_teste
3	ball	rolamento_ball	rolamento_ball e variantes de campanha
4	comb	rolamento_combinat	rolamento_combination e variantes
5	desb abal abanc balanc	desbalanceamento	desbalanceado, ddesbalanceado, desbanlanceado, desabalanceado, desabanceado — cinco grafias do mesmo defeito
6	desalinh	desalinhamento	desalinhado, desalinhado_2
7	falta_fase	falta_fase	falta_fase e variantes
8	eccentric	eccentric_rotor	eccentric_rotor e variantes
9	ventoinha	ventoinha	ventoinha e variantes
10	correia	correia	correia e variantes
11	polia	polia	polia e variantes
12	cock	cocked_rotor	cocked_rotor, cocked_adxl_0, cockecocked_adxl_0, cocked_rotor_pos_2

Estados operacionais — avaliados por último

13	baseline	baseline	baseline e variantes
14	acelerando	acelerando	acelerando
15	desligado	motor_desligado	motor_desligado, mortor_desligado_novo
16	norm	normal	normal, normla_carga_3_3
17	^(new_)?tes	teste	teste, new_teste, new_tes

A ordem não é acidental. A regra 2 (**outer**) precede a regra 17 (**teste**) para que **rolamento_outer_novo_teste** caia em **rolamento_outer**. Inverter a ordem classificaria um defeito real de rolamento como registro de teste – e o sistema deixaria de prescrever correção para ele.

C.3 Distribuição de ocorrências e cobertura

Família	Ocorrências	Cobertura	F1 no <i>holdout</i> por sessão
eccentric_rotor	16.497	sem documento	0,037
ventoinha	12.299	sem documento	0,047
falta_fase	800	sem documento	0,940
rolamento_inner	—	Doc1 .pdf	0,019
rolamento_outer	—	Doc1 .pdf	0,130
rolamento_ball	—	Doc1 .pdf	0,054
rolamento_combination	—	Doc1 .pdf	0,070
desalinhamento	—	Doc2 .pdf	0,261
desbalanceamento	—	Doc3 .pdf	0,070
correia	—	Doc4 .pdf	0,282
polia	—	Doc5 .pdf	0,254
cocked_rotor	14.275	Doc6 .pdf	0,005
motor_desligado	—	(estado)	0,546
normal	—	(estado)	0,076
teste	—	(estado)	0,135

NOTA

A leitura conjunta das duas últimas colunas é o argumento central da Seção 8.6: a família que o modelo reconhece melhor (**falta_fase**, F1 de 0,940) é justamente uma das que *não* têm procedimento documentado – e as famílias de rolamento, plenamente documentadas, são as que menos generalizam. Diagnóstico e documentação não estão correlacionados, o que reforça que o teto de generalização é do dado, não do acervo.

Apêndice D

Payloads e respostas de referência

Este apêndice reúne os *payloads* usados nos critérios de aceite da Seção 3.8.

D.1 CA-1 — Evento do enunciado (falha documentada)

Listing D.1 – Requisição

```
1 curl -X POST http://localhost:8000/api/v1/diagnose \  
2     -H 'Content-Type: application/json' -d @evento.json
```

Listing D.2 – evento.json

```
1 {  
2     "id": 114387,  
3     "created_at": "2026-06-01 21:32:53.911176+00:00",  
4     "z_rms_velocity_mm_s": 1.517,  
5     "temperature_c": 24.69,  
6     "x_rms_velocity_mm_s": 2.0,  
7     "z_peak_acceleration_g": 0.484,  
8     "x_peak_acceleration_g": 0.631,  
9     "z_peak_vel_comp_freq_hz": 61.0,  
10    "x_peak_vel_comp_freq_hz": 61.0,  
11    "z_rms_acceleration_g": 0.09,  
12    "x_rms_acceleration_g": 0.114,  
13    "z_kurtosis": 2.392,  
14    "x_kurtosis": 2.77,  
15    "z_crest_factor": 3.747,  
16    "x_crest_factor": 4.269,  
17    "z_peak_velocity_mm_s": 2.146,  
18    "x_peak_velocity_mm_s": 2.829,  
19    "z_high_freq_rms_accel_g": 0.129,  
20    "x_high_freq_rms_accel_g": 0.147,  
21    "fault": "cocked_rotor_2",  
22    "rpm": 1000.0  
23 }
```

Resultado observado: família `cocked_rotor` com probabilidade de aproximadamente 92 %, 14.275 ocorrências semelhantes com distribuição temporal e frequência diária, e instruções de correção citando o `Doc6.pdf`.

D.2 CA-2 – Falha sem documento

Basta submeter um evento cuja família prevista seja `ventoinha`, `eccentric_rotor` ou `falta_fase`. A resposta traz `documented=false`, `prescription=null` e o campo `suggestion` preenchido – e nenhuma chamada ao serviço de linguagem é feita.

Listing D.3 – Verificação por linha de comando

```
1 curl -s -X POST http://localhost:8000/api/v1/diagnose \  
2   -H 'Content-Type: application/json' -d @evento-ventoinha.json \  
3   | jq '{fault: .predicted_fault, documented, prescription, suggestion}'
```

D.3 CA-3 – Cobertura após cadastro

Listing D.4 – Ciclo completo: lacuna, cadastro e nova cobertura

```
1 # 1. Antes: a familia nao esta coberta  
2 curl -s http://localhost:8000/api/v1/health | jq '.documented_families'  
3  
4 # 2. Cadastrar o procedimento  
5 curl -X POST http://localhost:8000/api/v1/documents \  
6   -F 'file=@procedimento-ventoinha.pdf' \  
7   -F 'title=Procedimento para Correcao de Falhas em Ventoinhas' \  
8   -F 'families=ventoinha'  
9  
10 # 3. Depois: a familia aparece na cobertura  
11 curl -s http://localhost:8000/api/v1/health | jq '.documented_families'  
12  
13 # 4. O mesmo evento agora recebe prescricao  
14 curl -s -X POST http://localhost:8000/api/v1/diagnose \  
15   -H 'Content-Type: application/json' -d @evento-ventoinha.json \  
16   | jq '.documented, .prescription.citations'
```

D.4 CA-5 – Payload inválido e fila de rejeição

Listing D.5 – Demonstração da fila de rejeição

```
1 # Terminal 1: observar as filas de saida  
2 mosquito_sub -h localhost -t 'sensors/+/alerts' -t 'sensors/+/dlq' -v  
3  
4 # Terminal 2: publicar um payload invalido  
5 python scripts/simulate_sensor.py --invalid
```

Listing D.6 – Mensagem observada na fila de rejeição (abreviada)

```

1 {
2   "reason": "1 validation error for SensorEvent\n  temperature_c\n    Field
3         required [type=missing, input_value={...}, input_type=dict]",
4   "payload": "{\n  \"id\": 999999, \"x_rms_velocity_mm_s\": 2.0, ...}",
5   "ts": "2026-08-23T14:07:21.418293+00:00"
6 }

```

D.5 Exemplos de perguntas para a conversa

Pergunta	Ferramenta acionada	Comportamento esperado
"Quantas ocorrências de <code>cocked_rotor</code> nos últimos 30 dias?"	<code>consultar_historico</code>	Números vindos de SQL sobre o histórico, com a janela ancorada no evento mais recente do banco.
"Como corrijo um desalinhamento?"	<code>buscar_documentos</code>	Procedimento com citações de documento, seção e página.
"Como corrijo a ventoinha?"	<code>buscar_documentos</code>	<i>Guardrail</i> : aviso de falha não documentada e sugestão de registro. Nenhum procedimento é descrito.
"Quais falhas têm procedimento documentado?"	<code>cobertura_documental</code>	Lista de famílias documentadas e sem documento, com os documentos cadastrados.
Colar o JSON de um evento no campo de conversa	<code>diagnosticar_evento</code>	Diagnóstico completo explicado em linguagem clara.

Apêndice E

Configuração e variáveis de ambiente

Toda a configuração vem de variáveis de ambiente com valores padrão sensatos, conforme o padrão de doze fatores ([Wiggins, 2017](#)). Não existe configuração em código, nem arquivo de configuração específico por ambiente.

E.1 Variáveis de ambiente

Variável	Valor padrão	Função
Banco de dados		
<code>DATABASE_URL</code>	<code>postgresql+psycopg2://fiesc:fiesc@localhost:5432/fiesc</code>	URL de conexão. Aceita também os formatos <code>postgresql://</code> e <code>postgres://</code> entregues por provedores gerenciados — a normalização é automática.
Serviço de linguagem		
<code>GOOGLE_CLOUD_PROJECT</code>	(vazio)	Projeto de nuvem. Vazio desabilita o agente e o serviço de linguagem, sem impedir o restante do sistema.
<code>GOOGLE_CLOUD_LOCATION</code>	<code>global</code>	Endpoint regional. Os modelos usados são servidos pelo endpoint global; regiões específicas retornam erro de modelo inexistente.
<code>GOOGLE_APPLICATION_CREDENTIALS</code>	(vazio)	Caminho do arquivo de credencial — uso em desenvolvimento local.
<code>GOOGLE_CREDENTIALS_B64</code>	(vazio)	Mesma credencial em base64 — uso em implantação. Materializada em arquivo temporário com permissão <code>0600</code> .

Variável	Valor padrão	Função
GEMINI_CHAT_MODEL	gemini-3.6-flash	Modelo de geração. É parâmetro: trocar de modelo não exige alterar código.
GEMINI_EMBEDDING_MODEL	gemini-embedding-1.5	Modelo de <i>embeddings</i> .
EMBEDDING_DIM	768	Dimensionalidade do vetor. Alterar exige reindexar a base documental — vetores de dimensões distintas não são comparáveis.
Recuperação documental		
CHROMA_DIR	./data/chroma	Diretório do índice vetorial. Em nuvem, aponta para o volume persistente.
RAG_TOP_K	6	Número de trechos recuperados por consulta.
RAG_BOOTSTRAP	false	Habilita a indexação automática quando o índice está vazio. Verdadeiro em nuvem, falso em desenvolvimento.
DOCS_DIR	./documentos	Diretório dos PDFs da carga inicial.
Ingestão industrial		
MQTT_HOST	localhost	Endereço do <i>broker</i> . Em nuvem, o domínio privado do serviço.
MQTT_PORT	1883	Porta do <i>broker</i> .
MQTT_TELEMETRY_TOPIC	sensors/+/telemetry	Tópico assinado pelo <i>worker</i> , com curinga de máquina.
MQTT_USERNAME	(vazio)	Usuário do <i>broker</i> . Vazio implica acesso anônimo — aceitável apenas sem exposição pública.
MQTT_PASSWORD	(vazio)	Senha do <i>broker</i> .
Serviços		
API_URL	http://localhost:8080	Endereço da API usado pela interface e pelo <i>worker</i> . Em nuvem, o domínio <i>privado</i> .
ARTIFACTS_DIR	./artifacts	Diretório dos artefatos de aprendizado de máquina.

E.2 Constantes de código

Estas não são configuráveis por ambiente — alterá-las é decisão de projeto que exige retreino ou reindexação.

Constante	Valor	Onde	Consequência de alterar
<code>KNN_NEIGHBORS</code>	25	<code>ml/train.py</code>	Exige retreino; altera a granularidade da concordância.
<code>SESSION_TEST_FRAC</code>	0,2	<code>ml/train.py</code>	Altera o protocolo principal de avaliação.
<code>RANDOM_STATE</code>	42	<code>ml/train.py</code>	Quebra a reprodutibilidade das métricas publicadas.
<code>CHUNK_SIZE</code>	1200	<code>rag/chunking.py</code>	Exige reindexação de toda a base documental.
<code>CHUNK_OVERLAP</code>	150	<code>rag/chunking.py</code>	Idem.
<code>MIN_SESSION_EVENTS</code>	30	<code>api/analytics.py</code>	Altera quais campanhas entram nas análises de assinatura e de vazamento.
<code>COLLECTION</code>	<code>manuals</code>	<code>rag/store.py</code>	Índice existente ficaria inacessível.

E.3 Arquivo de configuração local

Listing E.1 – `.env` de desenvolvimento

```

1  # --- Banco de dados (docker compose sobe o Postgres com estes valores) ---
2  DATABASE_URL=postgresql+psycopg2://fiesc:fiesc@localhost:5432/fiesc
3
4  # --- Servico de linguagem ---
5  # Autenticacao: `gcloud auth application-default login` OU aponte
6  # GOOGLE_APPLICATION_CREDENTIALS para o JSON da service account.
7  GOOGLE_CLOUD_PROJECT=meu-projeto-gcp
8  GOOGLE_CLOUD_LOCATION=global
9  # GOOGLE_APPLICATION_CREDENTIALS=./credentials/service-account.json
10 GEMINI_CHAT_MODEL=gemini-3.6-flash
11 GEMINI_EMBEDDING_MODEL=gemini-embedding-2
12 EMBEDDING_DIM=768
13
14 # --- Recuperacao documental ---
15 CHROMA_DIR=./data/chroma
16 RAG_TOP_K=6
17
18 # --- Ingestao industrial ---
19 MQTT_HOST=localhost
20 MQTT_PORT=1883
21 MQTT_TELEMETRY_TOPIC=sensors/+/telemetry

```

```
22  
23 # --- Servicos ---  
24 API_URL=http://localhost:8000  
25 ARTIFACTS_DIR=./artifacts
```

O arquivo `.env` **não é versionado**. O repositório traz apenas `.env.example`, com valores de exemplo e sem segredo algum. A credencial de nuvem, quando usada como arquivo, fica em diretório ignorado pelo controle de versão.

Apêndice F

Guia de operação

F.1 Instalação do zero

Listing F.1 – Instalação e primeira execução completa

```
1 # 1. Ambiente Python (requer Python 3.12+)
2 python -m venv .env
3 .env\Scripts\activate           # Windows
4 # source .env/bin/activate      # Linux / macOS
5 pip install -e .[dev]
6
7 # 2. Configuracao
8 copy .env.example .env         # e preencha GOOGLE_CLOUD_PROJECT
9 gcloud auth application-default login
10
11 # 3. Infraestrutura local (banco + broker)
12 docker compose up -d postgres mosquito
13
14 # 4. Carga do historico: 166.796 eventos, 151 rotulos -> 17 familias
15 python scripts/ingest_data.py --csv documentos/banner.csv
16
17 # 5. Treino do motor de diagnostico (scaler + KNN + LightGBM -> artifacts/)
18 python -m src.ml.train
19
20 # 6. Base documental: Doc1..Doc6 -> ChromaDB (requer credenciais)
21 python scripts/ingest_docs.py --docs-dir documentos
22
23 # 7. Servicos
24 uvicorn src.api.main:app --port 8000      # API -> http://localhost:8000/docs
25 streamlit run src/ui/app.py              # UI -> http://localhost:8501
26 python -m src.ingestion.mqtt_worker      # worker MQTT (opcional)
```

Alternativa integralmente em contêineres, após os passos 4 a 6: `docker compose up -build`.

F.2 Operação diária

Objetivo	Comando
----------	---------

Verificar saúde do serviço	<code>curl -s localhost:8000/api/v1/health jq</code>
Diagnosticar um evento	<code>curl -X POST localhost:8000/api/v1/diagnose -H 'Content-Type: application/json' -d @evento.json</code>
Consultar cobertura documental	<code>curl -s localhost:8000/api/v1/documents jq '.documented_families'</code>
Cadastrar procedimento	<code>curl -X POST ../documents -F 'file=@doc.pdf' -F 'title=...' -F 'families=...'</code>
Reindexar a base documental	<code>python scripts/ingest_docs.py</code>
Retreinar o modelo	<code>python -m src.ml.train</code>
Simular telemetria	<code>python scripts/simulate_sensor.py -family cocked_rotor -limit 5 -rate 2</code>
Demonstrar a fila de rejeição	<code>python scripts/simulate_sensor.py -invalid</code>
Observar as filas de saída	<code>mosquitto_sub -h localhost -t 'sensors/+/alerts' -t 'sensors/+/dlq' -v</code>
Executar a suíte de testes	<code>pytest</code>
Verificar estilo de código	<code>ruff check src tests scripts</code>

F.3 Diagnóstico de problemas

Sintoma	Causa provável	Ação
<code>artifacts_loaded: false</code> em <code>/health</code>	Treino não executado ou diretório de artefatos incorreto	Executar <code>python -m src.ml.train</code> e conferir <code>ARTIFACTS_DIR</code> .
<code>documented_families</code> vazio	Base documental não indexada, ou indexação inicial em curso	Executar <code>ingest_docs.py</code> ; em nuvem, acompanhar o <i>log</i> de indexação. Lista vazia no primeiro minuto após implantação é estado esperado.
<code>503</code> no <i>endpoint</i> de conversa	Credenciais de nuvem ausentes ou expiradas	A própria mensagem indica as variáveis e o comando de autenticação. O diagnóstico continua funcionando sem elas.
Selo de “recuperação direta” na conversa	Agente indisponível (biblioteca ausente ou projeto não configurado)	Comportamento previsto (RNF-12). Conferir o <i>log</i> de inicialização, que registra o motivo exato.

Sintoma	Causa provável	Ação
Interface exibe “API indisponível”	API não iniciada ou endereço incorreto	Conferir <code>API_URL</code> ; em nuvem deve ser o domínio <i>privado</i> , não o público.
ETL interrompido com rótulo desconhecido	Rótulo novo no conjunto de dados	Comportamento desejado. Acrescentar a regra em <code>src/etl/label_map.yaml</code> e reexecutar — nunca contornar removendo a linha.
Eventos MQTT não aparecem no painel	<i>Worker</i> desconectado, ou tópico divergente	Conferir o <i>log</i> do <i>worker</i> (registra a conexão e a assinatura) e a variável de tópico.
Mensagens acumulando na fila de rejeição	<i>Payload</i> do <i>gateway</i> fora do esquema	Ler o campo de motivo da mensagem: ele nomeia exatamente o campo inconsistente.
Testes de análise pulados	PostgreSQL indisponível	Comportamento previsto: essas agregações exigem recursos estatísticos ausentes no SQLite. Subir o banco os habilita.
Índice vetorial perdido após implantação	Volume não montado ou região do volume alterada	Conferir o ponto de montagem e jamais omitir a região na declaração de infraestrutura — omiti-la é operação destrutiva.

F.4 Compilação deste documento

Listing F.2 – Compilação do documento LaTeX

```

1  cd latex
2
3  # Opcao 1 - automatica (recomendada)
4  latexmk -pdf main.tex
5
6  # Opcao 2 - manual, quatro passagens (indices, referencias e bibliografia)
7  pdflatex main
8  biber    main
9  pdflatex main
10 pdflatex main
11
12 # Opcao 3 - Windows PowerShell
13 .\compilar.ps1

```

Requisito

Observação

Distribuição T _E X	MiKTeX ou T _E X Live com pdflatex e biber .
Fontes	Pacotes roboto e roboto-mono (Roboto e Roboto Mono em formato Type1/OTF). O documento compila com pdfLaTeX puro — não exige XeLaTeX nem LuaLaTeX.
Pacotes principais	tikz e pgfplots (diagramas e gráficos), tcolorbox , booktabs , tabularx , ltablex , listings , titlesec , biblatex com estilo abnt .
Bibliografia	Estilo ABNT NBR 6023 (ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2018) via biblatex-abnt , com biber como processador.
Diagramas	Todos vetoriais em TikZ; nenhuma dependência de renderização externa. As fontes Mermaid equivalentes estão no Apêndice A .

Referências

ANSARI, F.; GLAWAR, R.; NEMETH, T. PriMa: a prescriptive maintenance model for cyber-physical production systems. **International Journal of Computer Integrated Manufacturing**, v. 32, n. 4–5, p. 482–503, 2019. DOI: [10.1080/0951192X.2019.1571236](https://doi.org/10.1080/0951192X.2019.1571236).

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 5462: confiabilidade e manutenibilidade – terminologia**. Rio de Janeiro, 1994.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023: informação e documentação – referências – elaboração**. Rio de Janeiro, 2018.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software architecture in practice**. 3. ed. Upper Saddle River: Addison-Wesley, 2012. ISBN 9780321815736.

BREIMAN, L. Random forests. **Machine Learning**, v. 45, n. 1, p. 5–32, 2001. DOI: [10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).

BROWN, S. **The C4 model for visualising software architecture**. 2026. Disponível em: <https://c4model.com>. Acesso em: 23 ago. 2026.

CHEN, T.; GUESTRIN, C. XGBoost: a scalable tree boosting system. *In*: PROCEEDINGS of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco: ACM, 2016. p. 785–794. DOI: [10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).

COHN, M. **Succeeding with agile: software development using Scrum**. Upper Saddle River: Addison-Wesley, 2009. ISBN 9780321579362.

COVER, T. M.; HART, P. E. Nearest neighbor pattern classification. **IEEE Transactions on Information Theory**, v. 13, n. 1, p. 21–27, 1967. DOI: [10.1109/TIT.1967.1053964](https://doi.org/10.1109/TIT.1967.1053964).

FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. **The Annals of Statistics**, v. 29, n. 5, p. 1189–1232, 2001. DOI: [10.1214/aos/1013203451](https://doi.org/10.1214/aos/1013203451).

GRINSZTAJN, L.; OYALLON, E.; VAROQUAUX, G. Why do tree-based models still outperform deep learning on typical tabular data? *In*: ADVANCES in Neural Information Processing Systems 35 (NeurIPS 2022) – Datasets and Benchmarks Track. [S. l.]: Curran Associates, 2022. p. 507–520.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 10816-1: mechanical vibration – evaluation of machine vibration by measurements on non-rotating parts – part 1: general guidelines**. Geneva, 1995.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 13381-1: condition monitoring and diagnostics of machines – prognostics – part 1: general guidelines**. Geneva, 2015.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 17359: condition monitoring and diagnostics of machines – general guidelines**. Geneva, 2018.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 20816-1: mechanical vibration – measurement and evaluation of machine vibration – part 1: general guidelines**. Geneva, 2016.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010: systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models**. Geneva, 2011.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 29148: systems and software engineering – life cycle processes – requirements engineering**. Geneva, 2018.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 42010: systems and software engineering – architecture description**. Geneva, 2011.

JARDINE, A. K. S.; LIN, D.; BANJEVIC, D. A review on machinery diagnostics and prognostics implementing condition-based maintenance. **Mechanical Systems and Signal Processing**, v. 20, n. 7, p. 1483–1510, 2006. DOI: [10.1016/j.ymssp.2005.09.012](https://doi.org/10.1016/j.ymssp.2005.09.012).

Ji, Z. *et al.* Survey of hallucination in natural language generation. **ACM Computing Surveys**, v. 55, n. 12, p. 1–38, 2023. DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).

KAPOOR, S.; NARAYANAN, A. Leakage and the reproducibility crisis in machine-learning-based science. **Patterns**, v. 4, n. 9, p. 100804, 2023. DOI: [10.1016/j.patter.2023.100804](https://doi.org/10.1016/j.patter.2023.100804).

KAUFMAN, S. *et al.* Leakage in data mining: formulation, detection, and avoidance. **ACM Transactions on Knowledge Discovery from Data**, v. 6, n. 4, p. 1–21, 2012. DOI: [10.1145/2382577.2382579](https://doi.org/10.1145/2382577.2382579).

KE, G. *et al.* LightGBM: a highly efficient gradient boosting decision tree. *In*: ADVANCES in Neural Information Processing Systems 30 (NIPS 2017). Long Beach: Curran Associates, 2017. p. 3149–3157.

KRUCHTEN, P. The 4+1 view model of architecture. **IEEE Software**, v. 12, n. 6, p. 42–50, 1995. DOI: [10.1109/52.469759](https://doi.org/10.1109/52.469759).

KUSUPATI, A. *et al.* Matryoshka representation learning. *In*: ADVANCES in Neural Information Processing Systems 35 (NeurIPS 2022). [S. l.]: Curran Associates, 2022. p. 30233–30249.

LEWIS, P. *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. *In*: ADVANCES in Neural Information Processing Systems 33 (NeurIPS 2020). [S. l.]: Curran Associates, 2020. p. 9459–9474.

LUNDBERG, S. M.; LEE, S.-I. A unified approach to interpreting model predictions. *In: ADVANCES in Neural Information Processing Systems 30 (NIPS 2017)*. Long Beach: Curran Associates, 2017. p. 4765–4774.

MALKOV, Y. A.; YASHUNIN, D. A. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 42, n. 4, p. 824–836, 2020. DOI: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).

MUNZNER, T. **Visualization analysis and design**. Boca Raton: CRC Press, 2014. ISBN 9781466508910.

NYGARD, M. **Documenting architecture decisions**. 2011. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: 23 ago. 2026.

OASIS. **MQTT version 5.0: OASIS standard**. Burlington, 2019. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Acesso em: 23 ago. 2026.

OBJECT MANAGEMENT GROUP. **OMG unified modeling language (OMG UML), version 2.5.1**. Needham, 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1>. Acesso em: 23 ago. 2026.

OKABE, M.; ITO, K. **Color universal design (CUD): how to make figures and presentations that are friendly to colorblind people**. 2008. Disponível em: <https://jfly.uni-koeln.de/color/>. Acesso em: 23 ago. 2026.

RANDALL, R. B. **Vibration-based condition monitoring: industrial, aerospace and automotive applications**. Chichester: John Wiley & Sons, 2011. ISBN 9780470747858.

RANDALL, R. B.; ANTONI, J. Rolling element bearing diagnostics: a tutorial. **Mechanical Systems and Signal Processing**, v. 25, n. 2, p. 485–520, 2011. DOI: [10.1016/j.ymssp.2010.07.017](https://doi.org/10.1016/j.ymssp.2010.07.017).

REIMERS, N.; GUREVYCH, I. Sentence-BERT: sentence embeddings using Siamese BERT-networks. *In: PROCEEDINGS of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong: Association for Computational Linguistics, 2019. p. 3982–3992. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410).

ROBERTS, D. R. *et al.* Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. **Ecography**, v. 40, n. 8, p. 913–929, 2017. DOI: [10.1111/ecog.02881](https://doi.org/10.1111/ecog.02881).

SCULLEY, D. *et al.* Hidden technical debt in machine learning systems. *In: ADVANCES in Neural Information Processing Systems 28 (NIPS 2015)*. Montreal: Curran Associates, 2015. p. 2503–2511.

SHARMA, G.; WU, W.; DALAL, E. N. The CIEDE2000 color-difference formula: implementation notes, supplementary test data, and mathematical observations. **Color Research & Application**, v. 30, n. 1, p. 21–30, 2005. DOI: [10.1002/col.20070](https://doi.org/10.1002/col.20070).

SHWARTZ-ZIV, R.; ARMON, A. Tabular data: deep learning is not all you need. **Information Fusion**, v. 81, p. 84–90, 2022. DOI: [10.1016/j.inffus.2021.11.011](https://doi.org/10.1016/j.inffus.2021.11.011).

SOKOLOVA, M.; LAPALME, G. A systematic analysis of performance measures for classification tasks. **Information Processing & Management**, v. 45, n. 4, p. 427–437, 2009. DOI: [10.1016/j.ipm.2009.03.002](https://doi.org/10.1016/j.ipm.2009.03.002).

VARMA, S.; SIMON, R. Bias in error estimation when using cross-validation for model selection. **BMC Bioinformatics**, v. 7, p. 91, 2006. DOI: [10.1186/1471-2105-7-91](https://doi.org/10.1186/1471-2105-7-91).

WIGGINS, A. **The twelve-factor app**. 2017. Disponível em: <https://12factor.net>. Acesso em: 23 ago. 2026.

ZONTA, T. et al. Predictive maintenance in the Industry 4.0: a systematic literature review. **Computers & Industrial Engineering**, v. 150, p. 106889, 2020. DOI: [10.1016/j.cie.2020.106889](https://doi.org/10.1016/j.cie.2020.106889).